Válogatás az ELTE IK programtervező informatikus MSc szak

Algoritmusok tervezése és elemzése

és

Speciális algoritmusok

tantárgyainak anyagából

Lehetetlenségi tételek

Arrow lehetetlenségi tétele

Azt vizsgáljuk, hogy létezik-e olyan általános algoritmus, amely megnyugtató módon képes megoldani az egyéni preferenciák közösségivé való aggregálását.

Legyen $A = \{a_1, a_2, \ldots, a_m\}$ különböző alternatívák egy tetszőleges halmaza. Legyen továbbá $E = \{e_1, e_2, \ldots, e_n\}$ egyének egy halmaza, akik közül mindenkinek van egy teljes, reflexív, antiszimmetrikus és tranzitív preferencia-rendezése az alternatívák A halmazán. Jelölje ezeket a preferencia-rendezéseket $\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n$, itt $a_i \succ_t a_j$ azt jelenti, hogy az e_t egyén az a_i alternatívát preferálja az a_j alternatívával szemben. Megengedjük, hogy egy egyén egyformán preferáljon két alternatívát. Vezessük még be a $\boldsymbol{\succcurlyeq} = (\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n)$ jelölést, a $\boldsymbol{\succcurlyeq}$ rendezett sorozatot preferencia-profilnak fogjuk nevezni.

Egy az összes lehetséges $\boldsymbol{\succcurlyeq} = (\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n)$ preferencia-profilon értelmezett $\mathcal{R}$ függvényt, amely minden $\boldsymbol{\succcurlyeq}$ preferencia-profilhoz az A alternatíva halmaz egy teljes, reflexív, antiszimmetrikus és tranzitív $\trianglerighteq$ preferencia-rendezését rendeli, társadalmi jóléti függvénynek nevezünk.

Egy társadalmi jóléti függvényre viszonylag természetes megkövetelni a következő tulajdonságok teljesülését.

- *Egybehangzó vélemények érvényesülésének elve:* Legyenek a_i és a_j tetszőleges alternatívák. Ha egy $\boldsymbol{\succcurlyeq} = (\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n)$ preferencia-profilban $a_i \succ_t a_j$ minden $1 \leqslant t \leqslant n$ esetén, akkor $a_i \triangleright a_j$.

- *Irreleváns alternatíváktól való függetlenség elve:* Legyenek a_i és a_j tetszőleges alternatívák, valamint legyenek $\boldsymbol{\succcurlyeq} = (\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n)$ és $\boldsymbol{\succcurlyeq'} = (\succcurlyeq'_1, \succcurlyeq'_2, \ldots, \succcurlyeq'_n)$ tetszőleges preferencia-profilok. Ha minden $1 \leqslant t \leqslant n$ esetén az e_t egyén preferenciája a_i és a_j vonatkozásában a $\boldsymbol{\succcurlyeq}$ és $\boldsymbol{\succcurlyeq'}$ preferencia-profilokban ugyanaz, akkor a_i és a_j egymáshoz képesti pozíciója az $\trianglerighteq = \mathcal{R}(\boldsymbol{\succcurlyeq})$ és $\trianglerighteq' = \mathcal{R}(\boldsymbol{\succcurlyeq'})$ preferencia-rendezésekben ugyanaz.

- *Diktatúra hiánya:* Nincs olyan e_t egyén, hogy tetszőleges $\boldsymbol{\succcurlyeq} = (\succcurlyeq_1, \succcurlyeq_2$

$, \ldots, \succcurlyeq_n)$ preferencia profilra $a_i \succ_t a_j$ mindig maga után vonná $a_i \rhd a_j$ teljesülését.

Arrow megmutatta, hogy amennyiben az alternatívák száma legalább három, nem létezik olyan társadalmi jóléti függvény, amelyre a fenti három feltétel egyidejűleg teljesülne!

Arrow-tétel. Ha az alternatívák száma legalább három, és egy társadalmi jóléti függvényre teljesül az egybehangzó vélemények érvényesülésének elve és az irreleváns alternatíváktól való függetlenség elve, akkor az szükségképpen diktatórikus.

Bizonyítás. Legyen az alternatívák száma legalább három, és tekintsünk egy olyan $\mathcal{R}$ társadalmi jóléti függvényt, amelyre teljesül az egybehangzó vélemények érvényesülésének elve és az irreleváns alternatíváktól való függetlenség elve.

Legyenek a_i és a_j tetszőleges alternatívák, és tekintsünk egy olyan $\succcurlyeq = (\succcurlyeq_1, \succcurlyeq_2, \ldots, \succcurlyeq_n)$ preferencia-profilt, amelyben $a_i \succ_t a_j$ minden $1 \leqslant t \leqslant n$ esetén. Cseréljük fel egymás után az a_i és a_j alternatívákat a $\succcurlyeq_t$ preferencia-rendezésekben. Az egybehangzó vélemények érvényesülésének elve szerint az első csere előtt $a_i \rhd a_j$, míg az utolsó csere után $a_j \rhd a_i$ teljesül. Jelölje $e_{t_{ij}}$ az első olyan egyént, akinél a_i és a_j cseréje után $a_i \rhd a_j$ már nem áll fenn. Az irreleváns alternatíváktól való függetlenség elvével összhangban $e_{t_{ij}}$ független $\succcurlyeq$-től.

Legyenek ezután a_i, a_j és a_k tetszőleges alternatívák, és tekintsünk egy olyan $\succcurlyeq'$ preferencia-profilt, amelyben a_i, a_j és a_k egymáshoz képesti pozíciói a következők:

e_1	$\cdots$	$e_{t_{ij}-1}$	$e_{t_{ij}}$	$e_{t_{ij}+1}$	$\cdots$	e_n
a_j	$\cdots$	a_j	a_i	a_i	$\cdots$	a_i
a_k	$\cdots$	a_k				
a_i	$\cdots$	a_i	a_j	a_j	$\cdots$	a_j
			a_k	a_k	$\cdots$	a_k

Ekkor $\mathcal{R}(\succcurlyeq')$-ben $a_i \rhd a_j \rhd a_k$. Itt az első reláció $e_{t_{ij}}$ definíciójából adódik, a második pedig az egybehangzó vélemények érvényesülésének elvéből következik.

Tekintsünk ezután egy olyan $\succcurlyeq''$ preferencia-profilt, amelyben a_i, a_j és a_k egymáshoz képesti pozíciói a következők (a négyzet a_k lehetséges pozícióit jelöli a másik két alternatívához képest, ne feledjük, holtverseny lehetséges):

e_1	$\cdots$	$e_{t_{ij}-1}$	$e_{t_{ij}}$	$e_{t_{ij}+1}$	$\cdots$	e_n
$\square$	$\cdots$	$\square$				
$\square\, a_j$	$\cdots$	$\square\, a_j$	a_j	a_i	$\cdots$	a_i
$\square$	$\cdots$	$\square$		$\square$	$\cdots$	$\square$
a_i	$\cdots$	a_i	a_i	$\square\, a_j$	$\cdots$	$\square\, a_j$
			a_k	$\square$	$\cdots$	$\square$

Ekkor $\mathcal{R}(\succcurlyeq'')$-ben $a_j \trianglerighteq a_i \rhd a_k$. Az első reláció itt is $e_{t_{ij}}$ definíciójából adódik, a második pedig az irreleváns alternatíváktól való függetlenség elvéből következik. Valóban, az egyéni preferenciák a_i és a_k vonatkozásában megegyeznek a $\succcurlyeq'$ és $\succcurlyeq''$ profilokban.

Ez viszont az irreleváns alternatíváktól való függetlenség elvével összhangban azt jelenti, hogy $a_j \succ_{t_{ij}} a_k$ maga után vonja $a_j \rhd a_k$ teljesülését (a többi egyén preferenciája a_j és a_k vonatkozásában nem számít).

Ebből következik, hogy az $e_{t_{jk}}$ egyént definiáló felcserélési eljárásban $a_j \rhd a_k$ nem változhat addig, amíg az $e_{t_{ij}}$ egyén a_k elé sorolja a_j-t, így $t_{jk} \geqslant t_{ij}$. Másrészt az $e_{t_{kj}}$ egyént definiáló felcserélési eljárásban $a_j \rhd a_k$ biztos teljesül miután az $e_{t_{ij}}$ egyén a_k elé sorolja a_j-t, így $t_{kj} \leqslant t_{ij}$. Ennélfogva $t_{jk} \geqslant t_{ij} \geqslant t_{kj}$.

Az a_j és a_k alternatívák szerepét felcserélve, hasonlóan adódik, hogy $t_{kj} \geqslant t_{ik} \geqslant t_{jk}$. Következésképpen $t_{jk} = t_{ij} = t_{ik} = t_{kj}$. És mivel ez bármely három különböző a_i, a_j és a_k alternatívára teljesül, ezért az $e_{t_{ij}}$ egyén az i és j indexektől függetlenül mindig ugyanaz, és az $\mathcal{R}$ társadalmi jóléti függvény minden preferencia-profilhoz ennek az egyénnek a preferencia-rendezését rendeli, vagyis diktatórikus.

Kleinberg lehetetlenségi tétele

Mutatunk még egy ilyen jellegű, negatív eredményt. Legyen X tetszőleges nem üres halmaz, és legyen k egy pozitív egész szám. Az X halmaz k nem üres részhalmazának egy

$$\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$$

családját az X egy k-klaszterezésének nevezzük, ha

- $C_1 \cup C_2 \cup \cdots \cup C_k = X$,
- $C_i \cap C_j = \emptyset$, ha $i \neq j$.

Azt mondjuk, hogy $\mathcal{C}$ egy klaszterezése X-nek ha $\mathcal{C}$ egy k-klaszterezése X-nek valamely pozitív egész k esetén. Egy $\mathcal{C}$ klaszterezésben szereplő halmazokra klaszterekként fogunk hivatkozni. Egy klaszterezést triviálisnak nevezünk, ha minden klaszter egyetlen elemből áll, vagy minden elem ugyanahhoz a klaszterhez tartozik.

Legyen $\mathcal{C}$ egy klaszterezése X-nek, továbbá legyenek $x, y \in X$. Azt fogjuk írni, hogy $x \sim_{\mathcal{C}} y$, ha x és y a $\mathcal{C}$ ugyanabban a klaszterében vannak, illetve $x \not\sim_{\mathcal{C}} y$ ha x és y a $\mathcal{C}$ különböző klasztereiben vannak.

Legyen X tetszőleges nem üres halmaz. Egy $d\colon X \times X \to \mathbb{R}_0^+$ függvényt távolságfüggvénynek nevezünk, ha bármely $x, y \in X$ esetén

- $d(x, y) = 0$ akkor és csak akkor, ha $x = y$,
- $d(x, y) = d(y, x)$.

Jegyezzük meg, hogy a háromszög-egyenlőtlenséget nem követeljük meg automatikusan!

Klaszterezés függvénynek nevezünk egy olyan függvényt, amely egy (X, d) párhoz, ahol X tetszőleges nem üres véges halmaz és d egy távolságfüggvény X-en, az X halmaz egy klaszterezését rendeli.

Egy klaszterezés függvényre viszonylag természetes megkövetelni a következő tulajdonságok teljesülését.

- Egy F klaszterezés függvényt skála-invariánsnak nevezünk ha tetszőleges (X, d) párra és λ pozitív számra $F(X, d) = F(X, \lambda d)$.

- Egy F klaszterezés függvényt konzisztensnek nevezünk, ha $F(X,d) = F(X,d')$ tetszőleges (X,d) párra, továbbá bármely olyan d' távolságfüggvényre az X-en, amelyre teljesül, hogy tetszőleges $x, y \in X$ esetén
  - $d'(x,y) \leqslant d(x,y)$ ha $x \sim_{\mathcal{C}} y$,
  - $d'(x,y) \geqslant d(x,y)$ ha $x \not\sim_{\mathcal{C}} y$,

  ahol $\mathcal{C} = F(X,d)$.
- Egy F klaszterezés függvényt teljesnek nevezünk, ha tetszőleges X halmazhoz és annak tetszőleges $\mathcal{C}$ klaszterezéséhez létezik olyan d távolságfüggvény X-en, hogy $F(X,d) = \mathcal{C}$.

Kleinberg megmutatta, hogy nem létezik olyan klaszterezés függvény, amelyre a fenti három tulajdonság egyidejűleg teljesülne!

Kleinberg-tétel. Nem létezik olyan klaszterezés függvény, amely skála-invariáns, konzisztens és teljes.

Bizonyítás. Kicsit többet fogunk bizonyítani. Legyen X tetszőleges nem üres halmaz, és legyenek $\mathcal{C}_1$ valamint $\mathcal{C}_2$ klaszterezései X-nek. Azt mondjuk, hogy $\mathcal{C}_1$ finomítása $\mathcal{C}_2$-nek ha bármely $\mathcal{C}_2$-beli klaszter előáll $\mathcal{C}_1$-beli klaszterek uniójaként. Megmutatjuk, hogy ha F skála-invariáns és konzisztens klaszterezés függvény, X legalább kételemű véges halmaz, és d_1 és d_2 távolságfüggvények X-en, akkor az $F(X,d_1)$ és $F(X,d_2)$ klaszterezések vagy megegyeznek, vagy egyik se finomítása a másiknak.

Indirekt tegyük fel, hogy valamilyen d_1 és d_2 távolságfüggvényekre $\mathcal{C}_1 = F(X,d_1)$ és $\mathcal{C}_2 = F(X,d_2)$ különböző klaszterezések, és mondjuk $\mathcal{C}_1$ finomítása $\mathcal{C}_2$-nek. Legyen

$$m_1 = \min\{d_1(x_i,x_j) \mid x_i \sim_{\mathcal{C}_1} x_j\},$$
$$M_1 = \max\{d_1(x_i,x_j) \mid x_i \not\sim_{\mathcal{C}_1} x_j\},$$

illetve

$$m_2 = \min\{d_2(x_i,x_j) \mid x_i \sim_{\mathcal{C}_2} x_j\},$$
$$M_2 = \max\{d_2(x_i,x_j) \mid x_i \not\sim_{\mathcal{C}_2} x_j\},$$

továbbá legyenek $a_1 < b_1$ és $a_2 < b_2$ olyan pozitív számok, hogy

$$a_1 \leqslant m_1,$$
$$b_1 \geqslant M_1,$$
$$a_2 \leqslant m_2,$$
$$b_2 \geqslant M_2.$$

Tekintsük most azt a d távolságfüggvényt X-en, amelyre $d(x_i, x_j) = \frac{a_1 a_2}{b_1}$ ha $x_i \sim_{\mathcal{C}_1} x_j$, továbbá $d(x_i, x_j) = a_2$ ha $x_i \sim_{\mathcal{C}_2} x_j$ de $x_i \not\sim_{\mathcal{C}_1} x_j$, végül $d(x_i, x_j) = b_2$ ha $x_i \not\sim_{\mathcal{C}_2} x_j$. Mivel F konzisztens, ezért $F(X, d) = F(X, d_2) = \mathcal{C}_2$.

Tekintsük ezután a $d' = \frac{b_1}{a_2} d$ távolságfüggvényt X-en. Mivel F skálainvariáns, ezért $F(X, d') = F(X, d) = \mathcal{C}_2$. Ugyanakkor $d'(x_i, x_j) = \frac{b_1}{a_2} \frac{a_1 a_2}{b_1} = a_1$ ha $x_i \sim_{\mathcal{C}_1} x_j$, valamint $d'(x_i, x_j) \geqslant \frac{b_1}{a_2} a_2 = b_1$ ha $x_i \not\sim_{\mathcal{C}_1} x_j$, így F konzisztens volta miatt $F(X, d') = F(X, d_1) = \mathcal{C}_1$, ami ellentmondás.

Legyen X tetszőleges nem üres véges halmaz, és legyen d egy távolságfüggvény X-en. Egy összekapcsolás-alapú klaszterező algoritmus először X minden elemét külön klaszterbe sorolja, aztán minden lépésben összekapcsolja egy klaszterré a két legközelebbi klasztert, amíg a klaszterek száma egy előre megadott, 1 és $|X|$ közötti értéket el nem ér.

Két klaszter távolságát egy összekapcsolás-függvény segítségével határozzuk meg. Ennek legelterjedtebb fajtái:

- minimális távolság szerinti: $\min_{x \in C_i, y \in C_j} d(x, y)$,
- átlagos távolság szerinti: $\frac{\sum_{x \in C_i, y \in C_j} d(x,y)}{|C_i||C_j|}$,
- maximális távolság szerinti: $\max_{x \in C_i, y \in C_j} d(x, y)$.

Az algoritmus megállási feltétele nem csak az lehet, hogy a klaszterek száma elért egy előre megadott értéket. Két másik lehetőséget említünk:

- A két legközelebbi klaszter távolsága nagyobb, mint αM, ahol M a maximális távolság az X elemei között, és $\alpha < 1$ adott pozitív szám.
- A két legközelebbi klaszter távolsága nagyobb, mint r, ahol r adott pozitív szám.

Állítás. A minimális távolság szerinti összekapcsolás algoritmus (klaszterezés függvény)

- eredeti verziója skála-invariáns, konzisztens, de nem teljes,
- második verziója $|X| \geqslant 3$ esetén skála-invariás, teljes, de nem konzisztens,
- harmadik verziója $|X| \geqslant 2$ esetén teljes, konzisztens, de nem skálainvariáns.

Oszd meg és uralkodj algoritmusok

Idézzük fel, hogy az összefésüléses rendezés alapötlete az volt, hogy először rendezzük külön-külön egy $A[1:n]$ tömb első felét és második felét, majd ezek tartalmát fésüljük össze. Az $A[1:n]$ tömb rendezésének feladatát tehát egy összefésülés árán két feleakkora résztömb rendezésére vezettük vissza. A résztömböket természetesen ugyanezen ötlettel rendeztük.

Ezt a (rekurzív) megközelítést oszd meg és uralkodj elvnek nevezik. Egy oszd meg és uralkodj algoritmus a rekurzió minden szintjén a következő három lépést hajtja végre.

(1) A feladatot több részfeladatra osztja, amelyek hasonlóak az eredeti feladathoz, de méretük kisebb.

(2) Rekurzív módon megoldja a részfeladatokat (ha a részfeladatok mérete elég kicsi, akkor közvetlenül oldja meg azokat).

(3) A részfeladatok megoldásait kombinálva előállítja az eredeti feladat megoldását.

Inverziók száma

Adott különböző számoknak egy $A[1:n]$ tömbje. Azt mondjuk, hogy valamely $1 \leqslant i < j \leqslant n$ esetén az (i,j) pár az A egy inverziója, ha $A[i] > A[j]$. Adjunk $O(n \log n)$ költségű algoritmust, amely meghatározza az $A[1:n]$ tömb inverzióinak számát!

Módosítsuk az összefésüléses rendezés algoritmust a következőképpen.

```
InverziókSzámaÖsszefésüléssel(A,p,q,r)
k=q-p+1
l=r-q
```

```
for i=1 to k do
  B[i]=A[p+i-1]
for j=1 to l do
  C[j]=A[q+j]
B[k+1]=INFINITY
C[l+1]=INFINITY
i=1
j=1
inv=0
for m=p to r do
  if B[i]<C[j]
    then
      A[m]=B[i]
      i=i+1
    else
      A[m]=C[j]
      inv=inv+k-i+1
      j=j+1
return inv

InverziókSzáma(A,p,r)
inv=0
if p<r then
  q=[(p+r)/2]
  inv=inv+InverziókSzáma(A,p,q)
  inv=inv+InverziókSzáma(A,q+1,r)
  inv=inv+InverziókSzámaÖsszefésüléssel(A,p,q,r)
return inv

InverziókSzáma(A,1,n)
```

Először az $A[1:n]$ tömböt az $A[1:q]$ és az $A[q+1:n]$ résztömbökre bontjuk, ahol $q = [(n+1)/2]$, és rekurzívan meghatározzuk az egyes résztömbök inverziószámát. Ezután következik azon (i', j') inverziók összeszámlálása, amelyekre $1 \leqslant i' \leqslant q < j' \leqslant n$. Ezt valósítja meg az `InverziókSzámaÖsszefésüléssel` eljárás. Jegyezzük meg, hogy mire az eljárás meghívásra kerül, az $A[1:q]$ és az $A[q+1:n]$ résztömbök már rendezettek.

Belátjuk az `InverziókSzámaÖsszefésüléssel` eljárás helyességét. Tegyük fel, hogy az eljárás során éppen az $A[p:q]$ résztömb $A[i']$ és az $A[q+1:r]$ résztömb $A[j']$ értékének összehasonlításánál tartunk, ahol $p \leqslant i' \leqslant q < j' \leqslant$

r. Azt is tegyük fel, hogy már minden olyan inverziót felderítettünk, amelynek az első komponense p és $i' - 1$ között van, vagy a második komponense $q + 1$ és $j' - 1$ között.

- Ha $A[i'] < A[j']$, akkor $A[j'] < A[j' + 1] < \cdots < A[n]$ miatt nincs olyan eddig nem látott inverzió, amelynek első komponense i', így ennél a lépésnél a felderített inverziók száma nem nő.
- Ha $A[i'] > A[j']$, akkor $A[q] > \cdots > A[i' + 1] > A[i']$ miatt (i', j'), $(i' + 1, j'), \ldots, (q, j')$ eddig nem látott új inverziók, így ennél a lépésnél a felderített inverziók száma $(q - i' + 1)$-gyel nő.

Innen az `InverziókSzámaÖsszefésüléssel` eljárás helyessége adódik.

Az algoritmus teljes költsége az összefésüléses rendezéséhez hasonlóan $O(n \log n)$.

Mester-módszer

Az oszd meg és uralkodj algoritmusok elemzését gyakran megkönnyíti a következő tétel.

Tétel. Legyen $T(n)$ a nemnegatív egész számok halmazán a

$$T(n) = aT(n/b) + f(n)$$

rekurzív egyenlettel definiált függvény, ahol $a \geq 1$ és $b > 1$ állandók, $f(n)$ pedig egy pozitív függvény (a rekurzív egyenletet tekintve itt kicsit pongyolán fogalmaztunk, igazából $T(n/b)$ helyett $T(\lfloor n/b \rfloor)$ vagy $T(\lceil n/b \rceil)$ írandó). Ekkor

- ha $f(n) = O(n^{\log_b a - \varepsilon})$ valamely $\varepsilon > 0$ állandóra, akkor
$$T(n) = \Theta(n^{\log_b a});$$
- ha $f(n) = \Theta(n^{\log_b a})$, akkor
$$T(n) = \Theta(n^{\log_b a} \log n);$$
- ha $f(n) = \Omega(n^{\log_b a + \varepsilon})$ valamely $\varepsilon > 0$ állandóra, továbbá
$$af(n/b) \leqslant cf(n)$$
valamely $c < 1$ állandóra minden elég nagy n esetén, akkor
$$T(n) = \Theta(f(n)).$$

Tegyük fel hogy egy algoritmus költségére a következő rekurzív képletet sikerült felállítanunk:

$$T(n) = 9T(n/3) + n.$$

Itt $a = 9$, $b = 3$ és $f(n) = n$. Mivel $n^{\log_b a} = n^{\log_3 9} = n^2$, továbbá $f(n) = O(n^{\log_3 9 - \varepsilon})$, ahol $\varepsilon = 1$, így alkalmazható a tétel első pontja, amely szerint

$$T(n) = \Theta(n^2).$$

Második példánk legyen a

$$T(n) = T(2n/3) + 1$$

rekurzív egyenlet. Itt $a = 1$, $b = 3/2$ és $f(n) = 1$. Most $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$, így alkalmazható a tétel második pontja, amely szerint

$$T(n) = \Theta(\log n).$$

Harmadik példánk a

$$T(n) = 3T(n/4) + n \log n$$

egyenlet. Itt $a = 3$, $b = 4$, $f(n) = n \log n$ és $n^{\log_b a} = n^{\log_4 3} = O(n^{0,793})$. Mivel $f(n) = \Omega(n^{\log_4 3 + \varepsilon})$, ahol $\varepsilon = 0,2$, így alkalmazható a tétel harmadik pontja, ha a regularitási feltétel is teljesül. Most

$$af(n/b) = 3(n/4)\log(n/4) \leqslant (3/4)n \log n = cf(n),$$

ahol $c = 3/4$. Így

$$T(n) = \Theta(n \log n).$$

Végül vizsgáljuk meg a

$$T(n) = 2T(n/2) + n \log n$$

egyenletet. Itt $a = 2$, $b = 2$, $f(n) = n \log n$ és $n^{\log_b a} = n^{\log_2 2} = n$, így úgy tűnik a tétel harmadik esetével van dolgunk, hiszen $f(n) = n \log n$ aszimptotikusan nagyobb, mint $n^{\log_b a} = n$. Vegyük észre azonban, hogy nem polinomiálisan nagyobb, ugyanis

$$\frac{f(n)}{n^{\log_b a}} = \frac{n \log n}{n} = \log n$$

aszimptotikusan kisebb, mint n^ε bármely ε pozitív konstans esetén. Ezért ebben az esetben a tétel nem alkalmazható. Kifinomultabb technika felhasználásával belátható, hogy esetünkben

$$T(n) = \Theta(n \log^2 n).$$

Gyors mátrixszorzás

Adott két $n \times n$-es mátrix:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \quad \text{és} \quad B = \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \vdots & \vdots & & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nn} \end{pmatrix}.$$

Ekkor a $C = AB$ szorzat egy szintén $n \times n$-es mátrix:

$$C = \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \vdots & \vdots & & \vdots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{pmatrix},$$

ahol

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{in}b_{nj} \quad (1 \leqslant i, j \leqslant n).$$

Ha a kijelölt műveleteket a fenti módon végezzük el, ez n^3 elemi szorzást és $n^2(n-1)$ elemi összeadást igényel. Létezik ennél hatékonyabb eljárás?

Az egyszerűség kedvéért először tegyük fel, hogy n kettőhatvány. Ekkor az A, B, C mátrixok mindegyikét négy $n/2 \times n/2$-es méretű mátrixra bonthatjuk:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \qquad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \qquad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Vegyük észre, hogy a szorzatra vonatkozó összefüggések a részmátrixokkal is fennállnak:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22}. \end{aligned}$$

Az előbbi képletek mindegyike két $n/2 \times n/2$-es mátrixszorzást és a szorzatok összeadását tartalmazza.

Jelölje $S(n)$ két $n \times n$-es mátrix összeszorzásához szükséges elemi összeadások, $M(n)$ pedig az elemi szorzások számát. Mátrixszorzásunk költségére most a következő rekurzív képletek érvényesek:

$$\begin{aligned} S(n) &= 8S(n/2) + 4(n/2)^2, \\ M(n) &= 8M(n/2). \end{aligned}$$

Nyilvánvaló módon itt $S(1) = 0$ és $M(1) = 1$.

Legyen $n = 2^k$. Ekkor

$$\begin{aligned}
S(n) &= n^2 + 8S(n/2) \\
&= n^2 + 8((n/2)^2 + 8S(n/2^2)) \\
&= n^2 + 2n^2 + 8^2 S(n/2^2) \\
&= n^2 + 2n^2 + 8^2((n/2^2)^2 + 8S(n/2^3)) \\
&= n^2 + 2n^2 + 2^2 n^2 + 8^3 S(n/2^3) = \cdots \\
&= n^2 + 2n^2 + 2^2 n^2 + \cdots + 2^{k-1} n^2 + 8^k S(n/2^k) \\
&= n^2(1 + 2 + 2^2 + \cdots + 2^{k-1}) + 8^k S(n/2^k) \\
&= n^2 \frac{2^k - 1}{2 - 1} + 8^k S(n/2^k) \\
&= n^2(2^k - 1) + 8^k S(1) \\
&= n^2(2^k - 1) \\
&= n^2(n - 1) = n^3 - n^2.
\end{aligned}$$

$$\begin{aligned}
M(n) &= 8M(n/2) = 8^2 M(n/2^2) = 8^3 M(n/2^3) = \cdots \\
&= 8^k M(n/2^k) = 8^k M(1) = 8^k = 8^{\log_2 n} = n^{\log_2 8} = n^3.
\end{aligned}$$

Következésképpen

$$\begin{aligned}
S(n) &= \Theta(n^3), \\
M(n) &= \Theta(n^3).
\end{aligned}$$

Megmutatjuk, hogy a C_{11}, C_{12}, C_{21}, C_{22} részmátrixok kiszámításához nyolc helyett hét $n/2 \times n/2$-es mátrixszorzás is elegendő. Képezzük a következő 7 darab szorzatot:

$$\begin{aligned}
P_1 &= A_{11}(B_{12} - B_{22}), \\
P_2 &= (A_{11} + A_{12})B_{22}, \\
P_3 &= (A_{21} + A_{22})B_{11}, \\
P_4 &= A_{22}(B_{21} - B_{11}), \\
P_5 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\
P_6 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\
P_7 &= (A_{11} - A_{21})(B_{11} + B_{12}).
\end{aligned}$$

Ellenőrizhető, hogy ekkor

$$\begin{aligned}
C_{11} &= P_5 + P_4 - P_2 + P_6,\\
C_{12} &= P_1 + P_2,\\
C_{21} &= P_3 + P_4,\\
C_{22} &= P_5 + P_1 - P_3 - P_7.
\end{aligned}$$

A régi képletek 8 szorzást és 4 összeadást igényeltek, új módszerünkkel 7 szorzásra és 18 összeadásra (ill. kivonásra) van szükség. Így a költségre vonatkozó megfelelő rekurzív képletek most

$$\begin{aligned}
S(n) &= 7S(n/2) + 18(n/2)^2,\\
M(n) &= 7M(n/2).
\end{aligned}$$

Nyilvánvaló módon itt is $S(1) = 0$ és $M(1) = 1$.

Legyen ismét $n = 2^k$. Ekkor

$$\begin{aligned}
S(n) &= 18(n/2)^2 + 7S(n/2)\\
&= \tfrac{18}{4}n^2 + 7(18(n/2^2)^2 + 7S(n/2^2))\\
&= \tfrac{18}{4}n^2 + \tfrac{18}{4}\,\tfrac{7}{4}n^2 + 7^2S(n/2^2)\\
&= \tfrac{18}{4}n^2 + \tfrac{18}{4}\,\tfrac{7}{4}n^2 + 7^2(18(n/2^3)^2 + 7S(n/2^3))\\
&= \tfrac{18}{4}n^2 + \tfrac{18}{4}\,\tfrac{7}{4}n^2 + \tfrac{18}{4}\left(\tfrac{7}{4}\right)^2 n^2 + 7^3S(n/2^3) = \cdots\\
&= \tfrac{18}{4}n^2 + \tfrac{18}{4}\,\tfrac{7}{4}n^2 + \tfrac{18}{4}\left(\tfrac{7}{4}\right)^2 n^2 + \cdots + \tfrac{18}{4}\left(\tfrac{7}{4}\right)^{k-1} n^2 + 7^kS(n/2^k)\\
&= \tfrac{18}{4}n^2\left(1 + \tfrac{7}{4} + \left(\tfrac{7}{4}\right)^2 + \cdots + \left(\tfrac{7}{4}\right)^{k-1}\right) + 7^kS(n/2^k)\\
&= \tfrac{18}{4}n^2\,\frac{\left(\frac{7}{4}\right)^k - 1}{\frac{7}{4} - 1} + 7^kS(n/2^k)\\
&= \tfrac{18}{4}n^2\,\frac{\left(\frac{7}{4}\right)^k - 1}{\frac{3}{4}} + 7^kS(n/2^k)\\
&= 6n^2\left(\left(\tfrac{7}{4}\right)^k - 1\right) + 7^kS(1) = 6n^2\left(\left(\tfrac{7}{4}\right)^k - 1\right)\\
&= 6n^2\left(\left(\tfrac{7}{4}\right)^{\log_2 n} - 1\right) = 6n^2(n^{\log_2 7/4} - 1)\\
&= 6n^2(n^{\log_2 7 - \log_2 4} - 1) = 6n^2(n^{(\log_2 7)-2} - 1)\\
&= 6n^{\log_2 7} - 6n^2.
\end{aligned}$$

$$\begin{aligned}
M(n) &= 7M(n/2) = 7^2M(n/2^2) = 7^3M(n/2^3) = \cdots\\
&= 7^kM(n/2^k) = 7^kM(1) = 7^k = 7^{\log_2 n} = n^{\log_2 7}.
\end{aligned}$$

Következésképpen

$$S(n) = \Theta(n^{\log_2 7}) = O(n^{2,81}),$$
$$M(n) = \Theta(n^{\log_2 7}) = O(n^{2,81}).$$

Ha n nem kettőhatvány, akkor legyen m a legkisebb n-et meghaladó kettőhatvány és tekintsük az

$$A^* = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} & 0 & \dots & 0 \\ a_{21} & a_{22} & \dots & a_{2n} & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \end{pmatrix}$$

és

$$B^* = \begin{pmatrix} b_{11} & b_{12} & \dots & b_{1n} & 0 & \dots & 0 \\ b_{21} & b_{22} & \dots & b_{2n} & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nn} & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \end{pmatrix}$$

$m \times m$-es mátrixokat. Ekkor a $C^* = A^*B^*$ szorzat egy szintén $m \times m$-es mátrix:

$$C^* = \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1n} & 0 & \dots & 0 \\ c_{21} & c_{22} & \dots & c_{2n} & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ c_{n1} & c_{n2} & \dots & c_{nn} & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \end{pmatrix}.$$

A fenti algoritmust alkalmazva az A^* és B^* mátrixokra, a C^* és így a C mátrix meghatározása az előzőek szerint $O(m^{2,81}) = O((2n)^{2,81}) = O(n^{2,81})$ elemi összeadást és ugyanennyi elemi szorzást igényel.

Mátrixok szorzása még egyszer

Két $n \times n$-es mátrix összeszorzása elég gyakran jön elő különböző számítási feladatok megoldása során. Mint láttuk, a szokásos iskolai algoritmusnak ehhez $O(n^3)$ elemi aritmetikai műveletre van szüksége. A Strassen által kifejlesztett eljárást képes $O(n^{\log_2 7})$ elemei aritmetikai művelettel megoldani a feladatot. Ezt többször megjavították, a jelenlegi rekord $O(n^{2.373})$.

Itt egy kicsit egyszerűbb problémát fogunk tekinteni. Legyenek A, B, C adott $n \times n$-es mátrixok, döntsük el, hogy $AB = C$ teljesül-e. Kiszámolva az AB szorzatot ez egyszerű. Célunk egy ennél hatékonyabb algoritmus kifejlesztése. Az ötlet Freivaldstól származik. Válasszunk véletlenszerűen egy $\alpha \in \{0,1\}^n$ vektort, majd számítsuk ki a $\beta = A(B\alpha)$ és a $\gamma = C\alpha$ vektorokat. Ha $\beta = \gamma$, akkor az algoritmus térjen vissza azzal a válasszal, hogy $AB = C$, ellenkező esetben pedig azzal, hogy $AB \neq C$.

Az algoritmus csupán $O(n^2)$ elemi aritmetikai műveletet végez nyilvánvaló módon. Ha $AB = C$, akkor $\beta = \gamma$ tetszőleges $\alpha \in \{0,1\}^n$ esetén, így ebben az esetben az algoritmus biztos nem téved. Mit mondhatunk abban az esetben, ha $AB \neq C$? Megmutatjuk, hogy ekkor legalább a $\alpha \in \{0,1\}^n$ vektorok felére $\beta \neq \gamma$. Így ha α-t véletlenszerűen választjuk, akkor a tévedés valószínűsége legfeljebb $1/2$. Többször megismételve az eljárást a tévedés valószínűsége tetszőlegesen kicsivé tehető.

Tegyük fel, hogy $AB \neq C$, de $A(B\alpha) = C\alpha$, vagyis $(AB - C)\alpha$ nullvektor valamely $\alpha \in \{0,1\}^n$ esetén. Mivel $AB - C$ nem nullmátrix, ezért van olyan eleme, amelyik nullától különbözik. Legyen d egy ilyen elem, mondjuk az i-edik sor és j-edik oszlop kereszteződésében. Legyen $\alpha' \in \{0,1\}^n$ az a vektor, amelyet úgy kapunk α-ból, hogy a j-edik koordinátáját megváltoztatjuk: ha 0 volt, akkor 1 lesz, ha 1 volt, akkor 0. Ekkor $(AB - C)\alpha'$ biztos nem nullvektor, hiszen az i-edik koordinátája d-vel eltér az $(AB - C)\alpha$ nullvektor i-edik koordinátájától, így $A(B\alpha') \neq C\alpha'$. Jegyezzük még meg, hogy különböző olyan α vektorokhoz, amelyekre $A(B\alpha) = C\alpha$, különböző olyan α' vektorok tartoznak, amelyekre $A(B\alpha') \neq C\alpha'$, hiszen mindegyiknél ugyanazt a koordinátát változtatjuk meg. Innen az állítás adódik.

Polinomok szorzása

Adott egy $(m-1)$-edfokú

$$A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{m-1} x^{m-1}$$

és egy $(n-1)$-edfokú

$$B(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_{n-1} x^{n-1}$$

polinom. Ekkor a $C(x) = A(x)B(x)$ szorzat egy $(m+n-2)$-edfokú polinom:

$$C(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{m+n-2} x^{m+n-2},$$

ahol

$$c_k = \sum_{\substack{i<m,\, j<n \\ i+j=k}} a_i b_j \quad (0 \leqslant k \leqslant m+n-2).$$

Ha a számítást a szokásos módon végezzük, a szorzat meghatározása $O(mn)$ elemi aritmetikai műveletet igényel. Létezik ennél hatékonyabb eljárás?

Az egyszerűség kedvéért legyen $n = m$ kettőhatvány. Értékeljük ki először az $A(x)$ és $B(x)$ polinomokat az $\omega_{j,2n} = e^{2\pi ij/2n}$ helyeken (ezek a $2n$-edik komplex egységgyökök) minden $1 \leqslant j \leqslant 2n$ esetén. Ehhez a következő oszd meg és uralkodj algoritmust használjuk.

Nem nehéz ellenőrizni, hogy egy

$$A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$$

polinom felírható

$$A(x) = A_{\mathrm{ps}}(x^2) + x A_{\mathrm{ptl}}(x^2).$$

alakban, ahol

$$A_{\mathrm{ps}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2} x^{(n-2)/2}$$

és

$$A_{\mathrm{ptl}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1} x^{(n-2)/2}.$$

Így

$$A(\omega_{j,2n}) = A_{\mathrm{ps}}(\omega_{j,2n}^2) + \omega_{j,2n} A_{\mathrm{ptl}}(\omega_{j,2n}^2).$$

minden $1 \leqslant j \leqslant 2n$ esetén. Vegyük még észre, hogy

$$\omega_{j,2n}^2 = (e^{2\pi ji/2n})^2 = e^{2\pi ji/n}$$

egy n-edik komplex egységgyök. Innen azonnal adódik, hogy rekurzívan kiértékelve az $(n-2)/2$-edfokú $A_{\mathrm{ps}}(x)$ és $A_{\mathrm{ptl}}(x)$ polinomokat az n-edik komplex egységgyökökön, az $A(\omega_{j,2n})$ értékek konstans számú elemi aritmetikai művelettel megkaphatók minden $1 \leqslant j \leqslant 2n$ esetén.

Jelölje $T(n)$ azon elemi aritmetikai műveletek számát, amelyek egy $(n-1)$-edfokú polinom kiértékeléséhez szükségesek a $2n$-edik komplex egyéggyökökön. A fentiek szerint

$$T(n) \leqslant 2T(n/2) + O(n).$$

Ugyanezzel a rekurzív képletettel találkoztunk az összefésüléses rendezésnél, ennek megoldása $T(n) = O(n \log n)$.

Értékeljük ki ezután a $C(x) = A(x)B(x)$ polinomot az $\omega_{j,2n}$ helyeken minden $1 \leqslant j \leqslant 2n$ esetén. Ez az $A(\omega_{j,2n})$ és $B(\omega_{j,2n})$ értékek ismeretében csupán $O(n)$ elemi aritmetikai műveletet igényel.

Nem maradt más hátra, mint a $C(x)$ polinom együtthatóinak meghatározása az $\omega_{j,2n}$ helyeken felvett értékeiből ($1 \leqslant j \leqslant 2n$). Hogyan lehet kifejezni egy legfeljebb $(2n-1)$-edfokú

$$C(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_{2n-1} x^{2n-1}$$

polinom együtthatóit a $2n$-edik egységgyökökön felvett $C(\omega_{s,2n})$ értékekből? A módszer neve gyors Fourier transzformált. Definiáljuk a

$$D(x) = d_0 + d_1 x + d_2 x^2 + \cdots + d_{2n-1} x^{2n-1}$$

polinomot, ahol $d_s = C(\omega_{s,2n})$ minden $0 \leqslant s \leqslant 2n-1$ esetén. A $D(x)$ polinomnak egy $2n$-edik egységgyökökön felvett értéke

$$\begin{aligned}
D(\omega_{j,2n}) &= \sum_{s=0}^{2n-1} C(\omega_{s,2n}) \omega_{j,2n}^s \\
&= \sum_{s=0}^{2n-1} \left(\sum_{t=0}^{2n-1} c_t \omega_{s,2n}^t \right) \omega_{j,2n}^s \\
&= \sum_{t=0}^{2n-1} c_t \left(\sum_{s=0}^{2n-1} \omega_{s,2n}^t \omega_{j,2n}^s \right) \\
&= \sum_{t=0}^{2n-1} c_t \left(\sum_{s=0}^{2n-1} e^{2\pi i (st+js)/2n} \right) \\
&= \sum_{t=0}^{2n-1} c_t \left(\sum_{s=0}^{2n-1} \omega_{t+j,2n}^s \right).
\end{aligned}$$

Vegyük azonban észre, hogy ha ω egy 1-től különböző $2n$-edik egységgyök, akkor

$$\sum_{s=0}^{2n-1} \omega^s = 0.$$

Ez azonnal következik abból a tényből, hogy ω definíció szerint gyöke az

$$x^{2n} - 1 = (x-1)(x^{2n-1} + x^{2n-2} + \cdots + x^2 + x + 1)$$

polinomnak, és mivel $\omega \neq 1$, ezért szükségképpen gyöke a jobb oldali szorzat második tényezőjének is.

Ennélfogva az utolsó sor külső összegében csak az az egy tag különbözhet nullától, amelynél $\omega_{t+j,2n} = 1$, vagyis amikor $t + j$ többszöröse $2n$-nek. Ez viszont csak akkor lehetséges, ha $t = 2n - j$. Erre az értékre

$$\sum_{s=0}^{2n-1} \omega_{t+j,2n}^{s} = \sum_{s=0}^{2n-1} 1 = 2n,$$

így

$$D(\omega_{j,2n}) = 2nc_{2n-j}.$$

A $D(\omega_{j,2n})$ értékek a már ismertetett oszd meg és uralkodj algoritmussal $O(n \log n)$ elemi aritmetikai művelettel meghatározhatók, következésképpen a $C(x)$ polinom

$$c_s = \frac{1}{2n} D(\omega_{2n-s,2n})$$

együtthatói ugyanilyen költséggel megkaphatók a $2n$-edik egységgyökökön felvett értékekből minden $0 \leqslant s \leqslant 2n - 1$ esetén.

Az algoritmus teljes költsége $O(n \log n)$.

Legközelebbi pontpár

Adott egy $\mathcal{P} = \{p_1, p_2, \ldots, p_n\}$ ponthalmaz a síkon. Szokásos módon, a $p_i = (x_i, y_i)$ és $p_j = (x_j, y_j)$ pontok távolsága

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

Adjunk hatékony algoritmust a két legközelebbi pont meghatározására (ha több ilyen pár is van, akkor megelégszünk az egyikkel)!

A legegyszerűbb megközelítési mód a következő: nézzük végig az összes pontpárt, és válasszuk ki hol lesz a távolság (négyzete) minimális. Az algoritmus költsége arányos a pontpárok számával, ami

$$\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2).$$

A következőkben a legközelebbi pontpár meghatározására egy oszd meg és uralkodj algoritmust ismertetünk, amelynek költsége $O(n \log n)$.

Minden rekurzív hívásnál átadásra kerülnek a $\mathcal{P}$ ponthalmaz egy $\mathcal{Q}$ részhalmazának pontjai egy, az x koordináták szerint monoton növekvően rendezett X tömbben (azonos x koordinátájú pontok esetén a kisebb y koordinátájú jön előbb), valamint egy, az y koordináták szerint monoton növekvően rendezett Y tömbben (azonos y koordinátájú pontok esetén a kisebb x koordinátájú jön előbb). Hogy ne kelljen minden egyes rekurzív hívás előtt rendezésre pazarolni az időt, már az algoritmus elején előállítjuk $\mathcal{P}$ pontjainak az x, illetve az y koordináták szerint monoton növekvően rendezett tömbjeit, így a rekurzív hívások előtt csak kiválogatásokat kell végezni.

Egy adott rekurzív lépés a következő. Ha $|\mathcal{Q}| \leq 3$, akkor páronkénti vizsgálattal határozzuk meg a minimális távolságot. Természetesen az érdekes eset, amikor $|\mathcal{Q}| > 3$, ilyenkor az alábbiak szerint járunk el.

Először egy olyan ℓ függőleges egyenest keresünk, amely $\mathcal{Q}$-t két olyan $\mathcal{Q}_L$ és $\mathcal{Q}_R$ részre osztja, amelyekre

- $|\mathcal{Q}_L| = \lceil |\mathcal{Q}|/2 \rceil$ és $|\mathcal{Q}_R| = \lfloor |\mathcal{Q}|/2 \rfloor$,
- $\mathcal{Q}_L$ minden pontja az ℓ egyenes bal oldalán van vagy illeszkedik ℓ-re,
- $\mathcal{Q}_R$ minden pontja az ℓ egyenes jobb oldalán van vagy illeszkedik ℓ-re.

Válogassuk szét a pontokat az X tömbből az X_L és X_R tömbökbe: X_L-be kerüljenek a $\mathcal{Q}_L$-beli pontok, X_R-be pedig a $\mathcal{Q}_R$-beliek. Az X_L és X_R tömbök is rendezettek az x koordináták szerint. Válogassuk szét a pontokat az Y tömbből is az Y_L és Y_R tömbökbe: Y_L-be kerüljenek a $\mathcal{Q}_L$-beli pontok, Y_R-be pedig a $\mathcal{Q}_R$-beliek. Az Y_L és Y_R tömbök is rendezettek az y koordináták szerint.

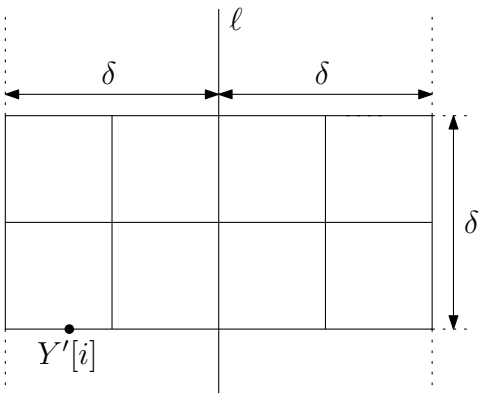
Ezután keressük meg rekurzívan a $\mathcal{Q}_L$ és $\mathcal{Q}_R$ halmazokban a legközelebbi pontpárt. Az első hívás bemenete az X_L és Y_L tömbök, míg a második hívásé az X_R és Y_R tömbök. Legyen $\mathcal{Q}_L$-ben, illetve $\mathcal{Q}_R$-ben a minimális távolság δ_L, illetve δ_R, és legyen $\delta = \min(\delta_L, \delta_R)$. Most a legközelebbi pontpár

- vagy az egyik rekurzív hívás által megtalált δ távolságú páros,
- vagy egy olyan pár, amelynek egyik pontja $\mathcal{Q}_L$-ben, a másik $\mathcal{Q}_R$-ben van.

A következő lépés annak meghatározása, hogy ez utóbbi párok között van-e olyan, amelyben a pontok távolsága kisebb, mint δ. Vegyük észre, hogy ha van ilyen pár, akkor annak egyik pontja sem lehet δ-nál nagyobb távolságra ℓ-től. Válogassuk ki az Y tömbből azokat a pontokat, amelyek

ℓ-től mért távolsága legfeljebb δ. Jelölje a kapott tömböt Y'. Az Y' tömb is rendezett az y koordináták szerint.

Most minden egyes $p \in Y'$ ponthoz keressük meg Y' azon pontjait, melyek p-től mért távolsága legfeljebb δ, és amelyek a p-n átmenő vízszintes egyenesen vagy az fölött vannak. Nem nehéz látni, hogy legfeljebb 7 ilyen pont jöhet számításba. Valóban, ha $i < j$ és $Y'[i]$ valamint $Y'[j]$ távolsága legfeljebb δ, akkor ez a két pont az ábrán látható $\delta \times 2\delta$ oldalú téglalapban van.



Ám a téglalapban lévő 8 kis négyzet egyikében sem lehet egynél több pont Y'-ből (az ℓ egyenesen lévő $\mathcal{Q}_L$-beli pontokat a bal oldali, a $\mathcal{Q}_R$-beli pontokat pedig a jobb oldali kis négyzetekbe sorolva), hiszen egy ilyen kis négyzet átmérője $\delta\sqrt{2}/2 < \delta$. Innen az állítás adódik.

Így minden $p \in Y'$ pontra elég kiszámolni p távolságát az Y'-ben utána következő 7 ponttól. Az így kapott távolságok minimuma legyen δ'. Világos, hogy ha $\delta' < \delta$, akkor $\mathcal{Q}$-ban a minimális távolság δ', egyébként a minimális távolság δ.

Jelölje $T(n)$ a költséget a kezdeti előrendezések nélkül egy n elemű ponthalmaz esetén. A rekurzív hívások költsége $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$, ehhez jön még az X_L, X_R, Y_L, Y_R és Y' rendezett tömbök meghatározásának, illetve δ' kiszámításának költsége. Ez utóbbi műveletek költsége $O(n)$, így a költségre az előrendezések nélkül a

$$T(n) = \begin{cases} T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) & \text{ha } n > 3, \\ O(1) & \text{ha } n \leq 3. \end{cases}$$

rekurzív képlet adódik. Ugyanezzel a rekurzív képletettel találkoztunk az összefésüléses rendezésnél, ennek megoldása

$$T(n) = O(n \log n).$$

Mivel az előrendezések is megvalósíthatók $O(n \log n)$ költséggel (például összefésüléses rendezést használva), ezért az algoritmus teljes költsége szintén $O(n \log n)$.

Dinamikus programozás

A dinamikus programozást elsősorban optimalizálási feladatok megoldására használják. Az ilyen feladatoknak gyakran nagyon sok megoldása lehetséges. Minden megoldás egy bizonyos értékkel bír, és mi az optimális (maximális vagy minimális) értékű megoldást keressük. Sok esetben az optimális megoldás nem egyértelmű, ilyenkor általában megelégszünk egy optimális megoldás megtalálásával.

A dinamikus programozás az oszd meg és uralkodj módszerhez hasonlóan a feladatot részfeladatokra osztással oldja meg. Az oszd meg és uralkodj módszer a feladatot független részfeladatokra osztja, amelyeket rekurzívan megold, majd a részfeladatok megoldásait kombinálva kapjuk az eredeti feladat megoldását. Ezzel szemben a dinamikus programozás akkor alkalmazható, ha a részproblémák nem függetlenek, azaz közös (rész)részproblémáik vannak. Ilyen esetben az oszd meg és uralkodj módszer a szükségesnél többet dolgozik, mert ismételten megoldja a részproblémák közös (rész)részproblémáit. A dinamikus programozás minden egyes (rész)részfeladatot csak egyszer old meg és az eredményt egy táblázatban jegyzi fel, elkerülve így az ismételt számítást mikor a (rész)részfeladat megint felmerül. A dinamikus programozási algoritmusok kifejlesztése négy lépésre bontható fel:

(1) Jellemezzük az optimális megoldás szerkezetét.

(2) Rekurzív módon definiáljuk az optimális megoldás értékét.

(3) Kiszámítjuk az optimális megoldás értékét alulról felfelé történő módon.

(4) A kiszámított információk alapján megszerkesztünk egy optimális megoldást. (Ez a lépés elhagyható ha csak az optimális értékre vagyunk kíváncsiak).

Mátrixok szorzása

Tegyük fel, hogy adott n darab mátrix, jelölje ezeket $A_1, A_2, \ldots, A_n$, és az

$$A_1 A_2 \cdots A_n$$

szorzatot szeretnénk kiszámítani. Ennek a kifejezésnek az értékét megkaphatjuk a mátrixok szokásos szorzásával, amit szubrutinként használunk, miután a szorzások sorrendjét zárójelezéssel meghatározzuk. A mátrixok szorzása asszociatív, így bármilyen zárójelezés ugyanazt az eredményt adja. Például az $A_1A_2A_3A_4$ szorzatot öt különböző módon lehet zárójelezni:

$$\begin{gathered}
A_1(A_2(A_3A_4)), \\
A_1((A_2A_3)A_4), \\
(A_1A_2)(A_3A_4), \\
(A_1(A_2A_3))A_4, \\
((A_1A_2)A_3)A_4.
\end{gathered}$$

Az a mód, ahogy egy szorzatot zárójelezünk, alapvetően befolyásolja a kifejezés kiértékelésének költségét.

Tekintsük először két mátrix összeszorzásának költségét. Az A és B mátrixot csak akkor szorozhatjuk össze, ha kompatibilisek, azaz A oszlopainak száma egyenlő B sorainak számával. Ha A egy $p \times q$ méretű, B pedig egy $q \times r$ méretű mátrix, akkor az eredményül kapott C mátrix $p \times r$ méretű. A C mátrix kiszámításához nyilván pqr darab (skalár) szorzás és ugyanennyi (szintén skalár) összeadás szükséges. A következőkben a költséget a (skalár) szorzások számával fogjuk mérni.

Annak illusztrálására, hogy a zárójelezés mennyire befolyásolja a számítás költségét, tekintsünk egy 10×100 méretű A_1, egy 100×5 méretű A_2 és egy 5×50 méretű A_3 mátrixot. Ha az $A_1A_2A_3$ szorzat kiszámítását az $(A_1A_2)A_3$ zárójelezés szerint végezzük, akkor a 10×5 méretű A_1A_2 mátrix kiszámításához $10 \cdot 100 \cdot 5 = 5000$ szorzást végzünk, majd ezt a mátrixot összeszorozni A_3-mal további $10 \cdot 5 \cdot 50 = 2500$ szorzás, ami összesen 7500 szorzást jelent. Ha viszont a szorzat kiszámítását az $A_1(A_2A_3)$ zárójelezés szerint végezzük, akkor a 100×50 méretű A_2A_3 mátrix kiszámításához $100 \cdot 5 \cdot 50 = 25000$ szorzást végzünk, majd ezt a mátrixot összeszorozni A_1-gyel további $10 \cdot 100 \cdot 50 = 50000$ szorzás, ami összesen 75000 szorzást jelent.

A feladat ezek után a következő: Adott mátrixoknak egy véges $A_1, A_2, \ldots, A_n$ sorozata, ahol az A_i mátrix mérete $p_{i-1} \times p_i$. Keressük meg az

$A_1A_2\cdots A_n$ szorzat azon zárójelezését, amely minimalizálja a szorzat kiszámításához szükséges (skalár) szorzások számát. Fontos megjegyezni, hogy a feladatnak nem része a szorzat kiszámítása!

Megmutatjuk, hogy az összes zárójelezés megvizsgálása nem hatékony eljárárás. Jelölje $P(n)$ egy n mátrixból álló szorzat összes zárójelezéseinek számát. Ha $n = 1$, akkor csak egyetlen mátrixunk van, amit nyilván csak egyetlen módon lehet zárójelezni. Ezután legyen $n > 1$. Ekkor a szorzat nyilván két zárójelezett szorzat szorzata. Mivel a két szorzat zárójelezéseinek száma független egymástól, így a következő rekurzív összefüggéshez jutunk:

$$P(n) = \begin{cases} 1 & \text{ha } n = 1, \\ \displaystyle\sum_{k=1}^{n-1} P(k)P(n-k) & \text{ha } n > 1. \end{cases}$$

Ennek a rekurzív egyenletnek a megoldásai a Catalan számok, amelyekről ismert, hogy $\Omega(4^n/n^{3/2})$ szerint nőnek. Tehát a zárójelezések száma exponenciális n-ben.

Az optimális zárójelezés szerkezete

Tekintsünk egy $A_iA_{i+1}\cdots A_j$ szorzatot. Ennek eredményét jelölje $A_{i..j}$. Ha a feladat nem triviális, azaz $i < j$, akkor az optimális zárójelezés az

$$A_iA_{i+1}\cdots A_j$$

szorzatot két részre vágja valamely A_k és A_{k+1} mátrixok között, ahol $i \leqslant k < j$. Az így adódó k mellett először kiszámítjuk az $A_{i..k}$ és $A_{k+1..j}$ mátrixokat, majd ezeket összeszorozva kapjuk az $A_{i..j}$ végeredményt. Ennélfogva az optimális zárójelezés költsége az $A_{i..k}$ és $A_{k+1..j}$ mátrixok kiszámításának és összeszorzásának együttes költsége.

A kulcsfontosságú észrevétel itt a következő: Az $A_iA_{i+1}\cdots A_j$ szorzat optimális zárójelezésében az $A_iA_{i+1}\cdots A_k$ és $A_{k+1}A_{k+2}\cdots A_j$ szorzatok zárójelezésének is optimálisnak kell lenni. Valóban, ha például az $A_iA_{i+1}\cdots A_k$ szorzatnak volna egy kisebb költségű zárójelezése, akkor az $A_iA_{i+1}\cdots A_j$ szorzat optimális zárójelezésében az első rész zárójelezését erre a kisebb költségűre cserélve a teljes $A_iA_{i+1}\cdots A_j$ szorzatnak is egy kisebb költségű zárójelezését kapnánk, ami ellentmondás. Erre a tulajdonságra optimális részstruktúra tulajdonságként szoktak hivatkozni.

Ebből következik, hogy az $A_iA_{i+1}\cdots A_j$ szorzat optimális zárójelezését megkaphatjuk úgy, hogy a feladatot két részre vágjuk, és az így kialakult

részfeladatok (az $A_iA_{i+1}\cdots A_k$ és $A_{k+1}A_{k+2}\cdots A_j$ szorzatok) optimális zárójelezését összetesszük. Fontos, hogy amikor keressük a kettévágás megfelelő helyét, akkor minden lehetséges pozíciót megvizsgálunk, így az optimumot adót is.

A rekurzív megoldás

A következőkben az optimális megoldás értékét rekurzívan fejezzük ki a részproblémák optimális megoldásainak értékeiből. A részproblémák az

$$A_iA_{i+1}\cdots A_j$$

alakú szorzatok optimális zárójelezésének meghatározása, ahol $1 \leqslant i \leqslant j \leqslant n$.

Jelölje $m[i,j]$ az $A_{i..j}$ mátrix kiszámításához szükséges (skalár) szorzások minimális számát. Ha $i = j$, akkor $A_{i..i} = A_i$. Ilyenkor természetesen egyetlen szorzásra sincs szükség, így $m[i,i] = 0$ minden $i = 1, 2, \ldots, n$ esetén. Ezután legyen $i < j$. Tegyük fel, hogy az optimális zárójelezés az A_k és A_{k+1} mátrixok között vágja szét az $A_iA_{i+1}\cdots A_j$ szorzatot, ahol $i \leqslant k < j$. Ekkor $m[i,j]$ megegyezik az $A_{i..k}$ és $A_{k+1..j}$ mátrixok kiszámításának minimális költségével, plusz a két mátrix összeszorzásának költségével:

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j.$$

(Ne feledjük $A_{i..k}$ egy $p_{i-1} \times p_k$ méretű, $A_{k+1..j}$ pedig egy $p_k \times p_j$ méretű mátrix.)

A fenti rekurzív egyenlet feltételezi, hogy ismerjük k értékét, bár ez nem igaz. Azonban k csak $j - i$ különböző értéket vehet fel, a szóba jövő lehetőségek $k = i, i+1, \ldots, j-1$. Mivel az optimális zárójelezés biztos használja ezek valamelyikét, ezért nincs más teendőnk, mint valamennyi esetet megvizsgálni, és a legjobbat kiválasztani. Ennélfogva

$$m[i,j] = \begin{cases} 0 & \text{ha } i = j, \\ \min\{m[i,k] + m[k+1,j] + p_{i-1}p_kp_j \mid i \leqslant k < j\} & \text{ha } i < j. \end{cases}$$

Az optimális megoldás értéke

Az optimális megoldás értékét alulról felfelé történő módon határozzuk meg. Az algoritmus bemenete a $p = (p_0, p_1, \ldots, p_n)$ sorozat.

```
MátrixSzorzás(p)
for i=1 to n do
   m[i,i]=0
for l=2 to n do
   for i=1 to n-l+1 do
      j=i+l-1
      m[i,j]=INFINITY
      for k=i to j-1 do
         q=m[i,k]+m[k+1,j]+p_{i-1}p_k p_j
         if q<m[i,j] then m[i,j]=q
return m
```

Az algoritmus az első két sorban az $m[i,i] = 0$ értékadásokat hajtja végre. Ezután az $m[i,i+1]$ értékeket, azaz az $l = 2$ hosszú szorzatok minimális költségét számítja ki a harmadik sorban kezdődő ciklus első lefutásakor. A ciklus második lefutásakor következnek az $m[i,i+2]$ értékek, azaz az $l = 3$ hosszú szorzatok minimális költségei, és így tovább. Jegyezzük meg, hogy amikor az $m[i,j]$ mennyiséget határozzuk meg, akkor az ehhez szükséges $m[i,k]$ és $m[k+1,j]$ mennyiségek rendelkezésünkre állnak, azokat már előbb kiszámítottuk. Mivel az $m[i,j]$ mennyiségek csak $i \leqslant j$ esetén definiáltak, ezért az m tömb elemei közül csak a főátló fölé esőket használjuk.

A `MátrixSzorzás(p)` eljárás három egymásba ágyazott ciklusának egyszerű vizsgálata $O(n^3)$ lépés, mivel a három ciklusváltozó mindegyike (l, i és k) legfeljebb n értéket vehet fel.

Egy optimális megoldás

Az optimális megoldás megtalálása érdekében még egy jelölést vezetünk be. Jelölje $s[i,j]$ azt a k indexet, ahol az $A_iA_{i+1}\cdots A_j$ szorzatot az optimális zárójelezés kettévágja, vagyis azt a k indexet, amelyre

$$m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j.$$

Az s tömb a `MátrixSzorzás(p)` eljárás során könnyen számítható.

```
MátrixSzorzás(p)
for i=1 to n do
   m[i,i]=0
for l=2 to n do
   for i=1 to n-l+1 do
      j=i+l-1
```

```
      m[i,j]=INFINITY
      for k=i to j-1 do
         q=m[i,k]+m[k+1,j]+p_{i-1}p_k p_j
         if q<m[i,j] then
            m[i,j]=q
            s[i,j]=k
return m,s
```

Az s tömb felhasználásával már könnyű a mátrixok összeszorzásának leggazdaságosabb módját megtalálni. Az $s[1,n]$ elem azt a k indexet tartalmazza, ahol az optimális zárójelezés az $A_1A_2\cdots A_n$ szorzatot kettévágja A_k és A_{k+1} között. Tehát a teljes $A_{1..n}$ szorzat optimális kiszámításakor a végső mátrixszorzás

$$A_{1..s[1,n]}A_{s[1,n]+1..n}.$$

A korábbi mátrixszorzásokat rekurzívan számíthatjuk ki:

- $s[1,s[1,n]]$ adja meg az utolsó szorzás helyét $A_{1..s[1,n]}$ számításakor,
- $s[s[1,n]+1,n]$ adja meg az utolsó szorzás helyét $A_{s[1,n]+1..n}$ számításakor.

Az eljárást az $(s,1,n)$ paraméterekkel kell meghívni.

```
OptimálisZárójelezés(s,i,j)
if j=i
   then
      print 'A'_i
   else
      print '('
      OptimálisZárójelezés(s,i,s[i,j])
      OptimálisZárójelezés(s,s[i,j]+1,j)
      print ')'
```

Szekvenciák hasonlósága

Biológiai alkalmazásokban gyakran akarjuk összehasonlítani két élőlény DNS láncát. A DNS lánc bizonyos – bázisnak nevezett – molekulákból áll. A lehetséges bázisok adenin, guanin, citozin és timin. A bázisokat a kezdőbetűjűkkel jelölve, a DNS lánc leírható egy a véges $\{A,C,G,T\}$ halmaz feletti sztringgel. Például az egyik élőlény esetén egy szekvencia lehet

$$S_1 = ACCGGTCGAGTGCGCGGAAGCCGGCCGAA,$$

míg a másiknál

$$S_2 = GTCGTTCGGAATGCCGTTGCTCTGTAAA.$$

Minél hasonlóbb a két DNS lánc (szekvencia), az élőlények annál "közelebb" állnak egymáshoz. A hasonlóság persze sokféleképpen mérhető:

- Például azt mondhatjuk, hogy a két DNS lánc akkor hasonló, ha egyik a másiknak részsztringje.
- Azt is mondhatjuk, hogy két lánc akkor hasonló, ha kevés változtatással átvihetők egymásba.
- Egy harmadik lehetőség, hogy keresünk egy olyan S_3 szekvenciát, amelynek a bázisai megjelennek S_1-ben és S_2-ben is, ráadásul ugyanabban a sorrendben, de nem feltétlenül egymás után. Minél hosszabb ilyen S_3 található, annál nagyobb a hasonlóság. Példánkban

$$S_3 = GTCGTCGGAAGCCGGCCGAA.$$

Leghosszabb közös részsorozat

Egy $Z = (z_1, z_2, \ldots, z_k)$ sorozat részsorozata egy $X = (x_1, x_2, \ldots, x_n)$ sorozatnak, ha léteznek olyan $1 \leqslant i_1 < i_2 < \cdots < i_k \leqslant n$ indexek, hogy minden $1 \leqslant j \leqslant k$ esetén $z_j = x_{i_j}$. Azt mondjuk, hogy egy Z sorozat közös részsorozata az X és Y sorozatoknak, ha Z részsorozata X-nek és Y-nak is. Adjunk hatékony algoritmust az $X = (x_1, x_2, \ldots, x_n)$ és $Y = (y_1, y_2, \ldots, y_m)$ sorozatok egy leghosszabb közös részsorozatának meghatározására!

A feladatot dinamikus programozással oldjuk meg. Szükségünk lesz a prefix fogalmára. Az $X = (x_1, x_2, \ldots, x_n)$ sorozat $X_i = (x_1, x_2, \ldots, x_i)$ részsorozatát az X sorozat i-edik prefixének nevezzük tetszőleges $0 \leqslant i \leqslant n$ esetén.

Állítás. Legyen $X = (x_1, x_2, \ldots, x_n)$ és $Y = (y_1, y_2, \ldots, y_m)$ két sorozat és $Z = (z_1, z_2, \ldots, z_k)$ ezek egy leghosszabb közös részsorozata. Ekkor

(1) ha $x_n = y_m$, akkor $z_k = x_n = y_m$ és Z_{k-1} egy leghosszabb közös részsorozata X_{n-1}-nek és Y_{m-1}-nek,

(2) ha $x_n \neq y_m$ és $z_k \neq x_n$, akkor Z egy leghosszabb közös részsorozata X_{n-1}-nek és Y-nak,

(3) ha $x_n \neq y_m$ és $z_k \neq y_m$, akkor Z egy leghosszabb közös részsorozata X-nek és Y_{m-1}-nek.

Bizonyítás.
(1) Ha $z_k \neq x_n$, akkor az $x_n = y_m$ elemet hozzávehetnénk Z-hez, ami által X és Y egy $k+1$ hosszú közös részsorozatát kapnánk, ellentmondás. Így $z_k = x_n = y_m$. Világos, hogy Z_{k-1} közös részsorozata X_{n-1}-nek és Y_{m-1}-nek. Ha lenne ennél hosszabb közös részsorozata X_{n-1}-nek és Y_{m-1}-nek, akkor ehhez z_k-t hozzáfűzve X-nek és Y-nak egy Z-nél hosszabb közös részsorozatát kapnánk, ami nem lehetséges. Így Z_{k-1} szükségképpen egy leghosszabb közös részsorozata X_{n-1}-nek és Y_{m-1}-nek.

(2) Ha $z_k \neq x_n$, akkor Z közös részsorozata X_{n-1}-nek és Y-nak. Ha X_{n-1}-nek és Y-nak volna Z-nél hosszabb közös részsorozata, az Z-nél hosszabb közös részsorozata lenne X-nek és Y-nak is, ellentmondás.

(3) Ugyanúgy, mint az előbb.

Az előző állítás szerint csak egy vagy két részfeladat van, amit meg kell vizsgálni az $X = (x_1, x_2, \ldots, x_n)$ és $Y = (y_1, y_2, \ldots, y_m)$ sorozatok egy leghosszabb közös részsorozatának megkeresésekor.

- Ha $x_n = y_m$, akkor elég X_{n-1} és Y_{m-1} egy leghosszabb közös részsorozatát megtalálni, amelyhez az $x_n = y_m$ elemet csatolva X és Y egy leghosszabb közös részsorozatához jutunk.

- Ha $x_n \neq y_m$, akkor két részfeladatot kell megoldani: X_{n-1} és Y, illetve X és Y_{m-1} egy-egy leghosszabb közös részsorozatát kell megkeresni. Ezek közül a hosszabb X-nek és Y-nak is egy leghosszabb közös részsorozata lesz.

Ezek után az X_i és Y_j sorozatok leghosszabb közös részsorozatának hosszát jelölje $c[i,j]$ minden $0 \leqslant i \leqslant n$ és $0 \leqslant j \leqslant m$ esetén. Ha $i = 0$ vagy $j = 0$, azaz a sorozatok legalább egyikének a hossza nulla, akkor természetesen a leghosszabb közös részsorozat hossza is nulla. A többi esetben a fenti észrevétel igazít el. Mindent összevetve a $c[i,j]$-re vonatkozó rekurzív képlet a következő:

$$c[i,j] = \begin{cases} 0 & \text{ha } i = 0 \text{ vagy } j = 0, \\ c[i-1, j-1] + 1 & \text{ha } i, j > 0 \text{ és } x_i = y_j, \\ \max(c[i-1,j], c[i,j-1]) & \text{ha } i, j > 0 \text{ és } x_i \neq y_j. \end{cases}$$

Egy leghosszabb közös részsorozat megtalálása érdekében még egy jelölést vezetünk be. Minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant m$ esetén jelölje $b[i,j]$ a c tömb

azon elemének indexét, ahonnan $c[i,j]$ értékének meghatározásakor a legjobb értéket kaptuk.

```
LeghosszabbKözösRészsorozat(X,n,Y,m)
for i=1 to n do
   c[i,0]=0
for j=0 to m do
   c[0,j]=0
for i=1 to n do
   for j=1 to m do
      if x_i=y_j
         then
            c[i,j]=c[i-1,j-1]+1
            b[i,j]=(i-1,j-1)
         else
            if c[i-1,j]>=c[i,j-1]
               then
                  c[i,j]=c[i-1,j]
                  b[i,j]=(i-1,j)
               else
                  c[i,j]=c[i,j-1]
                  b[i,j]=(i,j-1)
return c,b
```

A $c[0:n,0:m]$ tömb elemeit sorfolytonosan töltjük ki, először az első sort balról jobbra, aztán a másodikat, és így tovább. A leghosszabb közös részsorozat hossza $c[n,m]$. Az eljárás költsége nyilván $O(nm)$.

A b tömb felhasználásával könnyű egy leghosszabb közös részsorozatot megtalálni: a $b[n,m]$ elemből indulva egyszerűen csak végig kell haladni a "mutatók" mentén a tömbön. Amikor egy $b[i,j]$ elemről a $b[i-1,j-1]$ elemre lépünk, az azt jelenti, hogy az $x_i = y_j$ elem benne van a leghosszabb közös részsorozatban. Ezzel a módszerrel a leghosszabb közös részsorozat elemeit fordított sorrendben kapjuk meg.

A következő rekurzív eljárás a leghosszabb közös részsorozat elemeit a helyes, eredeti sorrendben nyomtatja ki. Az eljárást a (b,X,n,m) paraméterekkel kell meghívni.

```
Nyomtat(b,X,i,j)
if i=0 OR j=0 then return
if b[i,j]=(i-1,j-1)
```

```
then
   Nyomtat(b,X,i-1,j-1)
   print x_i
else
   if b[i,j]=(i-1,j)
      then Nyomtat(b,X,i-1,j)
      else Nyomtat(b,X,i,j-1)
```

Az eljárás költsége $O(n+m)$, hiszen minden rekurzív hívásnál i és j legalább egyikének csökken az értéke.

Szekvenciaillesztés

Tegyük fel, hogy adottak az $X = (x_1, x_2, \ldots, x_m)$ és $Y = (y_1, y_2, \ldots, y_n)$ karaktersorozatok. Tekintsük az $\{1, 2, \ldots, m\}$ és $\{1, 2, \ldots, n\}$ halmazokat, amelyekkel az X és Y karaktersorozatok pozícióit reprezentáljuk, és tekintsük a két halmaz egy M párosítását. Egy ilyen M párosítást szekvenciaillesztésnek nevezünk, ha tetszőleges $(i, j), (i', j') \in M$ és $i < i'$ esetén $j < j'$.

A szekvenciaillesztéseknek létezik egy elég természetes ábrázolási módja. Tekintsük például a `gatcggcat` és `caatgtgaatc` szekvenciákat és az

$$\{(1,1), (2,3), (3,4), (5,5), (6,7), (7,8), (8,9), (9,10)\}$$

szekvenciaillesztést. Ezt a következőképpen szemléltethetjük:

```
g - a t c g - g c a t -
c a a t - g t g a a t c
```

Úgy is interpretálhatjuk ezt a képet, hogy a `gatcggcat` szekvenciától eljuthatunk a `caatgtgaatc` szekvenciához, ha az első pozíción levő `g` karaktert felcseréljük a `c` karakterre, majd az első pozíció után beszúrjuk az `a` karaktert, ezután töröljük a negyedik pozíción levő `c` karaktert, majd az ötödik pozíció után beszúrjuk a `t` karaktert, aztán a hetedik pozíción levő `c` karaktert felcseréljük az `a` karakterre, végül a kilencedik pozíció után beszúrjuk a `c` karaktert.

Legyen M egy szekvenciaillesztés az X és Y karaktersorozatok között. Az M szekvenciaillesztés költségét a következőképpen definiáljuk.

- Létezik egy $g > 0$ konstans, amely a hézag költségét reprezentálja. Az X és Y minden olyan pozíciójára, amely nem szerepel egyik M-beli párban sem, felszámolunk g költséget.

- Az ábécé minden (`"p"`,`"q"`) karakterpárjához létezik egy $c[\texttt{"p"}, \texttt{"q"}] \geqslant 0$ konstans, amely a `"p"` karakternek a `"q"` karakterhez párosításának költségét reprezentálja (általában feltesszük, hogy $c[\texttt{"p"}, \texttt{"p"}] = 0$ az ábécé tetszőleges `"p"` karakterére, bár ez nem szükségszerű). Minden $(i, j) \in M$ párra felszámolunk $c[x_i, y_j]$ költséget.

Az M szekvenciaillesztés költsége legyen ezek után a fent definiált költségek összege.

Adjunk hatékony algoritmust az X és Y karaktersorozatok minimális költségű szekvenciaillesztésének meghatározására! Minél kisebb ez a költség, annál hasonlóbbnak fogjuk tekinteni az X és Y karaktersorozatokat.

A feladatot dinamikus programozással oldjuk meg. Szükségünk lesz egy egyszerű észrevételre.

Állítás. Legyen M optimális szekvenciaillesztés az X és Y karaktersorozatok között. Ekkor a következők legalább egyike fennáll:

(1) $(m, n) \in M$,

(2) az X karaktersorozat m-edik pozíciója nem szerepel az M-beli párok egyikében sem,

(3) az Y karaktersorozat n-edik pozíciója nem szerepel az M-beli párok egyikében sem.

Bizonyítás. Indirekt tegyük fel, hogy $(m, n) \notin M$ és léteznek olyan $i < m$ és $j < n$ pozitív egészek, amelyekre $(m, j) \in M$ és $(i, n) \in M$. Azonban ez ellentmond a szekvenciaillesztés definíciójának: itt $(i, n), (m, j) \in M$ és $i < m$, miközben $n > j$. Innen az állítás adódik.

Állítás. Legyen M optimális szekvenciaillesztés az X és Y karaktersorozatok között.

(1) Ha $(m, n) \in M$, akkor $M \setminus \{(m, n)\}$ optimális szekvenciaillesztés az X_{m-1} és Y_{n-1} prefixek között.

(2) Ha az X karaktersorozat m-edik pozíciója nem szerepel az M-beli párok egyikében sem, akkor M optimális szekvenciaillesztés az X_{m-1} prefix és Y között.

(3) Ha az Y karaktersorozat n-edik pozíciója nem szerepel az M-beli párok egyikében sem, akkor M optimális szekvenciaillesztés X és az Y_{n-1} prefix között.

Bizonyítás.
(1) Ha lenne $M \setminus \{(m,n)\}$-nél kisebb költségű szekvenciaillesztés az X_{m-1} és Y_{n-1} prefixek között, akkor ehhez hozzávéve az (m,n) párt egy M-nél kisebb költségű szekvenciaillesztést kapnánk X és Y között, ami ellentmond M optimalitásának.

(2) Ha lenne M-nél kisebb költségű szekvenciaillesztés az X_{m-1} prefix és Y között, akkor ugyanez M-nél kisebb költségű szekvenciaillesztés lenne X és Y között is, ami ellentmond M optimalitásának.

(3) Ha lenne M-nél kisebb költségű szekvenciaillesztés X és az Y_{n-1} prefix között, akkor ugyanez M-nél kisebb költségű szekvenciaillesztés lenne X és Y között is, ami ellentmond M optimalitásának.

Mit jelent ez?

- Ha az X és Y karaktersorozatok közötti optimális szekvenciaillesztés tartalmazza az (m,n) párt, akkor az optimális szekvenciaillesztés költsége nyilván $c[x_m, y_n]$ plusz az X_{m-1} és Y_{n-1} prefixek közötti optimális szekvenciaillesztés költsége.

- Ha az X és Y karaktersorozatok közötti optimális szekvenciaillesztés egyik párjában sem szerepel az X karaktersorozat m-edik pozíciója, akkor az optimális szekvenciaillesztés költsége nyilván g plusz az X_{m-1} prefix és Y közötti optimális szekvenciaillesztés költsége.

- Ha az X és Y karaktersorozatok közötti optimális szekvenciaillesztés egyik párjában sem szerepel az Y karaktersorozat n-edik pozíciója, akkor az optimális szekvenciaillesztés költsége nyilván g plusz az X és az Y_{n-1} prefix közötti optimális szekvenciaillesztés költsége.

Az optimális szekvenciaillesztés költségének meghatározásánál mindhárom lehetőséget számba kell venni és a legkisebb költségűt választani nyilvánvaló módon.

Jelölje ezek után $a[i,j]$ az X_i és Y_j prefixek közötti optimális szekvenciaillesztés költségét minden $0 \leqslant i \leqslant m$ és $0 \leqslant j \leqslant n$ esetén. Nyilván $a[i,0] = ig$ és $a[0,j] = jg$ minden $0 \leqslant i \leqslant m$ és $0 \leqslant j \leqslant n$ esetén. A többi esetben pedig az előző észrevétellel összhangban

$$a[i,j] = \min(c[x_i, y_j] + a[i-1, j-1], g + a[i-1, j], g + a[i, j-1]).$$

Egy minimális költségű szekvenciaillesztés megtalálása érdekében még egy jelölést vezetünk be. Minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant m$ esetén jelölje $b[i,j]$ az a tömb azon elemének indexét, ahonnan $a[i,j]$ értékének meghatározásakor a legkisebb értéket kaptuk.

```
Szekvenciaillesztés(X,m,Y,n)
for i=1 to m do
  a[i,0]=i*g
for j=0 to n do
  a[0,j]=j*g
for i=1 to m do
  for j=1 to n do
    a[i,j]=c[x_i,y_j]+a[i-1,j-1]
    b[i,j]=(i-1,j-1)
    if a[i,j]>g+a[i-1,j] then
      a[i,j]=g+a[i-1,j]
      b[i,j]=(i-1,j)
    if a[i,j]>g+a[i,j-1] then
      a[i,j]=g+a[i,j-1]
      b[i,j]=(i,j-1)
return a,b
```

Az $a[0:m,0:n]$ tömb elemeit sorfolytonosan töltjük ki, először az első sort balról jobbra, aztán a másodikat, és így tovább. Az optimális szekvenciaillesztés költsége $a[m,n]$. Az eljárás költsége nyilván $O(mn)$.

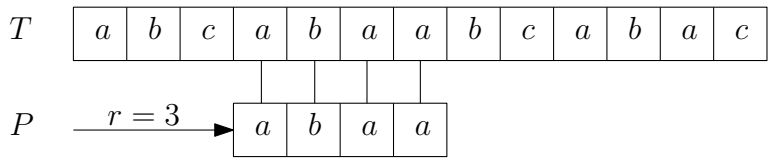
A b tömb felhasználásával könnyű egy optimális szekvenciaillesztést megtalálni: a $b[m,n]$ elemből indulva egyszerűen csak végig kell haladni a "mutatók" mentén a tömbön. Amikor egy $b[i,j]$ elemről a $b[i-1,j-1]$ elemre lépünk, az azt jelenti, hogy az (i,j) pár hozzátartozik az optimális szekvenciaillesztéshez. A következő rekurzív eljárás az optimális szekvenciaillesztés elemeit nyomtatja ki. Az eljárást a (b,m,n) paraméterekkel kell meghívni.

```
Nyomtat(b,i,j)
if i=0 OR j=0 then return
if b[i,j]=(i-1,j-1)
  then
    Nyomtat(b,i-1,j-1)
    print '(' i ',' j ')'
  else
    if b[i,j]=(i-1,j)
      then Nyomtat(b,i-1,j)
      else Nyomtat(b,i,j-1)
```

Az eljárás költsége $O(m+n)$, hiszen minden rekurzív hívásnál i és j legalább egyikének csökken az értéke.

Mintaillesztés

Szövegszerkesztő programokban is gyakori feladat megkeresni egy szövegben egy minta összes előfordulását. Tegyük fel, hogy a szöveget egy n elemű $T[1:n]$ tömb, a mintát pedig egy m elemű $P[1:m]$ tömb tartalmazza, ahol $m \leqslant n$. Azt mondjuk, hogy a P minta r eltolással előfordul a T szövegben, vagy másképpen fogalmazva a T szöveg $(r+1)$-edik pozíciójára illeszkedik, ha $0 \leqslant r \leqslant n-m$ és $T[r+1:r+m] = P[1:m]$, azaz minden $j = 1, 2, \ldots, m$ esetén $T[r+j] = P[j]$. Ha a P minta r eltolással előfordul T-ben, akkor r érvényes eltolás, ellenkező esetben r érvénytelen eltolás.



Adjunk hatékony algoritmust egy adott P minta összes érvényes eltolásának megtalálására egy rögzített T szövegben!

Kezdjük néhány elnevezéssel. Az x és y karakterláncok xy konkatenációja az a karakterlánc, melyben x karaktereit y karakterei követik. A w karakterlánc az x karakterlánc valódi prefixe, ha van olyan nem üres y karakterlánc, hogy $x = wy$. Hasonlóan, a w karakterlánc az x karakterlánc valódi szuffixe, ha van olyan nem üres y karakterlánc, hogy $x = yw$. A w karakterlánc az x karakterlánc szegélye, ha w az x leghosszabb olyan valódi prefixe, amely egyben valódi szuffixe is x-nek. Tehát w a leghosszabb olyan karakterlánc, amelyre alkalmas nem üres u és v karakterláncokkal $x = wu$ és $x = vw$.

Először a P mintához előállítunk egy $S[1:m]$ tömböt, ahol $S[j]$ definíció szerint a $P[1:j]$ karakterlánc szegélyének hossza minden $1 \leqslant j \leqslant m$ esetén. Ehhez dinamikus programozást használunk.

Nyilvánvaló módon $S[1] = 0$. Tegyük fel ezután, hogy $j > 1$ és az $S[1:j-1]$ résztömböt már kitöltöttük. Ekkor $S[j]$ a következőképpen számítható.

Jelölje w a $P[1:j]$ karakterlánc szegélyét és l a w hosszát. Ekkor $S[j] = l$. Először is jegyezzük meg, hogy $S[j]$ értéke legfeljebb $S[j-1]+1$ lehet, hiszen ha $P[1:l] = P[j-l+1:j]$, akkor $P[1:l-1] = P[j-l+1:j-1]$ is nyilván igaz, vagyis a $P[1:j-1]$ karakterláncnak van olyan $l-1$ hosszú valódi prefixe, amely egyben valódi szuffixe is.

Ha $P[j] = P[S[j-1]+1]$, akkor $S[j] = S[j-1]+1$, és viszont. Ezután tegyük fel, hogy $P[j] \neq P[S[j-1]+1]$. Ekkor persze $S[j] \leqslant S[j-1]$. Most a $w[1:l-1]$ karakterlánc valódi prefixe a $P[1:S[j-1]]$ karakterláncnak

és valódi szuffixe a $P[j - S[j-1] : j-1]$ karakterláncnak. Ám ez utóbbi két karakterlánc az S tömb definíciója szerint azonos, így a w karakterlánc valódi prefixe és egyben valódi szuffixe is annak a karakterláncnak, amit úgy kapunk, hogy a $P[1 : S[j-1]]$ karakterlánc végére fűzzük a $P[j]$ karaktert. Vegyük észre, hogy ha lenne ennek a karakterláncnak l-nél hosszabb olyan valódi prefixe, amely egyben valódi szuffixe is, akkor ez a $P[1 : j]$ karakterláncnak szintén egy l-nél hosszabb olyan valódi prefixe lenne, amely egyben valódi szuffixe is, ellentmondás. Következésképpen w szegélye annak a karakterláncnak, amit úgy kapunk, hogy a $P[1 : S[j-1]]$ karakterlánc végére fűzzük a $P[j]$ karaktert.

Lássuk ezek után az S tömb elemeit számító eljárást.

```
SzegélySzámítás(P,m)
S[1]=0
k=0
for j=2 to m do
  while k>0 AND P[k+1]<>P[j] do
    k=S[k]
  if P[k+1]=P[j] then k=k+1
  S[j]=k
return S
```

Meghatározzuk az S tömb kitöltésének a karakterösszehasonlítások számában mért költségét. Jelölje b_j az $S[j]$ elem számításánál felhasznált karakterösszehasonlítások számát (ebbe most $S[1 : j-1]$ számításának költségét nem értjük bele). A k változó értéke kezdetben $S[j-1]$, ez a while ciklus magjának minden lefutása után legalább eggyel csökken. A ciklusmag legalább $b_j - 2$ alkalommal végrehajtásra kerül, ezért a while ciklusból kilépve k értéke legfeljebb $S[j-1] - b_j + 2$. Ezután k értéke eggyel nőhet még, így végső értéke, ami $S[j]$, legfeljebb $S[j-1] - b_j + 3$. Ennélfogva

$$b_j \leqslant S[j-1] - S[j] + 3.$$

Az összehasonlítások száma ezek után

$$\sum_{j=2}^{m} b_j = \sum_{j=2}^{m} (S[j-1] - S[j] + 3) = S[1] - S[m] + 3(m-1) \leqslant 3(m-1).$$

Így az S tömb kitöltésének költége $O(m)$.

Ezután lássuk magát az algoritmust. Tegyük fel, hogy éppen a T szöveg i-edik karakterét a P minta $(k+1)$-edik karakterével hasonlítjuk össze, és

már tudjuk, hogy $T[i-k] = P[1], \ldots, T[i-1] = P[k]$, azaz a minta első k karakterénél illeszkedést találtunk.

Először tegyük fel, hogy $P[k+1] = T[i]$. Ha $k+1 < m$, akkor a szövegben és a mintában is egy hellyel jobbra lépünk (k és i értéke eggyel nő). Ha $k+1 = m$, akkor találtunk egy P-vel azonos részkarakterláncot T-ben. Ilyenkor a szövegben egy hellyel jobbra lépünk, a mintában pedig $m - S[m]$ hellyel vissza (k értéke $S[m]$ lesz, i értéke eggyel nő). Most egyrészt P első $S[m]$ darab karaktere illeszkedni fog a T szöveg i-edik karaktere előtti $S[m]$ hosszú részhez, másrészt ha kevesebb hellyel lépünk vissza, akkor S definíciója miatt a T szöveg i-edik karaktere előtt nem illeszkedhet minden karakter.

Tegyük fel ezután, hogy $P[k+1] \neq T[i]$. Ha $k = 0$, azaz éppen P első karakterénél tartunk, akkor a T szövegben egy hellyel jobbra lépünk (k értéke nem változik, i értéke eggyel nő). Ha $k > 0$, akkor a mintában $k - S[k]$ hellyel visszalépünk (k értéke $S[k]$ lesz, i nem változik). Ez utóbbi esetben egyrészt P első $S[k]$ darab karaktere illeszkedni fog a T szöveg i-edik karaktere előtt lévő $S[k]$ hosszú részhez, másrészt ha kevesebb hellyel lépünk vissza, akkor S definíciója miatt a T szöveg i-edik karaktere előtt nem illeszkedhet minden karakter.

```
Mintaillesztés(T,n,P,m)
S=SzegélySzámítás(P,m)
k=0
for i=1 to n do
  while k>0 AND P[k+1]<>T[i] do
    k=S[k]
  if P[k+1]=T[i] then k=k+1
  if k=m then
    print 'A minta illeszkedik a(z) ' i-m+1 '. pozícióra'
    k=S[k]
```

A költséget itt is a karakterösszehasonlítások számában mérjük. Vizsgáljuk az i és az $i-k$ mennyiségeket az egymás utáni karakterösszehasonlítások pillanataiban. Az első karakterösszehasonlításkor $i = 1$ és $k = 0$, tehát $i - k = 1$. Vegyük észre, hogy az i és $i - k$ mennyiségek semelyik karakterösszehasonlításkor sem kisebbek, mint az azt megelőző karakterösszehasonlításkor. Vegyük észre azt is, hogy a while ciklusok utáni n darab karakterösszehasonlítást leszámítva, az i és $i-k$ mennyiségek legalább egyike minden karakterösszehasonlításkor nagyobb, mint az azt megelőző karakterösszehasonlításkor. Innen $i \leqslant n$ és $i - k \leqslant n$ felhasználásával következik, hogy a karakterösszehasonlítások száma összesen legfeljebb $2n - 1 + n = 3n - 1$.

Így az algoritmus teljes költsége az S tömb számításával együtt $O(m+n)$.

Optimális bináris keresőfa

Adott különböző kulcsoknak egy $k_1 < k_2 < \cdots < k_n$ rendezett sorozata, ezekből szeretnénk egy bináris keresőfát felépíteni. Ha minden k_i kulcsnak ismert a p_i keresési valószínűsége, akkor egy adott T bináris keresőfára megállapítható a keresés költségének várható értéke. Egy kulcs megtalálásának a költsége legyen a keresés során megvizsgált kulcsok száma, azaz a kulcs mélysége a fában (a gyökértől a kulcsot tartalmazó csúcsig vezető út hossza) plusz egy. Így a keresés költségének várható értéke

$$\sum_{i=1}^{n}(d_T(k_i)+1)p_i.$$

A kulcsok keresési valószínűségeinek ismeretében olyan bináris kersőfát szeretnénk szerkeszteni, amelyre a keresés költségének várható értéke minimális. Egy ilyen fát optimális bináris keresőfának nevezünk.

A feladatot dinamikus programozással oldjuk meg. Először is jegyezzük meg, hogy egy a $k_1, k_2, \ldots, k_n$ kulcsokat tartalmazó bináris keresőfa bármely részfája pontosan a $k_i, k_{i+1}, \ldots, k_j$ kulcsokat tartalmazza valamely $1 \leqslant i \leqslant j \leqslant n$ esetén (gondoljunk a fa inorder bejárására).

Minden $1 \leqslant i \leqslant j \leqslant n$ esetén jelölje $c[i,j]$ azon feladat egy optimális megoldásában a

$$\sum_{s=i}^{j}(d_T(k_s)+1)p_s.$$

értéket, amikor a $k_i, k_{i+1}, \ldots, k_j$ kulcsokból építünk bináris keresőfát. A formulák kompakt felírhatóságának érdekében legyen $c[i,i-1]=0$, minden $1 \leqslant i \leqslant n+1$ esetén.

Legyen most $1 \leqslant i \leqslant j \leqslant n$, továbbá legyen $\mathcal{T}$ egy optimális megoldása annak a feladatnak, amikor a $k_i, k_{i+1}, \ldots, k_j$ kulcsokból építünk bináris keresőfát. Tegyük fel, hogy a $\mathcal{T}$ gyökerében a k_h kulcs van. A szokásos módon igazolható, hogy ekkor $\mathcal{T}$ bal oldali $\mathcal{T}'$ részfája egy optimális megoldása annak a feladatnak, amikor a $k_i, \ldots, k_{h-1}$ kulcsokból építünk bináris keresőfát, míg $\mathcal{T}$ jobb oldali $\mathcal{T}''$ részfája egy optimális megoldása annak a feladatnak, amikor a $k_{h+1}, \ldots, k_j$ kulcsokból építünk bináris keresőfát (optimális részstruktúra tulajdonság). Ennélfogva

$$\begin{aligned} c[i,j] &= c[i,h-1] + \sum_{s=i}^{h-1} p_s + p_h + c[h+1,j] + \sum_{s=h+1}^{j} p_s \\ &= c[i,h-1] + c[h+1,j] + \sum_{s=i}^{j} p_s, \end{aligned}$$

hiszen $\mathcal{T}'$ és $\mathcal{T}''$ minden csúcsának mélysége eggyel nagyobb lesz $\mathcal{T}$-ben. Ez a rekurzív egyenlet feltételezi, hogy ismerjük h értékét, bár ez nem igaz. Azonban h csak $j-i+1$ különböző értéket vehet fel, a szóba jövő lehetőségek $h = i, i+1, \ldots, j$. Mivel az optimális megoldás biztos használja ezek valamelyikét, ezért nincs más teendőnk, mint valamennyi esetet megvizsgálni, és a legjobbat kiválasztani. Ennélfogva

$$c[i,j] = \min_{i \leqslant h \leqslant j} \left\{ c[i,h-1] + c[h+1,j] + \sum_{s=i}^{j} p_s \right\}.$$

A $c[i,i-1] = 0$ értékadások végrehajtása után először a $c[i,i]$, ezután a $c[i,i+1]$ értékeket számítjuk ki, majd következnek a $c[i,i+2]$ értékek, és így tovább (a főátlóval párhuzamos átlók mentén). Jegyezzük meg, hogy amikor a $c[i,j]$ értéket határozzuk meg, akkor az ehhez szükséges $c[i,h-1]$ és $c[h+1,j]$ értékek rendelkezésünkre állnak, azokat már előbb kiszámítottuk. Az algoritmus költsége $O(n^3)$.

Egy optimális bináris keresőfa megszerkesztéséhez bevezetünk egy $d[i,j]$ tömböt, amely megadja egy a $k_i, k_{i+1}, \ldots, k_j$ kulcsokból álló optimális bináris keresőfa gyökérelemének kulcsát. Ennek segítségével a fa rekurzívan felépíthető.

Hátizsák feladat

Adottak a $t_1, t_2, \ldots, t_n$ tárgyak. A t_i tárgy súlya w_i, értéke v_i, ahol w_i és v_i pozitív egész számok. Kiválasztandó a tárgyaknak egy olyan részhalmaza, amelyben a tárgyak értékének összege a lehető legnagyobb, súlyuk összege viszont nem halad meg egy adott W pozitív egész számot (a hátizsák kapacitását)!

A feladatot dinamikus programozással oldjuk meg. Minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant W$ esetén jelölje $c[i,j]$ azon feladat egy optimális megoldásában a tárgyak értékeinek az összegét, amikor a hátizsák kapacitása j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk. A formulák kompakt felírhatóságának érdekében legyen $c[i,j] = 0$, ha $i = 0$ vagy $j = 0$.

Legyen most $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant W$, továbbá legyen $\mathcal{O}$ egy optimális megoldása annak a feladatnak, amikor a hátizsák kapacitása j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk. Ha $w_i > j$, akkor t_i nyilván nem szerepelhet $\mathcal{O}$-ban, így $\mathcal{O}$ szükségképpen egy optimális megoldása annak a feladatnak, amikor a hátizsák kapacitása j és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Ebben az esetben az összérték $c[i-1,j]$. Tegyük fel ezután, hogy $w_i \leqslant j$.

- Ha t_i szerepel az $\mathcal{O}$-beli tárgyak között, akkor $\mathcal{O}' = \mathcal{O} \setminus \{t_i\}$ egy optimális megoldása annak a feladatnak, amikor a hátizsák kapacitása $j - w_i$ és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Valóban, ha lenne $\mathcal{O}'$-nél nagyobb összértékű megoldása annak a feladatnak, amikor a hátizsák kapacitása $j - w_i$ és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk, akkor ehhez hozzávéve a t_i tárgyat egy $\mathcal{O}$-nál nagyobb összértékű megoldását kapnánk annak a feladatnak, amikor a hátizsák kapacitása j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk, ellentmondás. Az összérték ekkor $v_i + c[i-1, j-w_i]$.

- Ha t_i nem szerepel az $\mathcal{O}$-beli tárgyak között, akkor $\mathcal{O}$ nyilván egy optimális megoldása annak a feladatnak, mikor a hátizsák kapacitása j és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Az összérték ekkor $c[i-1, j]$.

Mivel nem tudjuk, hogy t_i szerepel-e az $\mathcal{O}$-beli tárgyak között, ezért mindkét lehetőséget megvizsgáljuk, és a kedvezőbbet választjuk. Így

$$c[i,j] = \begin{cases} c[i-1,j] & \text{ha } w_i > j, \\ \max(v_i + c[i-1, j-w_i], c[i-1,j]) & \text{ha } j \geqslant w_i. \end{cases}$$

Annak érdekében, hogy tárgyak egy optimális megoldást adó részhalmazát is meg tudjuk adni, még egy jelölést vezetünk be. Minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant W$ esetén legyen $d[i,j]$ a `TRUE` logikai érték, ha $c[i,j]$ meghatározásánál azt találtuk, hogy a t_i tárgy beválasztásával nagyobb értékű megoldást kapunk, mint anélkül, egyébként legyen $d[i,j]$ a `FALSE` logikai érték.

```
Hátizsák(n,v,w,W)
for j=0 to W do
  c[0,j]=0
for i=1 to n do
  c[i,0]=0
  d[i,0]=FALSE
  for j=1 to W do
    if w[i]>j
      then
        c[i,j]=c[i-1,j]
        d[i,j]=FALSE
      else
        if v[i]+c[i-1,j-w[i]]>c[i-1,j]
          then
            c[i,j]=v[i]+c[i-1,j-w[i]]
```

```
            d[i,j]=TRUE
          else
            c[i,j]=c[i-1,j]
            d[i,j]=FALSE
return c,d
```

A $c[i,j]$ értékek kiszámítását úgy tekinthetjük, mintha egy $(n+1)\times(W+1)$ méretű táblázatot töltenénk ki felülről lefelé, soronként balról jobbra. A feladat optimális megoldásának értéke $c[n,W]$.

Minden $c[i,j]$ érték konstans költséggel számítható, így az algoritmus teljes költsége $O(nW)$. Jegyezzük meg, hogy az algoritmus nem polinomiális!

A d tömb felhasználásával tárgyak egy optimális megoldást adó részhalmazát a következőképpen határozhatjuk meg:

```
TárgyakNyomtatása
j=W
for i=n downto 1 do
  if d[i,j]=TRUE then
    print i '. tárgy'
    j=j-w[i]
```

Hátizsák feladat még egyszer

A hátizsák feladat megoldására mutatunk egy alternatív, szintén dinamikus programozás algoritmust. A részproblémákat kicsit másképp fogjuk definiálni: minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant v_1 + v_2 + \cdots + v_i$ esetén keressük a $t_1, t_2, \ldots, t_i$ tárgyak egy olyan részhalmazát, amelyben a tárgyak összértéke legalább j, összsúlya pedig a lehető legkisebb. Jelölje $c[i,j]$ az összsúlyt ennek a részproblémának az optimális megoldásában. Bár ezen részproblémák egyike se egyezik meg az eredeti hátizsák feladattal, annak megoldása egyszerűen megkapható ezekből: a legnagyobb olyan j érték, amelyre $c[n,j] \leqslant W$. A formulák kompakt felírhatóságának érdekében legyen $c[i,0] = 0$ minden $i = 0, 1, \ldots, n$ esetén.

Legyen most $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant v_1+v_2+\cdots+v_i$, továbbá legyen $\mathcal{O}$ egy optimális megoldása annak a feladatnak, amikor az elérni kívánt érték j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk. Ha $j > v_1 + v_2 + \cdots + v_{i-1}$, akkor t_i biztosan szerepel az $\mathcal{O}$-beli tárgyak között, és $\mathcal{O}' = \mathcal{O} \setminus \{t_i\}$ egy optimális megoldása annak a feladatnak, amikor az elérni kívánt érték $\max(0, j - v_i)$ és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Valóban, ha lenne $\mathcal{O}'$-nél

kisebb összsúlyú megoldása ennek a feladatnak, akkor ehhez hozzávéve a t_i tárgyat egy $\mathcal{O}$-nál kisebb összsúlyú megoldását kapnánk annak a feladatnak, amikor az elérni kívánt érték j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk, ellentmondás. Így ekkor $c[i,j] = w_i + c[i-1, \max(0, j - v_i)]$. Tegyük fel ezután, hogy $j \leqslant v_1 + v_2 + \cdots + v_{i-1}$. Most két eset van.

- Ha t_i szerepel az $\mathcal{O}$-beli tárgyak között, akkor $\mathcal{O}' = \mathcal{O} \setminus \{t_i\}$ egy optimális megoldása annak a feladatnak, amikor az elérni kívánt érték $\max(0, j - v_i)$ és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Ismét, ha lenne $\mathcal{O}'$-nél kisebb összsúlyú megoldása ennek a feladatnak, akkor ehhez hozzávéve a t_i tárgyat egy $\mathcal{O}$-nál kisebb összsúlyú megoldását kapnánk annak a feladatnak, amikor az elérni kívánt érték j és a $t_1, t_2, \ldots, t_i$ tárgyak közül választhatunk, ellentmondás. Az összsúly ekkor $w_i + c[i-1, \max(0, j - v_i)]$.

- Ha t_i nem szerepel az $\mathcal{O}$-beli tárgyak között, akkor $\mathcal{O}$ nyilván egy optimális megoldása annak a feladatnak, mikor az elérni kívánt érték j és a $t_1, t_2, \ldots, t_{i-1}$ tárgyak közül választhatunk. Az összsúly ekkor $c[i-1, j]$.

Mivel nem tudjuk, hogy t_i szerepel-e az $\mathcal{O}$-beli tárgyak között, ezért mindkét lehetőséget megvizsgáljuk, és a kedvezőbbet választjuk. Így ekkor

$$c[i,j] = \min(c[i-1, j], w_i + c[i-1, \max(0, j - v_i)]).$$

A $c[i,j]$ értékek kiszámítását úgy tekinthetjük, mintha egy $(n+1) \times (v_1 + v_2 + \cdots + v_n + 1)$ méretű táblázatot töltenénk ki felülről lefelé, soronként balról jobbra. Minden $c[i,j]$ érték konstans költséggel számítható, így ha bevezetjük a $v^* = \max_i v_i$ jelölést, akkor $v_1 + v_2 + \cdots + v_n \leqslant nv^*$ miatt az algoritmus teljes költsége $O(n^2 v^*)$.

Ha a $c[i,j]$ értékek számításánál azt is feljegyezzük, hogy kedvezőbb volt-e a t_i tárgyat kiválasztani vagy nem, akkor magát egy optimális megoldást is könnyen rekonstruálhatunk.

Ezek után megmutatjuk, hogy a hátizsák feladatra tetszőleges ε pozitív valós számhoz van olyan polinomiális algoritmus, amely által szolgáltatott megoldás értéke az optimális megoldás értékének legalább $1/(1+\varepsilon)$-szorosa.

Feltehetjük, hogy minden tárgy súlya legfeljebb W, amelyekre ez nem teljesül, azok biztos nem lehetnek benne az optimális megoldásban (valójában semmilyen megoldásban), így ezeket elhagyhatjuk. Legyen ε pozitív valós szám. Az egyszerűség kedvéért tegyük fel, hogy ε reciproka egész szám.

Legyen $b = \frac{\varepsilon}{2n} \max_i v_i$, és tekintsük azt a feladatot, amikor minden $1 \leqslant i \leqslant n$ esetén a t_i tárgy értéke $\tilde{v}_i = \lceil v_i / b \rceil b$, súlya változatlanul w_i, és a

hátizsák kapacitása is ugyanúgy W. Jegyezzük meg, hogy $v_i \leqslant \tilde{v}_i \leqslant v_i + b$ minden $1 \leqslant i \leqslant n$ esetén, azaz a v_i és $\tilde{v}_i$ értékek elég közel vannak egymáshoz, ami a különbség, hogy a $\tilde{v}_i$ értékek b egész számú többszörösei. Legyen még $\hat{v}_i = \tilde{v}_i/b$ minden $1 \leqslant i \leqslant n$ esetén. Vegyük észre, hogy a hátizsák feladatnak egy optimális megoldása a $\tilde{v}_i$ értékekkel optimális megoldás lesz a $\hat{v}_i$ értékekkel is, és viszont, a különbség csupán az összértékben van, az utóbbinak b-szerese az előbbi. Futtassuk most a fenti algoritmust a v_i értékek helyett a $\hat{v}_i$ értékekkel, és adjuk vissza azokat a tárgyakat, amelyeket az algoritmus beválasztott a megoldásba.

Jelölje a megoldásba beválasztott tárgyak indexeinek a halmazát I. Először is jegyezzük meg, hogy $\sum_{i \in I} w_i \leqslant W$, hiszen a súlyokon nem változtattunk.

Az algoritmus költségének meghatározásához szükségünk van a $\max_i \hat{v}_i$ értékre. Vegyük észre, hogy ha $v_j = \max_i v_i$, akkor $\hat{v}_j = \max_i \hat{v}_i$, így

$$\max_i \hat{v}_i = \hat{v}_j = \lceil v_j/b \rceil = 2n\varepsilon^{-1}.$$

Következésképpen az algoritmus költsége $O(n^3\varepsilon^{-1})$, ami polinomiális n-ben minden rögzített $\varepsilon > 0$ esetén.

Vizsgáljuk meg milyen messze van az algoritmus által adott megoldás az optimumtól! Tekintsük a tárgyaknak egy tetszőleges olyan $\{t_i \mid i \in I^*\}$ részhalmazát, amelyre $\sum_{i \in I^*} w_i \leqslant W$. Mivel az algoritmus által adott megoldás a $\tilde{v}_i$ értékekkel optimális, ezért

$$\sum_{i \in I} \tilde{v}_i \geqslant \sum_{i \in I^*} \tilde{v}_i,$$

és így

$$\sum_{i \in I^*} v_i \leqslant \sum_{i \in I^*} \tilde{v}_i \leqslant \sum_{i \in I} \tilde{v}_i \leqslant \sum_{i \in I} (v_i + b) \leqslant nb + \sum_{i \in I} v_i.$$

Ha most ismét $v_j = \max_i v_i$, akkor egyrészt $v_j = 2\varepsilon^{-1}nb$, másrészt $v_j = \tilde{v}_j$. Feltételünk szerint $w_i \leqslant W$ minden $1 \leqslant i \leqslant n$ esetén, ennélfogva

$$\sum_{i \in I} \tilde{v}_i \geqslant \tilde{v}_j = 2\varepsilon^{-1}nb.$$

A fenti egyenlőtlenségláncból kiolvashatjuk, hogy

$$\sum_{i \in I} v_i \geqslant \sum_{i \in I} \tilde{v}_i - nb,$$

ahonnan

$$\sum_{i \in I} v_i \geqslant (2\varepsilon^{-1} - 1)nb$$

adódik. Ezt átrendezve kapjuk, hogy

$$nb \leqslant \varepsilon \sum_{i \in I} v_i$$

tetszőleges $\varepsilon \leqslant 1$ pozitív számra, következésképpen

$$\sum_{i \in I^*} v_i \leqslant \sum_{i \in I} v_i + nb \leqslant (1 + \varepsilon) \sum_{i \in I} v_i,$$

amit bizonyítani akartunk.

Részhalmaz összeg

Adottak a $w_1, w_2, \ldots, w_n, W$ természetes számok. Döntsük el, hogy

$$\sum_{i \in I} w_i = W$$

teljesül-e valamilyen $I \subseteq \{1, 2, \ldots, n\}$ indexhalmaz esetén!

Ez a hátizsák feladat speciális esete. A t_i tárgy súlya és értéke is legyen w_i minden $1 \leqslant i \leqslant n$ esetén, valamint legyen a hátizsák kapacitása W. Ha a feladat optimális megoldásában a tárgyak összértéke W, akkor a részhalmaz összeg feladatnak van megoldása, ha kisebb, akkor nincs.

Mohó algoritmusok

Egy optimalizálási feladat megoldására kifejlesztett algoritmus tipikusan olyan lépések sorozatából áll, ahol minden lépésben több lehetőség közül kell valamilyen szempont szerint választanunk. Sok esetben a dinamikus programozás alkalmazása olyan, mintha ágyúval lőnénk verébre; egyszerűbb és hatékonyabb algoritmus is szerkeszthető. Egy mohó algoritmus úgy próbálja megtalálni egy probléma optimális megoldását, hogy minden döntési pontban azt a lépést választja, amely az adott pillanatban a legjobbnak tűnik. Ez a heurisztika természetesen nem mindig vezet el az optimális megoldáshoz, mindazonáltal számos probléma megoldható mohó algoritmussal. Az algoritmus kifejlesztésekor legtöbbször a következő lépéseket hajtjuk végre:

(1) Fogalmazzuk meg a problémát úgy, hogy minden egyes választás után csak egy megoldandó részprobléma keletkezzen.

(2) Bizonyítsuk be, hogy mindig van olyan optimális megoldása az eredeti problémának, amely tartalmazza a mohó választást (mohó választás tulajdonság).

(3) Mutassuk meg, hogy a mohó választással olyan részprobléma keletkezik, amelynek optimális megoldásához hozzávéve a mohó választást, az eredeti probléma egy optimális megoldását kapjuk (optimális részstruktúra tulajdonság).

Mely optimalizálási problémáknak van mohó algoritmusú megoldása? Erre a kérdésre általános válasz nem létezik, de a mohó választás tulajdonság és az optimális részstruktúra tulajdonság két kulcsfontosságú összetevő. Ha meg tudjuk mutatni, hogy egy feladat rendelkezik e két tulajdonsággal, akkor nagy eséllyel ki tudunk fejleszteni mohó algoritmusú megoldást.

Átlagos várakozási idő

Egy étterembe a déli nyitáskor n vendég érkezik. A vendégek jól ismerik a választékot, így azonnal rendelnek. A V_i vendég által rendelt étel elkészí-

tése t_i ideig tart. A szakács egyszerre csak egy étellel tud foglalkozni, és ha valamelyiket elkezdte, akkor azt addig nem hagyja abba, amíg teljesen el nem készült vele. Ha elkészült egy étel, azt rögtön felszolgálják a megfelelő vendégnek. Adjunk hatékony algoritmust annak eldöntésére, hogy a szakács milyen sorrendben készítse el az ételeket, ha azt szeretnénk, hogy a w_i várakozási idők átlaga a lehető legkisebb legyen!

Például ha három vendég van, akik olyan ételeket rendeltek, amelyek elkészítési ideje $t_1 = 2$, $t_2 = 4$, $t_3 = 3$, és a rendeléseket a $(3, 1, 2)$ sorrendben teljesíti az étterem, akkor a várakozási idők $w_3 = 3$, $w_1 = 5$, $w_2 = 9$, az átlagos várakozási idő pedig $17/3$.

Az algoritmus egyszerű: a szakács az elkészítési idő szerint monoton növekvő sorrendben készítse el az ételeket. Az algoritmus helyessége a következő két állításból adódik.

Állítás. Az ételek elkészítésének van olyan optimális ütemezése, amely szerint a legrövidebb elkészítési idejű ételt készítik el elsőnek.

Bizonyítás. Jelölje $i_{\min}$ valamelyik legrövidebb elkészítési idő indexét, és legyen $\sigma\colon \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$ az ételek elkészítésének egy optimális ütemezése. Ha $\sigma(1) = i_{\min}$, akkor kész vagyunk. Ezért tegyük fel, hogy $\sigma(1) \neq i_{\min}$.

Megmutatjuk, hogy az ételek elkészítésének van olyan σ' optimális ütemezése, amelyben $\sigma'(1) = i_{\min}$. Legyen

$$\sigma'\colon \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\},\ i \mapsto \begin{cases} i_{\min} & \text{ha } i = 1, \\ \sigma(1) & \text{ha } i = i^*, \\ \sigma(i) & \text{különben,} \end{cases}$$

ahol $i_{\min} = \sigma(i^*)$. A σ ütemezés mellett a várakozási idők

$$w_{\sigma(i)} = \sum_{j=1}^{i} t_{\sigma(j)}$$

minden $1 \leqslant i \leqslant n$ esetén, az átlagos várakozási idő pedig

$$W(\sigma) = \frac{1}{n} \sum_{i=1}^{n} w_{\sigma(i)} = \frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{i} t_{\sigma(j)} = \frac{1}{n} \sum_{i=1}^{n} (n - i + 1) t_{\sigma(i)}.$$

Hasonlóan, a σ' ütemezés mellett a várakozási idők

$$w_{\sigma'(i)} = \sum_{j=1}^{i} t_{\sigma'(j)}$$

minden $1 \leqslant i \leqslant n$ esetén, az átlagos várakozási idő pedig

$$W(\sigma') = \frac{1}{n}\sum_{i=1}^{n} w_{\sigma'(i)} = \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{i} t_{\sigma'(j)} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1)t_{\sigma'(i)}.$$

Vessük össze a σ és σ' ütemezések melletti átlagos várakozási időket:

$$\begin{gathered} W(\sigma') - W(\sigma) = \frac{1}{n}\sum_{i=1}^{n}(n-i+1)\left(t_{\sigma'(i)} - t_{\sigma(i)}\right) = \\ \frac{1}{n}\left(n\left(t_{\sigma'(1)} - t_{\sigma(1)}\right) + (n-i^*+1)\left(t_{\sigma'(i^*)} - t_{\sigma(i^*)}\right)\right) = \\ \frac{1}{n}\left(n\left(t_{i_{\min}} - t_{\sigma(1)}\right) + (n-i^*+1)\left(t_{\sigma(1)} - t_{i_{\min}}\right)\right) = \\ \frac{1}{n}(i^*-1)\left(t_{i_{\min}} - t_{\sigma(1)}\right) \leqslant 0, \end{gathered}$$

hiszen $i^* > 1$ és $t_{i_{\min}} \leqslant t_{\sigma(1)}$. Mivel σ egy optimális ütemezés, így $W(\sigma') < W(\sigma)$ nem lehetséges. Ennélfogva $W(\sigma') = W(\sigma)$, ami azt jelenti, hogy σ' is egy optimális ütemezés.

Állítás. Tegyük fel, hogy az ételek elkészítésének van olyan optimális ütemezése, amely szerint a V_k vendég által rendelt ételt készítik el elsőnek. Tegyük fel azt is, hogy

$$\sigma' \colon \{1, 2, \ldots, n-1\} \to \{1, \ldots, k-1, k+1, \ldots, n\}$$

a többi étel elkészítésének egy optimális ütemezése. Ekkor

$$\sigma \colon \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\},\ i \mapsto \begin{cases} k & \text{ha } i = 1, \\ \sigma'(i-1) & \text{ha } 2 \leqslant i \leqslant n \end{cases}$$

az összes étel elkészítésének egy optimális ütemezése.

Bizonyítás. Indirekt tegyük fel, hogy σ nem optimális ütemezése az összes étel elkészítésének. Legyen $\pi \colon \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$ az ételek elkészítésének egy olyan optimális ütemezése, amely szerint a V_k vendég által rendelt ételt készítik el elsőnek, azaz amelyre $\pi(1) = k$. Ekkor a π ütemezéshez tartozó $W(\pi)$ átlagos várakozási idő kisebb, mint a σ ütemezéshez tartozó $W(\sigma)$ átlagos várakozási idő.

Tekintsük most a V_k vendég által rendelt ételen kívüli ételek elkészítésének a

$$\pi' \colon \{1, 2, \ldots, n-1\} \to \{1, \ldots, k-1, k+1, \ldots, n\},\ i \mapsto \pi(i+1)$$

ütemezését. Vessük össze a π' és σ' ütemezések melletti átlagos várakozási időket:

$$W(\pi') - W(\sigma') = \frac{1}{n-1}\sum_{i=1}^{n-1} w_{\pi'(i)} - \frac{1}{n-1}\sum_{i=1}^{n-1} w_{\sigma'(i)} =$$

$$\frac{n}{n-1}\left(\frac{1}{n}\left(t_k + \sum_{i=1}^{n-1}\left(t_k + w_{\pi'(i)}\right)\right) - \frac{1}{n}\left(t_k + \sum_{i=1}^{n-1}\left(t_k + w_{\sigma'(i)}\right)\right)\right) =$$

$$\frac{n}{n-1}\left(\frac{1}{n}\sum_{i=1}^{n} w_{\pi(i)} - \frac{1}{n}\sum_{i=1}^{n} w_{\sigma(i)}\right) = \frac{n}{n-1}\left(W(\pi) - W(\sigma)\right) < 0,$$

ami ellentmondás, hiszen σ' egy optimális ütemezése a V_k vendég által rendelt ételen kívüli ételek elkészítésének. Következésképpen σ az összes étel elkészítésének egy optimális ütemezése.

Az algoritmus helyességének belátására mutatunk egy alternatív módszert is, amely szintén gyakran alkalmazható. Az egyszerűség kedvéért tegyük fel, hogy a rendelések már az elkészítési idejük szerint monoton növekvően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$t_1 \leqslant t_2 \leqslant \cdots \leqslant t_n.$$

A fenti algoritmus ekkor a következőképpen ütemezi az ételek elkészítését: a V_1 vendég által rendelt étel elkészítésének kezdési ideje $s_1 = 0$, befejezési ideje $f_1 = s_1 + t_1$, a V_2 vendég által rendelt étel elkészítésének kezdési ideje $s_2 = f_1$, befejezési ideje $f_2 = s_2 + t_2$, és így tovább.

Belátjuk, hogy ez az ütemezés optimális. Jelölje $\mathcal{P}$ az algoritmus által előállított ütemezést. Jegyezzük meg, hogy ebben nincs üresjárat, amikor a szakács nem foglalkozik egyik étel elkészítésével sem, miközben vannak még elkészítésre várók.

Nyilvánvaló, hogy az optimális ütemezések között is kell lenni üresjárat nélkülinek. Jelöljön $\mathcal{O}$ egy ilyen optimális ütemezést. Ha $\mathcal{O} = \mathcal{P}$, akkor készen vagyunk. Tegyük fel ezért, hogy $\mathcal{O} \neq \mathcal{P}$. Ekkor az $\mathcal{O}$ ütemezésben szükségképpen vannak olyan V_i és V_j vendégek, akik által rendelt ételeket közvetlenül egymás után készíti el a szakács, és amelyekre $i > j$.

Tekintsük most azt az $\mathcal{O}'$ ütemezést, amelyet $\mathcal{O}$-ból a V_i és V_j vendégek által rendelt ételek elkészítésének felcserélésével kapunk. Vizsgáljuk meg, hogy a cserével növekedhetett-e az átlagos várakozási idő, vagy ami ezzel ekvivalens, a várakozási idők összege!

Csak a V_i és V_j vendégek által rendelt ételek elkészítésének ütemezése változott meg, így a többi vendég várakozási ideje a csere után ugyanannyi, mint a csere előtt volt. A csere után a V_i vendég által rendelt étel nyilván ugyankkor készül el, mint V_j vendégé a csere előtt. Másrészt a V_j vendég által rendelt étel a csere után mindenképp elkészül addigra, mint a V_i vendégé a csere előtt, hiszen a V_j vendég által rendelt étel elkészítési ideje nem haladja meg a V_i vendég által rendelt étel elkészítési idejét (ne feledjük $j < i$). Így az elkészítési idők összege, és ennélfogva az átlaga az $\mathcal{O}'$ ütemezésnél nem lehet nagyobb, mint az $\mathcal{O}$ ütemezésnél.

Algoritmusunk helyessége innen már egyszerűen adódik. Az optimális $\mathcal{O}$ ütemezésből legfeljebb $\binom{n}{2}$ szomszédos étel elkészítésének cseréjével eljuthatunk a $\mathcal{P}$ ütemezéshez (vö. buborék rendezés). Mivel az átlagos várakozási idő egyik lépésben sem növekszik, így $\mathcal{P}$ szükségképpen szintén egy optimális ütemezés.

Minimális késés

Tegyük fel, hogy adott feladatok egy $\mathcal{A} = \{a_1, a_2, \ldots, a_n\}$ halmaza, amelyek végrehajtása egy közös erőforrás használatával lehetséges csak. Az erőforrás egyszerre egy feladattal tud foglalkozni, és ha valamelyiket elkezdte, akkor azt addig nem hagyja abba, amíg teljesen el nem készült vele. Minden a_i feladathoz adott a t_i végrehajtási ideje, és egy d_i határidő, amelyre a feladat végrehajtásával el kellene készülni.

Tegyük fel, hogy minden feladatot végre kell hajtanunk, de az megengedett, hogy bizonyosakat a határidejük után fejezzünk csak be. Minden a_i feladathoz rendeljük hozzá azt a t_i hosszú intervallumot, amikor az erőforrást a feladat végrehajtására akarjuk használni. Jelölje ezt az intervallumot $[s_i, f_i]$, ahol $s_i \geqslant 0$ és $f_i = s_i + t_i$ természetes módon. Mivel az erőforrás egyszerre egy feladattal tud csak foglalkozni, ezeknek az intervallumoknak a végpontjaiktól eltekintve páronként diszjunktaknak kell lenni.

Azt fogjuk mondani, hogy egy a_i feladatot késve hajtunk végre, ha $f_i > d_i$. Ebben az esetben jelölje $l_i = d_i - f_i$ a késés nagyságát. Ha egy a_i feladat a határidején belül végrehajtásra kerül, akkor legyen $l_i = 0$ definíció szerint.

Adjunk hatékony algoritmust a feladatok egy olyan ütemezésének megkeresésére, amelynél az $L = \max\{l_i \mid 1 \leqslant i \leqslant n\}$ maximális késés a lehető legkevesebb!

Tegyük fel, hogy a feladatok a határidejük szerint monoton növekvően

rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$d_1 \leqslant d_2 \leqslant \cdots \leqslant d_n.$$

Ütemezzük a feladatokat ebben a sorrendben: legyen a_1 kezdési ideje $s_1 = 0$, befejezési ideje $f_1 = s_1 + t_1$, legyen a_2 kezdési ideje $s_2 = f_1$, befejezési ideje $f_2 = s_2 + t_2$, és így tovább. A költség nyilván $O(n \log n)$.

Belátjuk, hogy ez az ütemezés optimális. Jelölje $\mathcal{P}$ az algoritmus által előállított ütemezést. Jegyezzük meg, hogy ebben nincs üresjárat, amikor az erőforrás nem foglalkozik egyik feladattal sem, miközben vannak még elvégzésre várók.

Nyilvánvaló, hogy az optimális ütemezések között is kell lenni üresjárat nélkülinek. Jelöljön $\mathcal{O}$ egy ilyen optimális ütemezést. Ha $\mathcal{O} = \mathcal{P}$, akkor készen vagyunk. Tegyük fel ezért, hogy $\mathcal{O} \neq \mathcal{P}$. Ekkor az $\mathcal{O}$ ütemezésben szükségképpen vannak olyan a_i és a_j közvetlenül egymás után következő feladatok, amelyekre $i > j$.

Tekintsük most azt az $\mathcal{O}'$ ütemezést, amelyet $\mathcal{O}$-ból az a_i és a_j feladatok felcserélésével kapunk. Vizsgáljuk meg, hogy a cserével növekedhetett-e a maximális késés!

Csak az a_i és a_j feladatok ütemezése változott meg, így a többi feladatnál a késés a csere után ugyanannyi, mint a csere előtt volt. A cserével az a_j feladat előbbre került, így a késése nyilván nem növekedett.

Nézzük ezután az a_i feladatot. Jelölje a_j befejezési idejét az $\mathcal{O}$ ütemezésnél f_j, késését pedig l_j. Ekkor a_i befejezési ideje az $\mathcal{O}'$ ütemezésnél $f'_i = f_j$. Jelölje a_i késését az $\mathcal{O}'$ ütemezésnél l'_i.

Ha $l'_i = 0$, vagyis a_i nem késik az $\mathcal{O}'$ ütemezésnél, akkor nyilván nem késhetett az $\mathcal{O}$ ütemezésnél sem, hiszen ott korábbra volt ütemezve. Ha pedig $l'_i > 0$, akkor $d_i \geqslant d_j$ figyelembe vételével

$$l'_i = f'_i - d_i = f_j - d_i \leqslant f_j - d_j = l_j$$

adódik, vagyis a_i késése az $\mathcal{O}'$ ütemezésnél nem lehet nagyobb, mint a_j késése az $\mathcal{O}$ ütemezésnél. Mindebből következik, hogy a maximális késés az $\mathcal{O}'$ ütemezésnél nem lehet nagyobb, mint az $\mathcal{O}$ ütemezésnél.

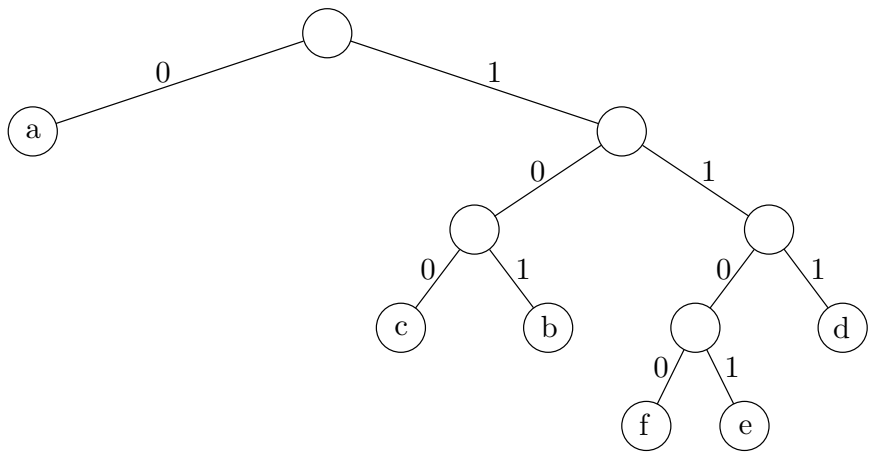
Algoritmusunk helyessége innen már egyszerűen adódik. Az optimális $\mathcal{O}$ ütemezésből legfeljebb $\binom{n}{2}$ szomszédos feladat cseréjével eljuthatunk a $\mathcal{P}$ ütemezéshez (vö. buborék rendezés). Mivel a maximális késés egyik lépésben sem növekszik, így $\mathcal{P}$ szükségképpen szintén egy optimális ütemezés.

Információtömörítés

Tegyük fel, hogy egy n különböző karakterből álló adatállományt akarunk tömörítetten, bitsorozatként tárolni. Ha fix hosszú kódszavakat használunk, akkor $\lceil \log_2 n \rceil$ bitre van szükség az n-féle karakter tárolására. Így egy m hosszú állomány tárolása összesen $m\lceil \log_2 n \rceil$ bitet igényel. Van ennél hatékonyabb módszer?

Ha megengedünk eltérő hosszú kódszavakat, akkor probléma lehet a dekódolással, vagyis az eredeti állománynak a kódolt bitsorozatból való visszanyerésével. Egy lehetséges megoldást kínál a következő. Egy bitsorozatokból álló kód prefix kód, ha egyik karakter kódja sem prefixe más karakter kódjának. Formálisan, ha x és y két különböző karakter kódja, akkor nincs olyan z bitsorozat, amelyre $xz = y$. Itt xz azt a bitsorozatot jelöli, melyben x bitjeit z bitjei követik. Egy prefix kóddal leírt állomány egyértelműen dekódolható. Az állomány elejétől addig megyünk a bitek mentén, amíg egy karakter kódszavát kapjuk. A prefix feltétel miatt csak ez lehet az első karakter. Ezt az eljárást addig folytatjuk, amíg a kódolt állomány végére nem érünk.

Egy prefix kódhoz természetes módon hozzárendelhetünk egy bináris fát. A fa levelei a kódolandó karakterek. Egy adott karakter kódját a gyökértől a karakterig vezető út ábrázolja: a 0 azt jelenti, hogy balra megyünk, az 1 pedig azt, hogy jobbra megyünk az úton a fában.



Ha adott egy prefix kód T fája, akkor egyszerűen kiszámítható az adatállomány kódolásához szükséges bitek száma. A C ábécé minden c karakterére jelölje $f[c]$ a c karakter előfordulási gyakoriságát az állományban, $d_T(c)$ pedig a c-t tartalmazó levél mélységét a T fában (ami éppen a c karakter kódjának

hossza). A kódoláshoz szükséges bitek száma ekkor

$$B_T(C) = \sum_{c \in C} f[c] d_T(c).$$

Ezt az értéket a fa költségének fogjuk nevezni.

A feladat ezek után egy minimális költségű prefix kód előállítása a karakterek előfordulási gyakoriságának ismeretében.

Először is vegyük észre, hogy egy adatállomány optimális prefix kódját mindig teljes bináris fa ábrázolja, azaz olyan fa, amelyben minden nem levél csúcsnak két gyermeke van. Valóban, ha lenne olyan belső csúcs, amelynek csak egy gyereke van, akkor ezt a csúcsot törölve (egyetlen gyerekét a helyére téve) a fa költségét csökkenteni tudnánk. Innen egyszerűen következik, hogy az optimális prefix kód fájának $|C|$ levele és $|C| - 1$ belső csúcsa van.

A következő algoritmus egy optimális prefix kód fáját konstruálja meg. Az algoritmus alulról felfelé haladva építi meg ezt a fát. Kezdetben $|C|$ számú csúcs van, amelyek mindegyike levél, majd $|C| - 1$ számú "összevonás" végrehajtása után alakul ki a végső fa. Két csúcs összevonásának eredménye egy új csúcs, melynek gyakorisága a két összevont csúcs gyakoriságának az összege. Mindig a két legkisebb gyakoriságú csúcsot vonjuk össze; ezek azonosítására egy f szerint kulcsolt Q kupacot használunk. Az algoritmus a megkonstruált fa gyökerével tér vissza.

```
OptimálisPrefixKód(C,f,n)
Q=KupacotÉpít(C,f,n)
for i=1 to n-1 do
  x=MinTöröl(Q)
  y=MinTöröl(Q)
  z=CsúcsotLétrehoz
  bal[z]=x
  jobb[z]=y
  f[z]=f[x]+f[y]
  Beszúr(Q,z)
return MinTöröl(Q)
```

A kupacépítés költsége $O(n)$. A for ciklus iterációinak száma $n - 1$, így mivel az egyes kupacműveletek költsége $O(\log n)$, a ciklus összköltsége $O(n \log n)$. Ennélfogva az algoritmus teljes költsége $O(n \log n)$. Az algoritmus helyessége a következő két állításból adódik.

Állítás. Legyen x és y a két legkisebb gyakoriságú karakter C-ben. Ekkor van olyan optimális prefix kód, amelyben x és y kódszava ugyanolyan hosszú, és a két kódszó csak az utolsó bitben különbözik.

Bizonyítás. Tekintsünk egy optimális prefix kódot ábrázoló T fát. Legyen a és b ebben a fában a legmélyebben fekvő "testvér" csúcspár. Az általánosság megszorítása nélkül feltehetjük, hogy $f[a] \leqslant f[b]$ és $f[x] \leqslant f[y]$. Mivel $f[x]$ és $f[y]$ a két legkisebb gyakoriság, ezért $f[x] \leqslant f[a]$ és $f[y] \leqslant f[b]$.

Cseréljük fel T-ben az x és a csúcsokat. Jelölje az új fát T'. Ezután cseréljük fel T'-ben az y és b csúcsokat. Jelölje az új fát T''. A T és T' fák költségének különbsége:

$$\begin{aligned} B_T(C) - B_{T'}(C) &= \sum_{c \in C} f[c] d_T(c) - \sum_{c \in C} f[c] d_{T'}(c) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_{T'}(x) - f[a]d_{T'}(a) \\ &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_T(a) - f[a]d_T(x) \\ &= (f[a] - f[x])(d_T(a) - d_T(x)) \geqslant 0, \end{aligned}$$

hiszen egyrészt az a karakter gyakorisága legalább akkora, mint az x karakteré, másrészt a legalább olyan mélyen van T-ben, mint x. Hasonlóan, a T' és T'' fák költségének különbsége:

$$\begin{aligned} B_{T'}(C) - B_{T''}(C) &= \sum_{c \in C} f[c] d_{T'}(c) - \sum_{c \in C} f[c] d_{T''}(c) \\ &= f[y]d_{T'}(y) + f[b]d_{T'}(b) - f[y]d_{T''}(y) - f[b]d_{T''}(b) \\ &= f[y]d_{T'}(y) + f[b]d_{T'}(b) - f[y]d_{T'}(b) - f[b]d_{T'}(y) \\ &= (f[b] - f[y])(d_{T'}(b) - d_{T'}(y)) \geqslant 0, \end{aligned}$$

hiszen egyrészt a b karakter gyakorisága legalább akkora, mint az y karakteré, másrészt b legalább olyan mélyen van T'-ben, mint y.

Kaptuk tehát, hogy

$$B_T(C) \geqslant B_{T'}(C) \quad \text{és} \quad B_{T'}(C) \geqslant B_{T''}(C),$$

így

$$B_T(C) \geqslant B_{T''}(C).$$

Figyelembe véve, hogy T optimális prefix kódot ábrázoló fa, ez csak úgy lehetséges, ha

$$B_T(C) = B_{T''}(C).$$

Ebből következik, hogy T'' is egy optimális prefix kódot ábrázoló fa, amelyben x és y maximális mélységű testvércsúcsok. Innen az állítás adódik.

Állítás. Legyen x és y a két legkisebb gyakoriságú karakter C-ben. Most tekintsük azt a C' ábécét, amelyet úgy kapunk C-ből, hogy eltávolítjuk x-et és y-t, majd hozzáadunk egy új z karaktert, melyre $f[z] = f[x] + f[y]$. A többi karakter gyakorisága nem változik. Legyen T' egy optimális prefix kódot ábrázoló fa a C' ábécéhez. Ekkor az a T fa, amelyet úgy kapunk T'-ből, hogy a z csúcshoz gyerekként hozzákapcsoljuk x-et és y-t, egy optimális prefix kódot ábrázoló fa a C ábécéhez.

Bizonyítás. Először is jegyezzük meg, hogy

$$\begin{aligned} B_T(C) - B_{T'}(C') &= f[x]d_T(x) + f[y]d_T(y) - f[z]d_{T'}(z) \\ &= (f[x] + f[y])(d_{T'}(z) + 1) - (f[x] + f[y])d_{T'}(z) \\ &= f[x] + f[y]. \end{aligned}$$

Ezek után indirekt tegyük fel, hogy T nem egy optimális prefix kódot ábrázoló fa a C ábécéhez. Legyen T'' egy optimális prefix kódot ábrázoló fa a C ábécéhez. Az előző állítás alapján feltehetjük, hogy x és y testvérek T''-ben. Most

$$B_T(C) > B_{T''}(C).$$

Legyen T''' az a fa, amelyet úgy kapunk T''-ből, hogy eltávolítjuk az x és y csúcsokat, ezek közös szülőjét z-vel jelöljük, és ehhez a z csúcshoz hozzárendeljük az $f[z] = f[x] + f[y]$ gyakoriságot. Ekkor

$$\begin{aligned} B_{T''}(C) - B_{T'''}(C') &= f[x]d_{T''}(x) + f[y]d_{T''}(y) - f[z]d_{T'''}(z) \\ &= (f[x] + f[y])(d_{T'''}(z) + 1) - (f[x] + f[y])d_{T'''}(z) \\ &= f[x] + f[y]. \end{aligned}$$

Így a T''' fára

$$B_{T'''}(C') = B_{T''}(C) - (f[x] + f[y]) < B_T(C) - (f[x] + f[y]) = B_{T'}(C'),$$

ami ellentmond annak, hogy T' egy optimális prefix kódot ábrázoló fa a C' ábécéhez. Ebből következik, hogy T egy optimális prefix kódot ábrázoló fa a C ábécéhez.

Matroidok

A mohó algoritmusokhoz egy szép elmélet kapcsolódik, ami számos esetben alkalmazható annak kimutatására, hogy egy mohó algoritmus optimális megoldást szolgáltat. Az elmélet egy kombinatorikus struktúra fogalmára épül.

Tekintsünk egy S nem üres véges halmazt és S részhalmazainak egy nem üres $\mathcal{F}$ családját. Az $\mathcal{M} = (S, \mathcal{F})$ párt matroidnak nevezzük, ha teljesülnek rá a következők.

(1) Ha $A \in \mathcal{F}$, akkor tetszőleges $B \subseteq A$ esetén $B \in \mathcal{F}$ is fennáll. Speciálisan $\emptyset \in \mathcal{F}$.

(2) Ha $A, B \in \mathcal{F}$ és A elemszáma kisebb, mint B elemszáma, akkor létezik olyan $x \in B \setminus A$ elem, hogy $A \cup \{x\} \in \mathcal{F}$ szintén fennáll.

Az $\mathcal{F}$-beli halmazokat a matroid független halmazainak nevezzük.

Lássunk két példát.

Tekintsünk egy valós számokból álló T mátrixot. Jelölje S a T mátrix oszlopvektorainak halmazát, $\mathcal{F}$ pedig az S olyan részhalmazainak családját, amelyekben a vektorok lineárisan függetlenek. Ekkor $\mathcal{M} = (S, \mathcal{F})$ matroid.

Tekintsünk egy $G = (V, E)$ irányítatlan gráfot. Legyen $S = E$, valamint $\mathcal{F}$ az S olyan részhalmazainak családját, amelyekben az élek erdőt alkotnak. Ekkor $\mathcal{M} = (S, \mathcal{F})$ matroid.

Egy $\mathcal{M} = (S, \mathcal{F})$ matroid $A \in \mathcal{F}$ független halmazát maximálisnak nevezzük, ha tetszőleges $x \in S \setminus A$ elemre $A \cup \{x\} \notin \mathcal{F}$. A matroidokra vonatkozó (2) tulajdonságból azonnal következik, hogy egy matroid minden maximális független halmaza ugyanolyan elemszámú.

Ha egy $\mathcal{M} = (S, \mathcal{F})$ matroid S alaphalmazán adott egy $w\colon S \to \mathbb{R}^+$ súlyfüggvény, akkor súlyozott matroidról beszélünk. A w súlyfüggvényt összegzéssel kiterjeszthetjük az $\mathcal{F}$-beli független részhalmazokra: ha $A \in \mathcal{F}$, akkor legyen

$$w(A) = \sum_{x \in A} w(x).$$

Sok probléma, amelyre a mohó stratégia optimális megoldást szolgáltat, megfogalmazható súlyozott matroid maximális súlyú független halmazának megkereséseként. Megjegyezzük, hogy egy ilyen független halmaz szükségképpen maximális, hiszen a w súlyfüggvény pozitív.

Például a minimális költségű feszítőfa probléma esetén adott egy $G = (V, E)$ súlyozott élű, irányítatlan, összefüggő gráf a w súlyfüggvénnyel, és keressük egy olyan feszítőfáját G-nek, amelyben az élsúlyok összege minimális. Tekintsük a gráfhoz a már fentebb definiált $\mathcal{M} = (E, \mathcal{F})$ matroidot azzal a w'

súlyfüggvénnyel, amelyre $w'(e) = w_0 - w(e)$, ahol w_0 nagyobb, mint a G-beli élsúlyok maximuma. Ebben a súlyozott matroidban minden súly pozitív, és egy optimális halmaz olyan feszítőfa lesz, amely éleinek az eredeti gráfban vett összsúlya minimális. Így minden olyan algoritmus, amely optimális független halmazt tud keresni egy súlyozott matroidban, meg tudja oldani a minimális költségű feszítőfa problémát is.

A következő mohó algoritmus egy ebben az értelemben optimális független halmazt állít elő egy súlyozott matroidban.

```
Mohó(M,w)
A = ∅
H = S
while H ≠ ∅ do
   töröljük H egy legnagyobb súlyú x elemét
   if A ∪ {x} ∈ 𝓕 then A = A ∪ {x}
return A
```

Dinamikus programozás vs. mohó módszer

Csillagászati megfigyelések

Tegyük fel, hogy adott csillagászati megfigyelések egy $M = \{m_1, m_2, \ldots, m_n\}$ halmaza, amelyeket különböző kutatócsoportok a Hubble űrtávcsővel szeretnének elvégezni. A megfigyelések nem szakíthatók meg és az űrtávcső egy időben legfeljebb egy megfigyelést tud végrehajtani. Minden m_i megfigyelésnek adott az s_i kezdési és az f_i befejezési időpontja, ahol $0 \leqslant s_i < f_i < \infty$. Ezen kívül minden m_i megfigyelésnek adott a w_i tudományos súlya, ahol $w_i > 0$. Az m_i és m_j megfigyeléseket egymást kizáróknak nevezzük, ha az $[s_i, f_i]$ és $[s_j, f_j]$ intervallumoknak van közös belső pontja. Ellenkező esetben azt mondjuk, hogy az m_i és m_j megfigyelések kompatibilisek. Feladatunk páronként kompatibilis M-beli megfigyelések egy olyan $\{m_{i_1}, m_{i_2}, \ldots, m_{i_k}\}$ részhalmazának a meghatározása, amelyre $w_{i_1} + w_{i_2} + \cdots + w_{i_k}$ maximális.

A feladatot dinamikus programozással oldjuk meg. Tegyük fel, hogy a megfigyelések a befejezési idejük szerint monoton növekvően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$f_1 \leqslant f_2 \leqslant \cdots \leqslant f_n.$$

Minden $2 \leqslant i \leqslant n$ esetén legyen $\gamma(i)$ az a legnagyobb $j < i$ index, amelyre az m_j és m_i megfigyelések kompatibilisek. Ha az m_j és m_i megfigyelések egymást kizáróak minden $1 \leqslant j < i$ esetén, akkor legyen $\gamma(i) = 0$, továbbá legyen $\gamma(1) = 0$. A $\gamma(i)$ értékek a megfigyelések befejezési idejeinek monoton növekvően rendezett sorozatában történő bináris kereséssel egyszerűen meghatározhatók.

Minden $1 \leqslant i \leqslant n$ esetén jelölje $v[i]$ azon feladat egy optimális megoldásában a megfigyelések összsúlyát, amikor az $m_1, m_2, \ldots, m_i$ megfigyelések közül választhatunk. A formulák kompakt felírhatóságának érdekében legyen $v[0] = 0$.

Legyen most $1 \leqslant i \leqslant n$ és legyen $\mathcal{O}$ egy optimális megoldása annak a feladatnak, mikor az $m_1, m_2, \ldots, m_i$ megfigyelések közül választhatunk.

- Ha m_i szerepel az $\mathcal{O}$-beli megfigyelések között, akkor $\mathcal{O}' = \mathcal{O} \setminus \{m_i\}$ egy optimális megoldása annak a feladatnak, amikor az $m_1, m_2, \ldots, m_{\gamma(i)}$ események közül választhatunk (ne feledjük, hogy az $m_{\gamma(i)+1}, \ldots, m_{i-2}$, m_{i-1} megfigyelések egymást kizáróak az m_i megfigyeléssel). Valóban, ha lenne $\mathcal{O}'$-nél nagyobb összsúlyú megoldása annak a feladatnak, amikor az $m_1, m_2, \ldots, m_{\gamma(i)}$ megfigyelések közül választhatunk, akkor ehhez hozzávéve az m_i megfigyelést egy $\mathcal{O}$-nál nagyobb összsúlyú megoldását kapnánk annak a feladatnak, mikor az $m_1, m_2, \ldots, m_i$ megfigyelések közül választhatunk, ellentmondás. Az $\mathcal{O}$-beli megfigyelések összsúlya ebben az esetben $w_i + v[\gamma(i)]$.

- Ha m_i nem szerepel az $\mathcal{O}$-beli megfigyelések között, akkor $\mathcal{O}$ nyilván egy optimális megoldása annak a feladatnak, mikor az $m_1, m_2, \ldots, m_{i-1}$ megfigyelések közül választhatunk. Ebben az esetben az $\mathcal{O}$-beli megfigyelések összsúlya $v[i-1]$.

Mivel nem tudjuk, hogy m_i szerepel-e az $\mathcal{O}$-beli megfigyelések között, ezért mindkét lehetőséget megvizsgáljuk és a kedvezőbbet választjuk. Így

$$v[i] = \max(w_i + v[\gamma(i)], v[i-1]).$$

A $v[i]$ értékek kiszámítását úgy tekinthetjük, mintha egy $n+1$ elemű tömböt töltenénk ki balról jobbra. A feladat optimális megoldásában a megfigyelések összsúlya $v[n]$.

Minden $v[i]$ érték $O(1)$ lépésben számítható, így a $v[0:n]$ tömb kitöltésének költsége $O(n)$. Ehhez jön még a kezdeti rendezés és a $\gamma(i)$ értékek meghatározásának költsége, ami $O(n \log n)$. Így az algoritmus teljes költsége $O(n \log n) + O(n) = O(n \log n)$.

Ha minden $1 \leqslant i \leqslant n$ esetén feljegyezzük, hogy $v[i]$ számításánál kedvezőbb volt-e az m_i megfigyelést kiválasztani vagy nem, akkor magát egy optimális megoldást is könnyen rekonstruálhatunk.

Ha minden megfigyelés tudományos súlya ugyanakkora, akkor a feladat egyszerűen páronként kompatibilis M-beli megfigyelések egy maximális elemszámú $\{m_{i_1}, m_{i_2}, \ldots, m_{i_k}\}$ részhalmazának a meghatározása. Erre a feladatra a következő mohó algoritmus is optimális megoldást ad. Tegyük fel ismét, hogy a megfigyelések a befejezési idejük szerint monoton növekvően rendezettek, vagyis $f_1 \leqslant f_2 \leqslant \cdots \leqslant f_n$.

A kiválasztott megfigyeléseket egy H halmazban fogjuk tárolni. Kezdetben legyen $H = \emptyset$. Használni fogunk még egy G listát, amely mindig a még nem vizsgált megfigyeléseket tartalmazza a befejezési idejük szerint monoton növekvően rendezetten. Kezdetben G az összes M-beli megfigyelésből áll. Először rakjuk át G-ből H-ba a legkisebb befejezési időpontú megfigyelést, majd töröljük G-ből az ezzel egymást kizáró megfigyeléseket, amíg egy kompatibilis megfigyelést nem találunk. Ismételjük ezek után a következőt, amíg G-ből nem töröltük az összes megfigyelést. Rakjuk át G-ből H-ba a legkisebb befejezési időpontú megfigyelést, majd töröljük G-ből az ezzel egymást kizáró megfigyeléseket, amíg egy kompatibilis megfigyelést nem találunk.

Legyen $\mathcal{H} = \{m_{j_1}, m_{j_2}, \ldots, m_{j_l}\}$ az algoritmus által kiválasztott megfigyelések halmaza, ahol $j_1 < j_2 < \cdots < j_l$. Világos módon a $\mathcal{H}$-beli megfigyelések páronként kompatibilisek. Belátjuk, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak. Tekintsük a feladat egy $\mathcal{O} = \{m_{i_1}, m_{i_2}, \ldots, m_{i_k}\}$ optimális megoldását, ahol $i_1 < i_2 < \cdots < i_k$. Nyilván $l \leqslant k$.

Először teljes indukcióval megmutatjuk, hogy $f_{j_r} \leqslant f_{i_r}$ minden $1 \leqslant r \leqslant l$ esetén. Ha $r = 1$, akkor ez nyilvánvaló, hiszen az algoritmus elsőként a legkisebb befejezési idejű m_1 megfigyelést választja ki. Legyen most $r > 1$ és tegyük fel, hogy $f_{j_{r-1}} \leqslant f_{i_{r-1}}$. Mivel az $\mathcal{O}$-beli megfigyelések páronként kompatibilisek, ezért $f_{i_{r-1}} \leqslant s_{i_r}$, így $f_{j_{r-1}} \leqslant s_{i_r}$. Ez azt jelenti, hogy az m_{i_r} megfigyelés benne volt abban a halmazban, amelyből az algoritmus a legkisebb befejezési időpontú m_{j_r} megfigyelést választotta, következésképpen $f_{j_r} \leqslant f_{i_r}$.

Indirekt tegyük fel ezután, hogy $\mathcal{H}$ nem egy optimális megoldása a feladatnak, vagyis $l < k$. Az előzőek szerint $f_{j_l} \leqslant f_{i_l}$. Mivel $l < k$, ezért $\mathcal{O}$ tartalmaz egy $f_{i_{l+1}}$ megfigyelést is. Mivel az $\mathcal{O}$-beli megfigyelések páronként kompatibilisek, ezért $f_{i_l} \leqslant s_{i_{l+1}}$, következésképpen $f_{j_l} \leqslant s_{i_{l+1}}$. Ez viszont azt jelenti, hogy miután töröltük G-ből az m_{j_l} megfigyeléssel egymást kizáró megfigyeléseket, az $m_{i_{l+1}}$ megfigyelés még G-ben maradt, így az algoritmus nem fejeződhetett be, ellentmondás. Ebből következik, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak.

Az algoritmus költsége a kezdeti rendezés nélkül $O(n)$, azzal együtt

$$O(n \log n) + O(n) = O(n \log n).$$

Fesztiválbérlet

Egy zenei fesztivál szervezői a fesztiválbérlet árát a következők alapján szeretnék meghatározni. Minden programhoz megállapítanak egy elvi részvételi

díjat, amit teljes egészében ki kellene fizetni egy látogatónak, ha a programon akárcsak részben részt vesz. Legalább mennyit kellene így fizetni egy látogatónak, ha a fesztivál ideje alatt végig részt venne valamilyen programon? Feltehetjük, hogy a fesztiválon soha nincs üresjárat, mindig zajlik legalább egy program.

Formálisan, adott a fesztiválnak az s kezdési és az f befejezési időpontja. Adott továbbá a programok $P = \{p_1, p_2, \ldots, p_n\}$ halmaza. Minden p_i programnak adott az s_i kezdési és az f_i befejezési időpontja, ahol $s \leqslant s_i < f_i \leqslant f$. Feltételünk szerint

$$[s_1, f_1] \cup [s_2, f_2] \cup \cdots \cup [s_n, f_n] = [s, f].$$

Ezen kívül minden p_i programnak adott a w_i részvételi díja, ahol $w_i > 0$. Feladatunk P-beli programok egy olyan $\{p_{i_1}, p_{i_2}, \ldots, p_{i_k}\}$ részhalmazának a meghatározására, amelyre

$$[s_{i_1}, f_{i_1}] \cup [s_{i_2}, f_{i_2}] \cup \cdots \cup [s_{i_k}, f_{i_k}] = [s, f]$$

és $w_{i_1} + w_{i_2} + \cdots + w_{i_k}$ minimális.

A feladatot dinamikus programozással oldjuk meg. Tegyük fel, hogy a programok a kezdési idejük szerint monoton növekvően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$s_1 \leqslant s_2 \leqslant \cdots \leqslant s_n.$$

Feltételünk szerint $s_1 = s$.

A formulák kompakt felírhatóságának érdekében legyen $s_{n+1} = f$ és legyen m az a legkisebb $2 \leqslant j \leqslant n+1$ index, amelyre $s_j > s$. Legyen továbbá minden $m \leqslant i \leqslant n+1$ esetén $\gamma(i)$ az a legnagyobb $j < i$ index, amelyre $s_j < s_i$.

Ezek után minden $m \leqslant i \leqslant n+1$ esetén jelölje $v[i]$ azon feladat egy optimális megoldásában a programokért fizetendő részvételi díjak összegét, amikor a $p_1, p_2, \ldots, p_{\gamma(i)}$ programok közül választhatunk és a fesztivál elejétől az s_i időpontig kell kitölteni az időt.

Legyen most $m \leqslant i \leqslant n+1$ és legyen $\mathcal{O}$ egy optimális megoldása annak a feladatnak, amikor a $p_1, p_2, \ldots, p_{\gamma(i)}$ programok közül választhatunk és a fesztivál elejétől az s_i időpontig kell kitölteni az időt. Világos, hogy ekkor van olyan $\mathcal{O}$-beli p_j program, amelyre $f_j \geqslant s_i$. Ez a program $\mathcal{O}$ optimalitása miatt egyértelmű és az összes többi $\mathcal{O}$-beli program, ha van ilyen, az s_j időpont előtt kezdődik. Ha $s_j = s$, akkor nyilván $\mathcal{O} = \{p_j\}$, a költség pedig

w_j. Ha viszont $s_j > s$, akkor $\mathcal{O}' = \mathcal{O} \setminus \{p_j\}$ szükségképpen egy optimális megoldása annak a feladatnak, amikor a $p_1, p_2, \ldots, p_{\gamma(j)}$ programok közül választhatunk (ezek kezdődnek az s_j időpont előtt), és a fesztivál elejétől az s_j időpontig kell kitölteni az időt. Valóban, ha lenne az utóbbi feladatnak $\mathcal{O}'$-nél kedvezőbb megoldása, akkor ezt a megoldást a p_j programmal kiegészítve egy $\mathcal{O}$-nál kedvezőbb megoldását kapnánk annak a feladatnak, amikor a $p_1, p_2, \ldots, p_{\gamma(i)}$ programok közül választhatunk és a fesztivál elejétől az s_i időpontig kell kitölteni az időt, ellentmondás. A költség ekkor $w_j + v[j]$. Egy probléma van: nem ismerjük j értékét. Azonban j legfeljebb $\gamma(i) \leqslant n$ különböző értéket vehet fel; vizsgáljuk meg az összes lehetőséget, és válasszuk ki a legkedvezőbbet. Így

$$\begin{aligned} v[i] = \min(\{w_j \mid 1 \leqslant j \leqslant \gamma(i),\ f_j \geqslant s_i \text{ és } s_j = s\} \cup \\ \cup \{w_j + v[j] \mid 1 \leqslant j \leqslant \gamma(i),\ f_j \geqslant s_i \text{ és } s_j > s\}). \end{aligned}$$

A $v[i]$ értékek kiszámítását úgy tekinthetjük, mintha egy $n + 2 - m$ elemű tömböt töltenénk ki balról jobbra. A feladat optimális megoldásában a programokért fizetendő részvételi díjak összege $v[n + 1]$.

Minden $v[i]$ érték $O(n)$ lépésben számítható, így a $v[m : n + 1]$ tömb kitöltésének költsége $O(n^2)$. Ehhez jön még a kezdeti rendezés költsége, ami $O(n \log n)$, valamint a $\gamma(i)$ értékek meghatározásának költsége, ami $O(n)$. Így az algoritmus teljes költsége $O(n \log n) + O(n) + O(n^2) = O(n^2)$.

Ha minden $m \leqslant i \leqslant n + 1$ esetén feljegyezzük, hogy $v[i]$ számításánál mely p_j program választása bizonyult a legkedvezőbbnek, akkor magát egy optimális megoldást is könnyen rekonstruálhatunk.

Ha minden program elvi részvételi díja ugyanakkora, akkor a feladat egyszerűen P-beli programok egy olyan minimális elemszámú $\{p_{i_1}, p_{i_2}, \ldots, p_{i_k}\}$ részhalmazának a meghatározása, amelyre

$$[s_{i_1}, f_{i_1}] \cup [s_{i_2}, f_{i_2}] \cup \cdots \cup [s_{i_k}, f_{i_k}] = [s, f].$$

Erre a feladatra a következő mohó algoritmus is optimális megoldást ad. Tegyük fel ismét, hogy a programok a kezdési idejük szerint monoton növekvően rendezettek, vagyis $s_1 \leqslant s_2 \leqslant \cdots \leqslant s_n$.

A kiválasztott programokat egy H halmazban fogjuk tárolni. Kezdetben legyen $H = \emptyset$. Használni fogunk még egy G listát, amely mindig a még nem vizsgált programokat tartalmazza a kezdési idejük szerint monoton növekvően rendezetten. Kezdetben G az összes P-beli programból áll. Először töröljük G-ből azokat a programokat, amelyek kezdési időpontja s, majd ezek közül vegyünk hozzá H-hoz egy legkésőbbi befejezési időpontút. Ismételjük

ezek után a következőt, amíg G-ből nem töröltük az összes programot. Tegyük fel, hogy H-hoz legutóbb az m_j programot vettük hozzá. Ha $f_j = f$, akkor töröljük G-ből az összes programot (ezzel befejeződik az algoritmus). Ha viszont $f_j < f$, akkor töröljük G-ből azokat a programokat, amelyek az f_j időpont előtt vagy éppen az f_j időpontban kezdődnek, majd ezek közül vegyünk hozzá H-hoz egy legkésőbbi befejezési időpontút (jegyezzük meg, hogy ez a program szükségképpen az f_j időpont után fejeződik be).

Legyen $\mathcal{H} = \{p_{j_1}, p_{j_2}, \ldots, p_{j_l}\}$ az algoritmus által kiválasztott programok halmaza, ahol $j_1 < j_2 < \cdots < j_l$. Világos módon

$$[s_{j_1}, f_{j_1}] \cup [s_{j_2}, f_{j_2}] \cup \cdots \cup [s_{j_l}, f_{j_l}] = [s, f].$$

Belátjuk, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak. Tekintsük a feladat egy $\mathcal{O} = \{p_{i_1}, p_{i_2}, \ldots, p_{i_k}\}$ optimális megoldását, ahol $i_1 < i_2 < \cdots < i_k$. Nyilván $l \geqslant k$.

Először teljes indukcióval megmutatjuk, hogy $f_{j_r} \geqslant f_{i_r}$ minden $1 \leqslant r \leqslant k$ esetén. Ha $r = 1$, akkor ez nyilvánvaló, hiszen az algoritmus elsőként az s időpontban kezdődő programok közül egy legnagyobb befejezési időpontút választ ki. Legyen most $r > 1$ és tegyük fel, hogy $f_{j_{r-1}} \geqslant f_{i_{r-1}}$. Mivel $\mathcal{O}$ egy optimális megoldása a feladatnak, ezért $s_{i_r} \leqslant f_{i_{r-1}}$. Másrészt az $f_{j_{r-1}}$ időpont előtt vagy éppen az $f_{j_{r-1}}$ időpontban kezdődő programok közül m_{j_r} egy legkésőbbi befejezési időpontú. Ennélfogva $f_{j_r} \geqslant f_{i_r}$.

Indirekt tegyük fel ezután, hogy $\mathcal{H}$ nem egy optimális megoldása a feladatnak, vagyis $l > k$. Az előzőek szerint $f_{j_k} \geqslant f_{i_k}$. Mivel $\mathcal{O}$ egy optimális megoldása a feladatnak, ezért $f_{i_k} = f$, következésképpen $f_{j_k} = f$ szintén. Ez viszont azt jelenti, hogy az algoritmus az m_{j_k} program kiválasztása után befejeződött, ami ellentmond annak, hogy $l > k$. Ebből következik, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak.

Az algoritmus költsége a kezdeti rendezés nélkül $O(n)$, azzal együtt

$$O(n \log n) + O(n) = O(n \log n).$$

Ütemezés

Egy megbízható vállalkozót sokan keresnek meg különböző munkákkal. Minden munkához tartozik egy elvégzési idő, egy határidő és egy munkadíj. A vállalkozó egyszerre csak egy munkán tud dolgozni és ha egy munkát elkezdett, azt annak befejezéséig nem szakítja meg. Egy munkáért a munkadíj akkor jár, ha az legkésőbb a határidőre elkészül. Mely munkákat vállalja el a vállalkozó és azokat mikor végezze el, ha a bevételét maximalizálni akarja?

Formálisan, adott a munkák $M = \{m_1, m_2, \ldots, m_n\}$ halmaza. Minden m_i munkának adott a t_i végrehajtási ideje és a d_i határideje, ahol t_i és d_i pozitív egész számok (mondjuk napok). Ezen kívül minden m_i munkához adott az érte kapható w_i munkadíj, ahol $w_i > 0$. Feladatunk M-beli munkák egy olyan $\{m_{i_1}, m_{i_2}, \ldots, m_{i_k}\}$ részhalmazának a meghatározása, amelyre a következők teljesülnek. Minden m_{i_j} munkához megadható egy s_{i_j} kezdési és egy f_{i_j} befejezési időpont, ahol s_{i_j} és f_{i_j} pozitív egészek számok (napok), úgy, hogy $f_{i_j} - s_{i_j} + 1 = t_{i_j}$ és $f_{i_j} \leqslant d_{i_j}$, bármely két különböző m_{i_j} és m_{i_h} munkára $f_{i_j} < s_{i_h}$ vagy $f_{i_h} < s_{i_j}$, és ezen feltételek mellett $w_{i_1} + w_{i_2} + \cdots + w_{i_k}$ maximális!

A feladatot dinamikus programozással oldjuk meg. Tegyük fel, hogy a munkák a határidejük szerint monoton növekvően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$d_1 \leqslant d_2 \leqslant \cdots \leqslant d_n.$$

Minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant d_n$ esetén jelölje $v[i, j]$ azon feladat egy optimális megoldásában az összbevételt, amikor az $m_i, m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a j-edik nap kezdhetjük el. A formulák kompakt felírhatóságának érdekében legyen $v[i, j] = 0$, ha $i = n + 1$ vagy $j = d_n + 1$.

Legyen most $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant d_n$ és legyen $\mathcal{O}$ egy optimális megoldása annak a feladatnak, amikor az $m_i, m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a j-edik nap kezdhetjük el. Ha $j + t_i - 1 > d_i$, akkor m_i nyilván nem szerepelhet $\mathcal{O}$-ban, így $\mathcal{O}$ szükségképpen egy optimális megoldása annak a feladatnak, amikor az $m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a j-edik nap kezdhetjük el. Ebben az esetben a bevétel $v[i+1, j]$. Tegyük fel ezután, hogy $j + t_i - 1 \leqslant d_i$.

- Ha m_i szerepel az $\mathcal{O}$-beli munkák között, akkor az az $\mathcal{O}'$ ütemezés, amelyet úgy kapunk $\mathcal{O}$-ból, hogy elhagyjuk az m_i munkát, az m_i elé ütemezett munkákat pedig mind t_i nappal későbbre ütemezzük (a munkáknak a határidők szerinti rendezettsége miatt ezek továbbra is határidőn belül vannak), egy optimális megoldása annak a feladatnak, amikor az $m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a $(j + t_i)$-edik nap kezdhetjük el. Valóban, ha lenne $\mathcal{O}'$-nél nagyobb bevételt hozó ütemezés arra a feladatra, amikor az $m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a $(j + t_i)$-edik nap kezdhetjük el, akkor ehhez hozzávéve az m_i munkát j-edik napi kezdéssel, egy $\mathcal{O}$-nál nagyobb bevételt hozó ütemezést kapnánk arra a feladatra, amikor az $m_i, m_{i+1}, \ldots, m_n$ munkák közül választhatunk és

az első munkát legkorábban a j-edik nap kezdhetjük el, ellentmondás. A bevétel ekkor $w_i + v[i+1, j+t_i]$.

- Ha m_i nem szerepel az $\mathcal{O}$-beli munkák között, akkor $\mathcal{O}$ szükségképpen egy optimális megoldása annak a feladatnak, amikor az $m_{i+1}, \ldots, m_n$ munkák közül választhatunk és az első munkát legkorábban a j-edik nap kezdhetjük el. A bevétel ekkor $v[i+1, j]$.

Mivel nem tudjuk, hogy m_i szerepel-e az $\mathcal{O}$-beli munkák között, ezért mindkét lehetőséget megvizsgáljuk és a kedvezőbbet választjuk. Így

$$v[i,j] = \begin{cases} v[i+1,j] & \text{ha } j+t_i-1 > d_i, \\ \max(w_i + v[i+1, j+t_i], v[i+1,j]) & \text{ha } j+t_i-1 \leqslant d_i. \end{cases}$$

A $v[i,j]$ értékek kiszámítását úgy tekinthetjük, mintha egy $n+1$ sorból és d_n+1 oszlopból álló táblázatot töltenénk ki alulról felfelé, soronként jobbról balra. A feladat optimális megoldásában az összbevétel $v[1,1]$.

Minden $v[i,j]$ érték $O(1)$ lépésben számítható, így a v tömb kitöltésének költsége $O(nd_n)$. Ez azt jelenti, hogy az algoritmus nem polinomiális. Vigasztalásul megjegyezzük, hogy a feladatra nem ismert polinomiális algoritmus.

Ha minden $1 \leqslant i \leqslant n$ és $1 \leqslant j \leqslant d_n$ esetén feljegyezzük, hogy $v[i,j]$ számításánál kedvezőbb volt-e az m_i munkát elvállalni vagy nem, akkor magát egy optimális megoldást is könnyen rekonstruálhatunk (a kiválasztott munkákat az első naptól kezdődően, egymás után, szünet nélkül osszuk be).

Ha minden munkáért ugyanannyi a munkadíj, akkor a feladat egyszerűen az adott feltételeknek eleget tevő munkák egy maximális elemszámú halmazának a meghatározása. Erre a feladatra a következő mohó algoritmus is optimális megoldást ad. Tegyük fel ismét, hogy a munkák a határidejük szerint monoton növekvően rendezettek.

Az elvállalt munkákat egy H halmazban fogjuk tárolni. Kezdetben legyen $H = \emptyset$. Most egymás után, minden $1 \leqslant i \leqslant n$ esetén vegyük hozzá az m_i munkát H-hoz. Ha így

$$\sum_{m_j \in H} t_j > d_i$$

adódik, akkor töröljünk egy leghosszabb elvégzési idejű munkát H-ból. Végül a H-beli munkákat az első naptól kezdve, szünet nélkül, a határidejük szerint monoton növekvő sorrendbe osszuk be. Jelölje $\mathcal{H}$ a munkák ilyen módon történő ütemezését. Jegyezzük meg, hogy a $\mathcal{H}$ ütemezésben minden munka legkésőbb a határidejére elkészül.

A munkák n száma szerinti teljes indukcióval bizonyítjuk, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak. Ha $n = 1$, akkor ez nyilvánvaló. Legyen ezután $n > 1$, és tegyük fel, hogy $n - 1$ munka esetén igaz az állítás.

Ha $\mathcal{H}$ az összes munkát tartalmazza, akkor optimális megoldása a feladatnak világos módon. Tegyük fel ezért, hogy van olyan munka, amely nem szerepel $\mathcal{H}$-ban. Legyen ezek közül m_k az a munka, amelyet először törölt az algoritmus H-ból, mondjuk az m_i munka hozzávételekor.

Megmutatjuk, hogy a feladatnak van olyan $\mathcal{O}$ optimális megoldása, amelyben a munkák az első naptól kezdve, szünet nélkül, a határidejük szerint monoton növekvő sorrendben követik egymást, és amelyben szintén nem szerepel az m_k munka. Az világos, hogy a feladatnak létezik olyan optimális megoldása, amelyben a munkák az első naptól kezdve, szünet nélkül, a határidejük szerint monoton növekvő sorrendben követik egymást. Tegyük fel, hogy egy ilyen $\mathcal{O}'$ optimális megoldásban szerepel az m_k munka. Mivel

$$\sum_{j=1}^{i} t_j > d_i,$$

ezért az $m_1, m_2, \ldots, m_i$ munkák között szükségképpen van olyan m_l munka, amely nem szerepel $\mathcal{O}'$-ben. Legyen $\mathcal{O}$ az az ütemezés, amelyet úgy kapunk $\mathcal{O}'$-ből, hogy az m_k munkát lecseréljük az m_l munkára, majd a munkákat az első naptól kezdve, szünet nélkül, a határidejük szerint monoton növekvő sorrendbe osszuk be. Most az $m_1, m_2, \ldots, m_i$ közül való $\mathcal{O}$-beli munkák legkésőbb a határidejükre mind elkészülnek, hiszen i megválasztása miatt ugyanez akkor is teljesül, ha az összes $m_1, \ldots, m_{k-1}, m_{k+1}, \ldots, m_i$ munkát osztjuk be az első naptól kezdve, szünet nélkül. Másrészt $t_k \geqslant t_l$ miatt az $m_{i+1}, m_{i+2}, \ldots, m_n$ közül való $\mathcal{O}$-beli munkák szintén legkésőbb a határidejükre mind elkészülnek, hiszen semelyik nincs későbbre ütemezve $\mathcal{O}$-ban, mint amikorra $\mathcal{O}'$-ben volt. Ebből következik, hogy $\mathcal{O}$ is egy optimális megoldása a feladatnak.

Nyilván a $\mathcal{H}$-beli munkák száma nem haladhatja meg az $\mathcal{O}$-beli munkák számát, hiszen az utóbbi ütemezés optimális megoldása a feladatnak. Másrészt a mohó algoritmust a munkák $M \setminus \{m_k\}$ halmazára futtatva szintén a $\mathcal{H}$ ütemezést kapjuk, ami az indukciós feltevés szerint optimális a munkák $M \setminus \{m_k\}$ halmazára. Mivel az $\mathcal{O}$ ütemezésben is csak $M \setminus \{m_k\}$ halmazbeli munkák vannak, így az $\mathcal{O}$-beli munkák száma sem haladhatja meg a $\mathcal{H}$-beli munkák számát. Ebből következik, hogy a $\mathcal{H}$-beli munkák száma megegyezik az $\mathcal{O}$-beli munkák számával, következésképpen $\mathcal{H}$ is optimális megoldása az eredeti feladatnak.

A H-beli munkákat az elvégzési idők szerint rendezett maximum kupacban tárolva a soron következő munka beszúrásának valamint egy maximális elvégzési idejű munka esetleges törlésének költsége $O(\log n)$. A törlés szükségessége $O(1)$ lépésben eldönthető, ha egy segédváltozóban nyilvántartjuk a kupacban levő munkák elvégzési idejének összegét. Ezt minden $1 \leqslant i \leqslant n$ esetén végrehajtva az összköltség $O(n \log n)$. Ehhez jön még a kezdeti rendezés $O(n \log n)$ költsége, így az algoritmus teljes költsége $O(n \log n)$.

Ha minden munka végrehajtási ideje 1 nap, akkor a következő mohó algoritmus is optimális megoldást ad. Tegyük fel most, hogy a munkák a munkadíjuk szerint monoton csökkenően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$w_1 \geqslant w_2 \geqslant \cdots \geqslant w_n.$$

Most egymás után minden $1 \leqslant i \leqslant n$ esetén ütemezzük be az m_i munkát a d_i-edik illetve az azt megelőző napok közül a legkésőbbi olyanra, amely még szabad. Ha az összes ilyen nap foglalt már, akkor a munkát ne vállaljuk el. Jelölje $\mathcal{H}$ a munkák ilyen módon történő ütemezését.

Először belátjuk, hogy $\mathcal{H}$ egy optimális megoldása a feladatnak. Tekintsük a munkák egy optimális $\mathcal{O}$ ütemezését. Ha $\mathcal{O} = \mathcal{H}$, akkor készen vagyunk. Tegyük fel ezért, hogy $\mathcal{O} \neq \mathcal{H}$. Legyen k a legkisebb olyan index, amelyre m_k ütemezése különbözik $\mathcal{O}$-ban és $\mathcal{H}$-ban. Ekkor minden $j < k$ esetén az m_j munka

- vagy nem szerepel sem $\mathcal{O}$-ban sem $\mathcal{H}$-ban,
- vagy szerepel $\mathcal{O}$-ban és $\mathcal{H}$-ban is, mindkétszer ugyanarra a napra ütemezve.

Négy esetet különböztethetünk meg.

(1) Az m_k munka szerepel $\mathcal{O}$-ban, de nem szerepel $\mathcal{H}$-ban. Mivel m_k szerepel $\mathcal{O}$-ban, ezért a $\mathcal{H}$-t előállító algoritmusunk k-adik lépésében a d_k-adik illetve az azt megelőző napok között szükségképpen van még szabad nap. Ám ez ellentmond annak, hogy $\mathcal{H}$-ból kihagytuk m_k-t.

(2) Az m_k munka szerepel $\mathcal{H}$-ban, de nem szerepel $\mathcal{O}$-ban. Ha arra a napra, amelyre $\mathcal{H}$ ütemezi m_k-t $\mathcal{O}$ nem ütemez egyetlen munkát sem, akkor m_k-t vegyük hozzá $\mathcal{O}$-hoz erre a napra ütemezve. Ezzel a bevétel nyilván nem csökken. Ha viszont arra a napra, amelyre $\mathcal{H}$ ütemezi m_k-t $\mathcal{O}$ is ütemez egy m_j munkát, akkor $\mathcal{O}$-ban cseréljük le m_j-t m_k-ra. Mivel $j > k$, ezért $w_j \leqslant w_k$, következésképpen a bevétel most sem csökken.

(3) Az m_k munka szerepel $\mathcal{O}$-ban és $\mathcal{H}$-ban is, de az előbbiben korábbi napra van ütemezve, mint az utóbbiban. Ha arra a napra, amelyre $\mathcal{H}$ ütemezi m_k-t $\mathcal{O}$ nem ütemez egyetlen munkát sem, akkor m_k-t ütemezzük át $\mathcal{O}$-ban erre a napra. Ezzel a bevétel nem változik. Ha viszont arra a napra, amelyre $\mathcal{H}$ ütemezi m_k-t $\mathcal{O}$ is ütemez egy m_j munkát, akkor $\mathcal{O}$-ban cseréljük fel m_j-t és m_k-t. Mivel m_j így előbbre kerül, a csere megengedett. A bevétel nyilván most sem változik.

(4) Az m_k munka szerepel $\mathcal{O}$-ban és $\mathcal{H}$-ban is, de az utóbbiban korábbi napra van ütemezve, mint az előbbiben. Ez ellentmond annak, hogy a $\mathcal{H}$-t előállító algoritmusunk a k-adik lépésben az m_k munkát a d_k-adik illetve az azt megelőző napok között a legkésőbbi, még szabad napra ütemezte.

Ezzel beláttuk, hogy az optimális $\mathcal{O}$ ütemezés áttranszformálható egy olyan optimális $\mathcal{O}'$ ütemezéssé, hogy $\mathcal{O}'$-ben és $\mathcal{H}$-ban már m_k ütemezése is megegyezik. Az eljárást folytatva $\mathcal{O}$ lépésről-lépésre áttranszformálható $\mathcal{H}$-ba a bevétel csökkenése nélkül. Következésképpen $\mathcal{H}$ is egy optimális ütemezés.

Az algoritmus költsége nagyban függ attól, hogy a k-adik lépésben hogyan találjuk meg a d_k-adik illetve az azt megelőző napok közül a legkésőbbi olyat, amely még szabad. Kézenfekvő ötlet, hogy a d_k-adik naptól egyesével haladjunk visszafelé, ekkor a költség nyilván $O(n)$. Ezt minden $1 \leqslant k \leqslant n$ esetén végrehajtva az összköltség $O(n^2)$. Ehhez jön még a kezdeti rendezés $O(n \log n)$ költsége, így az algoritmus teljes költsége $O(n^2)$.

Megjegyezzük, hogy a szakasz elején ismertetett dinamikus programozás algoritmussal is meghatározhatjuk a feladat egy optimális megoldását $O(n^2)$ lépésben. Ehhez elég észrevenni, hogy a feladatnak mindig van olyan optimális megoldása, amelyben az összes elvállalt munkát legkésőbb az n-edik nap elvégezzük, hiszen az üres napokat meg tudjuk szüntetni a feltételek megsértése nélkül a munkák alkalmas előbbre ütemezésével. Ebből következik, hogy ha minden olyan munkának a határidejét az n-edik napra hozzuk előre, amelyiknek a határideje az n-edik nap után van, akkor az így módosított feladat egy optimális megoldásában az összbevétel nyilván ugyanannyi lesz, mint az eredeti feladat esetén. Erre a módosított feladatra futtatva a dinamikus programozás algoritmust, a fenti költség adódik.

Az UNIÓ-HOLVAN adatszerkezet használatával lényegesen hatékonyabban is megvalósítható a mohó algoritmus. Először is hozzuk előre az n-edik napra minden olyan munkának a határidejét, amelyiknek a határideje az n-edik nap után van. Tartsuk ezután nyilván az egymás utáni foglalt napok maximális halmazait (amelyeket a közvetlenül előttük lévő szabad nappal azonosítunk), valamint a szabad napokat, mint egyelemű halmazokat. Az algoritmus k-adik lépésében ellenőrizzük, hogy a d_k-adik nap szabad-e, illetve

ha nem, akkor a foglalt napok mely blokkjához tartozik. Az első esetben m_k-t a d_k-adik napra ütemezzük, a másodikban a blokkot közvetlenül megelőző szabad napra (illetve eldobjuk, ha nincs a blokk előtt szabad nap). Ha az a nap, amelyre m_k-t ütemeztük, foglalt napok blokkjaival szomszédos, akkor képezzük a blokkok unióját. A költség így csupán $O(\log n)$. Az algoritmus ezen implementációjának teljes költsége ennélfogva $O(n \log n)$.

Végül jegyezzük meg, hogy ha minden munkának ugyanaz a határideje, jelölje ezt mondjuk D, akkor a feladat az M-beli munkák egy olyan $\{m_{i_1}, m_{i_2}, \ldots, m_{i_k}\}$ részhalmazának a meghatározása, amelyre $t_{i_1} + t_{i_2} + \cdots + t_{i_k} \leqslant D$ és $w_{i_1} + w_{i_2} + \cdots + w_{i_k}$ maximális. Ez a jól ismert hátizsák probléma, amelyre szintén nem ismert polinomiális algoritmus.

Visszajáró pénz

A visszajáró pénz problémánál adott pénzmennyiséget kell kifizetni a lehető legkevesebb érme felhasználásával. Legyen a kifizetendő pénzmennyiség $n \in \mathbb{N}$ és legyenek a felhasználható címletek $c_1, c_2, \ldots, c_m$, ahol

$$c_1 > c_2 > \cdots > c_m = 1$$

pozitív egész számok. Tegyük fel, hogy minden címletből tetszőlegesen sok érme áll a rendelkezésünkre. Ekkor bármely pénzmennyiség kifizethető, hiszen a címletek között az 1 is szerepel.

A feladatot dinamikus programozással oldjuk meg. Minden $0 \leqslant j \leqslant n$ esetén jelölje $v[j]$ a kifizetésben szereplő érmék minimális számát, amikor a kifizetendő pénzmennyiség j. Ha $j = 0$, akkor $v[j] = 0$ nyilvánvaló módon. Legyen ezután $1 \leqslant j \leqslant n$. Tekintsük most egy legkevesebb érmét felhasználó megoldását annak a feladatnak, amikor a kifizetendő pénzmennyiség j. Legyen a megoldásban felhasznált érmék száma s. Ha most ezek közül elhagyunk egyet, mondjuk egy c_i címletűt, akkor a megmaradt $s - 1$ érme egy legkevesebb érmét felhasználó megoldását adja annak a feladatnak, amikor a kifizetendő pénzmennyiség $j - c_i$. Valóban, ha ez utóbbi pénzmennyiség kifizethető lenne $(s-1)$-nél kevesebb érmével is, akkor ezeket az érméket az elhagyott c_i címletű érmével kiegészítve az eredeti feladat egy s-nél kevesebb érmét felhasználó megoldásához jutnánk, ami ellentmondás. Ebből következik, hogy esetünkben $v[j] = 1 + v[j - c_i]$. Ez a rekurzív egyenlet feltételezi, hogy ismerjük i értékét, miközben ez nem igaz. Azonban i legfeljebb m különböző értéket vehet fel; vizsgáljuk meg az összes lehetőséget, és válasszuk ki a legkedvezőbbet. Így

$$v[j] = 1 + \min\{v[j - c_i] \mid 1 \leqslant i \leqslant m,\ c_i \leqslant j\}.$$

A $v[j]$ értékek kiszámítását úgy tekinthetjük, mintha egy $n+1$ elemű tömböt töltenénk ki balról jobbra. A feladat optimális megoldásában az érmék száma $v[n]$.

Minden $v[j]$ érték $O(m)$ lépésben számítható, így a $v[0:n]$ tömb kitöltésének költsége $O(mn)$. Ez azt jelenti, hogy az algoritmus nem polinomiális. Vigasztalásul megjegyezzük, hogy a feladatra nem ismert polinomiális algoritmus.

Ha minden $1 \leqslant j \leqslant n$ esetén feljegyzünk egy olyan c_i címletet is, amelyre $v[j] = 1 + v[j - c_i]$, akkor magát egy legkevesebb érméből álló kifizetést is könnyen rekonstruálhatunk.

A legtöbb ismert pénzrendszer esetén a következő mohó algoritmus is optimális megoldást ad: amíg $n > 0$, addig keressük meg azt a legnagyobb c_i címletet, amely kisebb vagy egyenlő, mint n, adjunk ki egy c_i címletű érmét, majd oldjuk meg rekurzívan azt a feladatot, amikor a kifizetendő pénzmennyiség $n - c_i$. Az algoritmus költsége $O(m)$; egymás után egy-egy osztással meghatározhatjuk a szükséges c_i címletű érmék számát minden $1 \leqslant i \leqslant m$ esetén.

Vannak azonban kivételek. Az 1971 előtti angol pénzrendszerben a címletek penny-ben kifejezve a következők voltak: 1, 3, 6, 12, 24, 30, 60, 240 (igazság szerint forgalomban volt még félpenny is, de ez nem játszik szerepet a példánkban). Itt a mohó algoritmussal 48 penny-t 3 érmével fizetnénk ki: $30 + 12 + 6$. Ezzel szemben az optimális megoldás csak két érméből áll: $24 + 24$.

Melyek azok a pénzrendszerek, amelyekre működik a mohó algoritmus és melyek azok, amelyekre nem? Ez a kérdés meglepően hatékony módon eldönthető. Az algoritmus ismertetése előtt némi előkészület szükséges.

A címletek monoton csökkenő sorozatára vezessük be a

$$\boldsymbol{c} = (c_1, c_2, \ldots, c_m)$$

jelölést. Adott $n \in \mathbb{N}$ esetén egy

$$\boldsymbol{r} = (r_1, r_2, \ldots, r_m) \in \mathbb{N}^m$$

sorozatot az n egy $\boldsymbol{c}$-beli reprezentációjának fogunk nevezni, ha

$$\boldsymbol{r}\boldsymbol{c} = r_1c_1 + r_2c_2 + \cdots + r_mc_m = n.$$

Vezessük még be az $|\boldsymbol{r}| = r_1 + r_2 + \cdots + r_m$ jelölést, erre a mennyiségre az $\boldsymbol{r}$ reprezentáció méreteként fogunk hivatkozni. A visszajáró pénz feladat ebben

a megfogalmazásban adott $n \in \mathbb{N}$ egy minimális méretű $\boldsymbol{c}$-beli reprezentációjának a megtalálása.

Definiáljuk az $\mathbb{N}^m$-beli sorozatok halmazán a $\preccurlyeq_1$, $\preccurlyeq_2$, $\preccurlyeq_3$ rendezési relációkat a következőképpen. Legyenek $\boldsymbol{x} = (x_1, x_2, \ldots, x_m) \in \mathbb{N}^m$ és $\boldsymbol{y} = (y_1, y_2, \ldots, y_m) \in \mathbb{N}^m$. Azt fogjuk mondani, hogy

- $\boldsymbol{x} \preccurlyeq_1 \boldsymbol{y}$, ha vagy $\boldsymbol{x} = \boldsymbol{y}$ vagy pedig $x_i < y_i$ a legkisebb olyan $1 \leqslant i \leqslant m$ indexre, amelyre $x_i \neq y_i$,
- $\boldsymbol{x} \preccurlyeq_2 \boldsymbol{y}$, ha vagy $|\boldsymbol{x}| > |\boldsymbol{y}|$ vagy pedig $|\boldsymbol{x}| = |\boldsymbol{y}|$ és $\boldsymbol{x} \preccurlyeq_1 \boldsymbol{y}$,
- $\boldsymbol{x} \preccurlyeq_3 \boldsymbol{y}$, ha $x_i \leqslant y_i$ minden $1 \leqslant i \leqslant m$ esetén.

Megjegyezzük, hogy tetszőleges $\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z} \in \mathbb{N}^m$ sorozatokra

$$\boldsymbol{x} \preccurlyeq_k \boldsymbol{y} \iff \boldsymbol{x} + \boldsymbol{z} \preccurlyeq_k \boldsymbol{y} + \boldsymbol{z}$$

mindhárom rendezési reláció esetén.

A továbbiakban, ha ez nem okoz félreértést, a $\boldsymbol{c}$-beli reprezentáció kifejezés elejéről el fogjuk hagyni a $\boldsymbol{c}$-beli jelzőt. Adott n esetén jelölje $\boldsymbol{g}(n)$ a $\preccurlyeq_1$ rendezés szerint maximális reprezentációját n-nek. A $\boldsymbol{g}(n)$ sorozatra az n mohó reprezentációjaként fogunk hivatkozni. Vegyük észre, hogy ha $\boldsymbol{g}(n) = (g_1, g_2, \ldots, g_m)$, akkor a visszajáró pénz problémára adott mohó algoritmus éppen g_i darab c_i címletű érmét használ n kifizetéséhez minden $1 \leqslant i \leqslant m$ esetén. Jelölje továbbá adott n esetén $\boldsymbol{h}(n)$ a $\preccurlyeq_2$ rendezés szerint maximális reprezentációját n-nek. A $\boldsymbol{h}(n)$ sorozatra az n minimális reprezentációjaként fogunk hivatkozni. Jegyezzük meg, hogy $\boldsymbol{h}(n)$ az n minimális méretű reprezentációi közül a $\preccurlyeq_1$ rendezés szerint maximális.

Azt fogjuk mondani, hogy egy $\boldsymbol{c}$ pénzrendszer kanonikus, ha $\boldsymbol{g}(n) = \boldsymbol{h}(n)$ minden $n \in \mathbb{N}$ esetén. Ez azt jelenti, hogy a visszajáró pénz problémára adott mohó algoritmus az adott $\boldsymbol{c}$ pénzrendszert használva mindig optimális megoldást ad. Célunk ebben a megfogalmazásban a kanonikus pénzrendszerek azonosítása.

1. Állítás. Legyenek $n', n'' \in \mathbb{N}$. Ha $n' < n''$, akkor $\boldsymbol{g}(n') \prec_1 \boldsymbol{g}(n'')$.

Bizonyítás. Ha $n' < n''$, akkor $\boldsymbol{r} = \boldsymbol{g}(n') + (0, 0, \ldots, 0, n'' - n')$ egy reprezentációja n''-nek. Most egyrészt $\boldsymbol{g}(n') \prec_1 \boldsymbol{r}$ nyilvánvaló módon, másrészt $\boldsymbol{r} \preccurlyeq_1 \boldsymbol{g}(n'')$ a mohó reprezentáció definíciójával összhangban. Így $\boldsymbol{g}(n') \prec_1 \boldsymbol{g}(n'')$.

2. Állítás. Legyenek $n', n'' \in \mathbb{N}$, és legyen $\boldsymbol{r'}$ egy reprezentációja n'-nek, valamint $\boldsymbol{r''}$ egy reprezentációja n''-nek. Tegyük fel, hogy $\boldsymbol{r'} \preccurlyeq_3 \boldsymbol{r''}$.

(1) Ha $\boldsymbol{r''}$ mohó reprezentációja n''-nek, akkor $\boldsymbol{r'}$ is mohó reprezentációja n'-nek.

(2) Ha $\boldsymbol{r''}$ minimális reprezentációja n''-nek, akkor $\boldsymbol{r'}$ is minimális reprezentációja n'-nek.

Bizonyítás. Egy általános észrevétellel kezdjük a bizonyítást. Legyen $\boldsymbol{r}$ egy tetszőleges reprezentációja n'-nek. Ekkor $\boldsymbol{rc} = \boldsymbol{r'c}$. Innen $\boldsymbol{rc} + \boldsymbol{r''c} = \boldsymbol{r'c} + \boldsymbol{r''c}$, illetve átrendezve $\boldsymbol{r''c} - \boldsymbol{r'c} + \boldsymbol{rc} = \boldsymbol{r''c}$, majd kiemelve $(\boldsymbol{r''} - \boldsymbol{r'} + \boldsymbol{r})\boldsymbol{c} = \boldsymbol{r''c}$. Feltételünk szerint $\boldsymbol{r'} \preccurlyeq_3 \boldsymbol{r''}$, így $\boldsymbol{r''} - \boldsymbol{r'} + \boldsymbol{r} \in \mathbb{N}^m$, következésképpen $\boldsymbol{r''} - \boldsymbol{r'} + \boldsymbol{r}$ is egy reprezentációja n''-nek. Lássuk ezután a két állítás nagyon hasonló bizonyítását.

(1) Mivel $\boldsymbol{r''}$ mohó reprezentációja n''-nek, ezért $\boldsymbol{r''} - \boldsymbol{r'} + \boldsymbol{r} \preccurlyeq_1 \boldsymbol{r''}$. Ebből következik, hogy $\boldsymbol{r''} + \boldsymbol{r} \preccurlyeq_1 \boldsymbol{r''} + \boldsymbol{r'}$, ahonnan $\boldsymbol{r} \preccurlyeq_1 \boldsymbol{r'}$ adódik. Ez viszont csak akkor lehetséges n' teteszőleges $\boldsymbol{r}$ reprezentációjára, ha $\boldsymbol{r'}$ mohó reprezentációja n'-nek.

(2) Mivel $\boldsymbol{r''}$ minimális reprezentációja n''-nek, ezért $\boldsymbol{r''} - \boldsymbol{r'} + \boldsymbol{r} \preccurlyeq_2 \boldsymbol{r''}$. Ebből következik, hogy $\boldsymbol{r''} + \boldsymbol{r} \preccurlyeq_2 \boldsymbol{r''} + \boldsymbol{r'}$, ahonnan $\boldsymbol{r} \preccurlyeq_2 \boldsymbol{r'}$ adódik. Ez viszont csak akkor lehetséges n' teteszőleges $\boldsymbol{r}$ reprezentációjára, ha $\boldsymbol{r'}$ minimális reprezentációja n'-nek.

Tegyük fel ezek után, hogy egy $\boldsymbol{c}$ pénzrendszer nem kanonikus. Legyen n a legkisebb olyan természetes szám, amelyre $\boldsymbol{g}(n) \neq \boldsymbol{h}(n)$. Legyen $\boldsymbol{g}(n) = (g_1, g_2, \ldots, g_m)$ és $\boldsymbol{h}(n) = (h_1, h_2, \ldots, h_m)$. Megmutatjuk, hogy minden $1 \leqslant i \leqslant m$ esetén g_i és h_i közül legalább az egyik nulla. Indirekt tegyük fel, hogy ez nem teljesül, azaz valamely $1 \leqslant i \leqslant m$ esetén g_i és h_i is pozitív. Ekkor

$$\boldsymbol{r'} = (g_1, \ldots, g_{i-1}, g_i - 1, g_{i+1}, \ldots, g_m) \in \mathbb{N}^m$$

és

$$\boldsymbol{r''} = (h_1, \ldots, h_{i-1}, h_i - 1, h_{i+1}, \ldots, h_m) \in \mathbb{N}^m$$

az n-nél kisebb $n - c_i \in \mathbb{N}$ szám olyan reprezentációi, amelyekre $\boldsymbol{r'} \prec_3 \boldsymbol{g}(n)$ és $\boldsymbol{r''} \prec_3 \boldsymbol{h}(n)$, így a 2. Állítás szerint $\boldsymbol{r'} = \boldsymbol{g}(n - c_i)$ és $\boldsymbol{r''} = \boldsymbol{h}(n - c_i)$. Ez viszont azt jelenti, hogy $\boldsymbol{g}(n - c_i) \neq \boldsymbol{h}(n - c_i)$, ellentmondva n minimalitásának.

Az n szám minimális reprezentációjában legyen h_α az első és h_β az utolsó pozitív tag. Ekkor az előzőekkel összhangban az n szám mohó reprezentációjában $g_\alpha = 0$. Ám $\boldsymbol{h}(n) \prec_1 \boldsymbol{g}(n)$ miatt ekkor g_i szükségképpen pozitív valamely $i < \alpha$ esetén. Ebből rögtön adódik, hogy $\alpha > 1$.

Meglepő kapcsolat áll fenn n minimális és $c_{\alpha-1}-1$ mohó reprezentációja között. Legyen $\boldsymbol{g}(c_{\alpha-1}-1)=(f_1,f_2,\ldots,f_m)$. Ekkor

$$h_i=\begin{cases} f_i & \text{ha } i<\beta,\\ f_i+1 & \text{ha } i=\beta,\\ 0 & \text{ha } i>\beta.\end{cases}$$

Ez a következőképpen látható be. Mivel g_i pozitív valamely $i<\alpha$ esetén, ezért $n\geqslant c_{\alpha-1}$. Másrészt a 2. Állítással összhangban

$$(h_1,\ldots,h_{\beta-1},h_\beta-1,h_{\beta+1},\ldots,h_m)=\boldsymbol{h}(n-c_\beta).$$

Ugyanakkor $\boldsymbol{h}(n-c_\beta)=\boldsymbol{g}(n-c_\beta)$, hiszen $n-c_\beta<n$ és feltételünk szerint n a legkisebb olyan természetes szám, amelynek minimális és mohó reprezentációja különböző. Felhasználva, hogy $h_i=0$ minden $i<\alpha$ esetén, ebből $n-c_\beta<c_{\alpha-1}$ és így $n-c_\beta\leqslant c_{\alpha-1}-1$ azonnal adódik. Ennélfogva az 1. Állítás szerint

$$\boldsymbol{g}(n-c_\beta)\preccurlyeq_1\boldsymbol{g}(c_{\alpha-1}-1).$$

Tekintsük most a $c_{\alpha-1}-1$ számot. Mivel $c_{\alpha-1}-1\geqslant c_\alpha$, ezért $c_{\alpha-1}-1$ mohó reprezentációjában $f_\alpha\geqslant 1$. Ismét a 2. Állítással összhangban

$$(f_1,\ldots,f_{\alpha-1},f_\alpha-1,f_{\alpha+1},\ldots,f_m)=\boldsymbol{g}(c_{\alpha-1}-1-c_\alpha).$$

Másrészt

$$(h_1,\ldots,h_{\alpha-1},h_\alpha-1,h_{\alpha+1},\ldots,h_m)=\boldsymbol{h}(n-c_\alpha)$$

szintén a 2. Állítás szerint. Ennélfogva

$$(h_1,\ldots,h_{\alpha-1},h_\alpha-1,h_{\alpha+1},\ldots,h_m)=\boldsymbol{g}(n-c_\alpha),$$

hiszen $n-c_\alpha<n$ és feltételünk szerint n a legkisebb olyan természetes szám, amelynek minimális és mohó reprezentációja különböző. Idézzük fel, hogy $c_{\alpha-1}\leqslant n$, következésképpen $c_{\alpha-1}-1-c_\alpha<n-c_\alpha$, így az 1. Állítás szerint $\boldsymbol{g}(c_{\alpha-1}-1-c_\alpha)\prec_1\boldsymbol{g}(n-c_\alpha)$. Mindkét sorozat α-adik tagját eggyel megnövelve, az előbbiek figyelembe vételével,

$$\boldsymbol{g}(c_{\alpha-1}-1)\prec_1\boldsymbol{h}(n)$$

adódik.

Végül tekintsük a

$$\boldsymbol{g}(n-c_\beta)\preccurlyeq_1\boldsymbol{g}(c_{\alpha-1}-1)\prec_1\boldsymbol{h}(n)$$

láncot. Mivel a $\boldsymbol{g}(n-c_\beta)$ és a $\boldsymbol{h}(n)$ sorozatok csak a β-adik tagban különböznek, ezért $f_i=h_i$ minden $i<\beta$ esetén. Ugyanakkor $\boldsymbol{g}(c_{\alpha-1}-1)\prec_1\boldsymbol{h}(n)$

miatt $f_j < h_j$ valamely $j \geqslant \beta$ indexre. Ám $h_i = 0$ minden $i > \beta$ esetén, ezért $j = \beta$ lehetséges csak, ennélfogva $f_\beta < h_\beta$. Másrészt $\boldsymbol{g}(n - c_\beta) \preccurlyeq_1 \boldsymbol{g}(c_{\alpha-1} - 1)$ miatt $h_\beta - 1 \leqslant f_\beta$, így $h_\beta - 1 \leqslant f_\beta < h_\beta$, ahonnan $h_\beta = f_\beta + 1$ következik.

Ezek után egy $\boldsymbol{c}$ pénzrendszerről már könnyen eldönthető, hogy kanonikus vagy nem. Minden $2 \leqslant \alpha \leqslant \beta \leqslant m$ esetén állítsuk elő a $\boldsymbol{g}(c_{\alpha-1} - 1) = (f_1, f_2, \ldots, f_m)$ sorozatot a mohó algoritmussal, illetve az $\boldsymbol{r} = (r_1, r_2, \ldots, r_m)$ sorozatot, ahol

$$r_i = \begin{cases} f_i & \text{ha } i < \beta, \\ f_i + 1 & \text{ha } i = \beta, \\ 0 & \text{ha } i > \beta. \end{cases}$$

Ha $\boldsymbol{c}$ nem kanonikus, akkor a fentiek szerint az így kapott $\boldsymbol{r}$ sorozatok között találni fogunk olyat, amelyre $|\boldsymbol{g}(\boldsymbol{rc})| > |\boldsymbol{r}|$. Mivel a szóba jövő $\boldsymbol{r}$ sorozatok száma $O(m^2)$, és minden sorozatról $O(m)$ lépésben eldönthető ennek a tulajdonság teljesülése, ezért az algoritmus teljes lépésszáma $O(m^3)$.

Az algoritmus alkalmazásával egyszerűen igazolható, hogy tetszőleges $m \geqslant 2$ és $q \geqslant 2$ egészekre kanonikus az a $\boldsymbol{c} = (c_1, c_2, \ldots, c_m)$ pénzrendszer, ahol $c_i = q^{m-i}$ minden $1 \leqslant i \leqslant m$ esetén. Legyen $2 \leqslant \alpha \leqslant m$ tetszőleges és tekintsük a $\boldsymbol{g}(c_{\alpha-1} - 1) = (f_1, f_2, \ldots, f_m)$ sorozatot. Világos módon

$$f_i = \begin{cases} 0 & \text{ha } i < \alpha, \\ q - 1 & \text{ha } i \geqslant \alpha. \end{cases}$$

Legyen ezután $\alpha \leqslant \beta \leqslant m$ és tekintsük azt az $\boldsymbol{r} = (r_1, r_2, \ldots, r_m)$ sorozatot, ahol

$$r_i = \begin{cases} f_i & \text{ha } i < \beta, \\ f_i + 1 & \text{ha } i = \beta, \\ 0 & \text{ha } i > \beta, \end{cases}$$

vagyis

$$r_i = \begin{cases} 0 & \text{ha } i < \alpha, \\ q - 1 & \text{ha } \alpha \leqslant i < \beta, \\ q & \text{ha } i = \beta, \\ 0 & \text{ha } i > \beta. \end{cases}$$

Vegyük most észre, hogy $\boldsymbol{r}$ éppen a $c_{\alpha-1} = q^{m-\alpha+1}$ szám egy reprezentációja. Valóban

$$\begin{aligned} \boldsymbol{rc} &= (q-1)q^{m-\alpha} + (q-1)q^{m-\alpha-1} + \cdots + (q-1)q^{m-\beta+1} + qq^{m-\beta} \\ &= (q-1)q^{m-\alpha} + (q-1)q^{m-\alpha-1} + \cdots + (q-1)q^{m-\beta+1} + q^{m-\beta+1} \\ &= (q-1)q^{m-\alpha} + (q-1)q^{m-\alpha-1} + \cdots + qq^{m-\beta+1} \\ &= (q-1)q^{m-\alpha} + (q-1)q^{m-\alpha-1} + \cdots + q^{m-\beta+2} \\ &\vdots \end{aligned}$$

$$
\begin{aligned}
&\vdots \\
&= (q-1)q^{m-\alpha} + qq^{m-\alpha-1} \\
&= (q-1)q^{m-\alpha} + q^{m-\alpha} \\
&= qq^{m-\alpha} \\
&= q^{m-\alpha+1}.
\end{aligned}
$$

De $|\boldsymbol{g}(c_{\alpha-1})| = 1$, így szükségképpen $|\boldsymbol{r}| \geqslant |\boldsymbol{g}(c_{\alpha-1})|$. Mivel ez tetszőleges $2 \leqslant \alpha \leqslant \beta \leqslant m$ esetén fennáll, ezért a vizsgált $\boldsymbol{c}$ pénzrendszer kanonikus.

Maximális folyamok

Hálózatok és folyamok

Legyen adott egy $G = (V, E)$ irányított gráf, és ennek két különböző csúcsa s és t, melyeket forrásnak és nyelőnek hívunk. Az egyszerűség kedvéért tegyük fel, hogy G-ben nincsenek izolált csúcsok (így többek között $|V| = O(|E|)$). Legyen adott még egy, az éleken értelmezett $c\colon E \to \mathbb{R}_0^+$ nemnegatív értékű kapacitás függvény. A kényelem kedvéért terjesszük ki c-t az összes rendezett csúcspárra úgy, hogy minden $u, v \in V$ és $(u, v) \notin E$ esetén legyen $c(u, v) = 0$. Ekkor a (G, s, t, c) rendezett négyest hálózatnak nevezzük.

Egy (G, s, t, c) hálózaton egy $f\colon V \times V \to \mathbb{R}$ függvényt folyamnak hívunk, ha a következő feltételek teljesülnek:

(1) minden $u, v \in V$ esetén $f(u, v) = -f(v, u)$,

(2) minden $u \in V \setminus \{s, t\}$ esetén

$$\sum_{v \in V} f(u, v) = 0,$$

(3) minden $u, v \in V$ esetén $f(u, v) \leqslant c(u, v)$.

Az f folyam értékén az

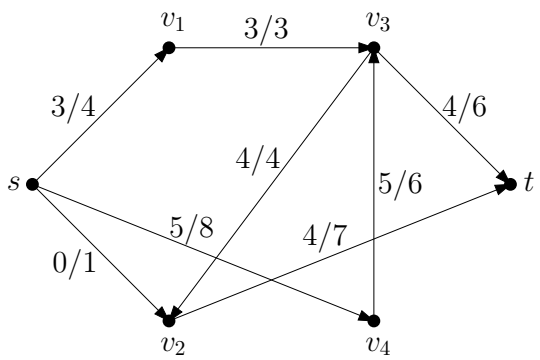
$$|f| = \sum_{v \in V} f(s, v)$$

mennyiséget értjük.

Jegyezzük meg, hogy ha (u, v) és (v, u) egyike sem éle a hálózatnak, akkor (1) és (3) miatt

$$f(u, v) = f(v, u) = 0.$$

Emiatt egy f folyam megadásához elegendő azon $f(u, v)$ értékek ismerete, amelyekre $(u, v) \in E$.



$|f| = 8$

A feladat a következő: adott (G, s, t, c) hálózathoz találjunk egy maximális értékű f folyamot!

Ford-Fulkerson algoritmus

Legyen f egy folyam a $(G = (V, E), s, t, c)$ hálózaton. Definiáljuk a c_f reziduális kapacitás függvényt a következőképpen:

$$c_f \colon V \times V \to \mathbb{R}, \quad (u, v) \mapsto c(u, v) - f(u, v).$$

A folyam növelésére nyilván azon élek mentén van esély, amelyeken c_f pozitív értéket vesz fel. Tekintsük ezért azt a $(G_f = (V, E_f), s, t, c_f)$ hálózat, ahol

$$E_f = \{(u, v) \in V \times V \mid c_f(u, v) > 0\}.$$

Ezt a hálózatot az f folyamhoz tartozó reziduális hálózatnak nevezzük. Jegyezzük meg, hogy a (G_f, s, t, c_f) reziduális hálózatban lehetnek olyan élek, amelyek nem élei a (G, s, t, c) hálózatnak. Mindazonáltal egy (u, v) él csak akkor szerepelhet a reziduális hálózatban, ha (u, v) és (v, u) legalább egyike éle az eredeti hálózatnak.

Nem nehéz belátni a következőt. Legyen f egy folyam a (G, s, t, c) hálózaton, f' pedig egy folyam a (G_f, s, t, c_f) reziduális hálózaton. Ekkor $f + f'$ egy $|f| + |f'|$ értékű folyam a (G, s, t, c) hálózaton.

Ha tehát találunk egy pozitív értékű f' folyamot a reziduális hálózatban, akkor $f + f'$ egy, az f-nél nagyobb értékű folyam lesz az eredeti hálózatban. Legyen f egy folyam a (G, s, t, c) hálózaton, (G_f, s, t, c_f) pedig az f-hez tartozó reziduális hálózat. A reziduális hálózat s-ből t-be vezető egyszerű útjait javító utaknak nevezzük. Egy P javító útra jelölje $c_f(P)$ az úton szereplő élek reziduális kapacitásainak minimumát. Tekintsük most az

$$f_P \colon V \times V \to \mathbb{R}, \quad (u, v) \mapsto \begin{cases} c_f(P) & \text{ha } (u, v) \text{ éle } P\text{-nek}, \\ -c_f(P) & \text{ha } (v, u) \text{ éle } P\text{-nek}, \\ 0 & \text{különben} \end{cases}$$

függvényt. Világos, hogy f_P egy $c_f(P)$ értékű folyam a reziduális hálózaton. Az előzőek szerint ezt a folyamot f-hez hozzáadva egy f-nél nagyobb értékű folyamot kapunk a (G, s, t, c) hálózaton. A Ford-Fulkerson algoritmus az azonosan nulla folyamból kiindulva lépésenként növeli a folyam értékét a javító utak mentén mindaddig, amíg a reziduális hálózatban található javító út. Mi történik ha a reziduális hálózatban már nem található javító út? Megmutatható, hogy ebben az esetben készen vagyunk: a folyam már maximális. A bizonyítás felhasználja a vágás fogalmát.

Legyen (G, s, t, c) egy hálózat, (S, T) pedig G csúcsainak egy olyan partíciója, hogy $s \in S$ és $t \in T$. (Partíció alatt azt értjük, hogy $S \cup T = V$ és $S \cap T = \emptyset$.) Ekkor az (S, T) halmazpárt a hálózat egy vágásának nevezzük. Az (S, T) vágás kapacitásán a

$$c(S, T) = \sum_{u \in S,\, v \in T} c(u, v)$$

mennyiséget értjük. Ha f egy folyam a (G, s, t, c) hálózaton, akkor az (S, T) vágáson áthaladó folyam értékét definiáljuk a következőképpen:

$$f(S, T) = \sum_{u \in S,\, v \in T} f(u, v).$$

Terjesszük ki az utóbbi jelölést: tetszőleges $X, Y \subseteq V$ esetén legyen

$$f(X, Y) = \sum_{x \in X,\, y \in Y} f(x, y).$$

Könnyű látni, hogy teljesülnek az alábbiak.

(1) Minden $X \subseteq V$ esetén $f(X, X) = 0$.

(2) Minden $X, Y \subseteq V$ esetén $f(X, Y) = -f(Y, X)$.

(3) Minden $X, Y, Z \subseteq V$ és $X \cap Y = \emptyset$ esetén

$$f(X \cup Y, Z) = f(X, Z) + f(Y, Z),$$
$$f(Z, X \cup Y) = f(Z, X) + f(Z, Y).$$

Állítás. Egy (G, s, t, c) hálózat tetszőleges f folyamára és tetszőleges (S, T) vágására $|f| = f(S, T)$.

Bizonyítás.

$$\begin{aligned} f(S,T) =& f(S, V \setminus S) = f(S,V) - f(S,S) = f(S,V) \\ =& f(s,V) + f(S \setminus \{s\}, V) = f(s,V) = |f|. \end{aligned}$$

Az utolsó előtti egyenlőségnél felhasználtuk, hogy bármely $u \in S \setminus \{s\}$ csúcsra $f(u,V) = 0$, következésképpen $f(S \setminus \{s\}, V) = 0$.

Állítás. Egy (G, s, t, c) hálózat tetszőleges f folyamára és tetszőleges (S,T) vágására $|f| \leqslant c(S,T)$.

Bizonyítás.

$$|f| = f(S,T) = \sum_{u \in S,\, v \in T} f(u,v) \leqslant \sum_{u \in S,\, v \in T} c(u,v) = c(S,T).$$

Tehát egy folyam értéke nem lehet nagyobb semelyik vágás kapacitásánál. A következő tételből kiderül, hogy az egyenlőség elérhető: a maximális folyam értéke megegyezik a minimális vágás kapacitásával.

Maximális folyam, minimális vágás tétel.
Legyen f egy folyam a (G, s, t, c) hálózaton. Ekkor a következők ekvivalensek.

(1) Az f folyam maximális értékű.

(2) A (G_f, s, t, c_f) reziduális hálózatban nem létezik javító út.

(3) A (G, s, t, c) hálózatban található olyan (S,T) vágás, amelyre $c(S,T) = |f|$.

Bizonyítás.
(1) $\Rightarrow$ (2): Ha az f-hez tartozó reziduális hálózatban létezik egy P javító út, akkor $f + f_P$ egy az f-nél nagyobb értékű folyam az eredeti hálózatban, ellentmondva f maximalitásának.

(2) $\Rightarrow$ (3): Tegyük fel, hogy az f-hez tartozó reziduális hálózatban nem létezik javító út. Ez azt jelenti, hogy a reziduális hálózatban nem vezet út s-ből t-be. Jelölje S a reziduális hálózat azon csúcsainak halmazát, amelyekbe vezet út s-ből, és legyen $T = V \setminus S$. Nyilvánvaló, hogy $s \in S$ és $t \in T$, így (S,T) a hálózat egy vágása.

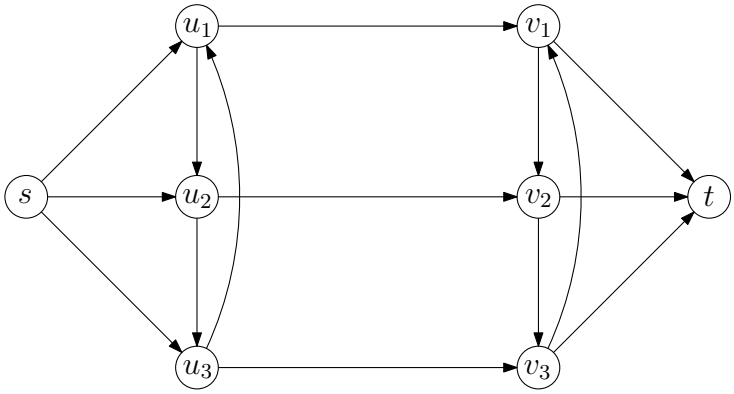
Ezek után legyenek $u \in S$ és $v \in T$ tetszőleges csúcsok. A reziduális hálózatnak (u,v) nem lehet éle, ellenkező esetben v-be is vezetne út s-ből, tehát

v-nek is S-hez kellene tartozni. Igen ám, de ha (u, v) nem éle a reziduális hálózatnak, akkor azon $f(u, v) = c(u, v)$. Ezeket az egyenlőségeket összegezve minden $(u, v) \in S \times T$ csúcspárra, kapjuk, hogy $f(S, T) = c(S, T)$. Másrészt $f(S, T) = |f|$, ahonnan $c(S, T) = |f|$ adódik.

(3) $\Rightarrow$ (1): A hálózat tetszőleges f folyamára és tetszőleges (S, T) vágására $|f| \leqslant c(S, T)$ teljesül. Az $|f| = c(S, T)$ egyenlőség ezért csak úgy lehetséges ha f egy maximális értékű folyam (és (S, T) egy minimális kapacitású vágás).

Ford-Fulkerson algoritmus költsége

Nem nehéz belátni, hogy a Ford-Fulkerson algoritmus egy iterációjának költsége $O(|E|)$. Mit mondhatunk az iterációk számáról? Tekintsük a következő hálózatot.



Legyen az (u_1, v_1) él kapacitása 1, az (u_2, v_2) él kapacitása r, az (u_3, v_3) él kapacitása r^2, az összes többi él kapacitása pedig $2 + r$, ahol

$$r = \frac{-1 + \sqrt{5}}{2} = 0.618\ldots$$

az $x^2 + x - 1 = 0$ másodfokú egyenlet pozitív gyöke. Megmutatjuk, hogy ezen a hálózaton futtatva a Ford-Fulkerson algoritmust, az nem feltétlenül fejeződik be véges sok iterációi után.

Először is jegyezzük meg, hogy $r^2 = 1 - r$, így $r^{n+2} = r^n - r^{n+1}$ tetszőleges n természetes számra. Jegyezzük meg azt is, hogy $0 < r < 1$ miatt az r^n sorozat szigorúan monoton csökenve tart nullához, továbbá

$$\sum_{n=0}^{\infty} r^n = \frac{1}{1 - r} = 2 + r.$$

A hálózatban $(\{s, u_1, u_2, u_3\}, \{v_1, v_2, v_3, t\})$ minimális vágás, hiszen ennek kapacitása $1 + r + r^2 = 2$, minden más vágás kapacitása pedig legalább $2 + r$. Így a maximális folyam minimális vágás tétel szerint a hálózat maximális folyama 2 értékű.

Futtassuk most a Ford-Fulkerson algoritmust a hálózaton a következő módon. Induljunk ki az azonosan 0 folyamból és az első iterációban növeljük a folyamot 1-gyel a reziduális hálózat

$$s \to u_1 \to v_1 \to t$$

javító útja mentén. Ezután az (u_1, v_1) él reziduális kapacitása 0, az (u_2, v_2) él reziduális kapacitása r, az (u_3, v_3) él reziduális kapacitása pedig r^2 lesz a reziduális hálózatban. A második iterációban növeljük a folyamot r^2-nal a reziduális hálózat

$$s \to u_3 \to v_3 \to v_1 \to u_1 \to u_2 \to v_2 \to t$$

javító útja mentén. Ezután az (u_1, v_1) él reziduális kapacitása r^2, az (u_2, v_2) él reziduális kapacitása $r - r^2 = r^3$, az (u_3, v_3) él reziduális kapacitása pedig 0 lesz a reziduális hálózatban. A harmadik iterációban növeljük a folyamot r^3-nal a reziduális hálózat

$$s \to u_2 \to v_2 \to v_3 \to u_3 \to u_1 \to v_1 \to t$$

javító útja mentén. Ezután az (u_1, v_1) él reziduális kapacitása $r^2 - r^3 = r^4$, az (u_2, v_2) él reziduális kapacitása 0, az (u_3, v_3) él reziduális kapacitása pedig r^3 lesz a reziduális hálózatban. A negyedik iterációban növeljük a folyamot r^4-nel a reziduális hálózat

$$s \to u_1 \to v_1 \to v_2 \to u_2 \to u_3 \to v_3 \to t$$

javító útja mentén. Ezután az (u_1, v_1) él reziduális kapacitása 0, az (u_2, v_2) él reziduális kapacitása r^4, az (u_3, v_3) él reziduális kapacitása pedig $r^3 - r^4 = r^5$ lesz a reziduális hálózatban. Világos, hogy a folyam növelését ilyen módon végtelen sokszor megismételhetjük, egyre nagyobb és nagyobb r hatványokkal. Ne feledjük, az (u_1, v_1), (u_2, v_2), (u_3, v_3) élektől különböző élek kapacitása $2 + r$, így azoknak mindig elég nagy a reziduális kapacitása a folyam növeléséhez. A folyam értéke 2-höz konvergál a megoldás elején tett második észrevétellel összhangban.

Megjegyezzük, hogy ha a hálózathoz hozzávesszük az (s, t) élt 1 kapacitással, akkor a javító utak alkalmas megválasztásával továbbra is egy 2 értékű folyamhoz tartunk, miközben az új hálózat maximális folyama 3 értékű.

Teljes indukcióval egyszerűen belátható viszont, hogy ha egy (G, s, t, c) hálózat minden élén a c kapacitásfüggvény egész értéket vesz fel, akkor a hálózat maximális folyamának értéke egész szám, továbbá ilyenkor van olyan f^* maximális folyam, amelyre minden $u, v \in V$ esetén $f^*(u, v)$ egész. A Ford-Fulkerson algoritmus éppen egy ilyet szolgáltat $O(|f^*||E|)$ költséggel (ez nem polinomiális).

Edmonds-Karp algoritmus

A Ford-Fulkerson algoritmus által végrehajtott iterációk száma $|V|$ és $|E|$ polinomjával lesz felülről becsülhető, ha mindig egy legrövidebb — vagyis egy legkevesebb élből álló — javító út mentén növelünk. Emlékezzünk rá, hogy legrövidebb javító utat szélességi kereséssel találhatunk. Az algoritmus ezen változatát Edmonds-Karp algoritmusnak nevezzük.

Az Edmonds-Karp algoritmus során végrehajtott iterációk számának becslésénél fontos szerepet játszanak a következő mennyiségek. Tetszőleges f folyam és $v \in V$ csúcs esetén jelölje $\delta_f(s, v)$ a G_f reziduális hálózatban a v csúcs s-től való távolságát, azaz az s-ből v-be vezető legrövidebb, azaz legkevesebb élből álló út hosszát.

Állítás. Ha a (G, s, t, c) hálózatban az Edmonds-Karp algoritmussal keresünk maximális folyamot, akkor az összes $v \in V \setminus \{s, t\}$ csúcsra a $\delta_f(s, v)$ mennyiség minden iterációs lépésben monoton nő.

Bizonyítás. Indirekt tegyük fel, hogy valamelyik iterációs lépésben valamelyik s-től és t-től különböző csúcs s-től való távolsága szigorúan monoton csökken. Legyen f az első ilyen iteráció előtti folyam, f' pedig az iteráció utáni folyam. Legyen továbbá v egy olyan s-től és t-től különböző csúcs, amelynek s-től való távolsága ebben az iterációs lépésben szigorúan monoton csökken, tehát amelyre $\delta_{f'}(s, v) < \delta_f(s, v)$, és amelynek az ilyen tulajdonságú csúcsok közül az s-től való távolsága minimális az f'-höz tartozó rezidális hálózatban.

Tekintsünk most egy s-ből v-be vezető legrövidebb utat az f'-höz tartozó rezidális hálózatban, jelölje ennek a v-t közvetlenül megelőző csúcsát u. Ekkor

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1.$$

A v csúcs megválasztása miatt nyilván

$$\delta_{f'}(s, u) \geqslant \delta_f(s, u).$$

Állítjuk, hogy $(u, v) \notin E_f$, ahol E_f szokásos módon az f-hez tartozó reziduális hálózat élhalmazát jelöli. Ellenkező esetben ugyanis

$$\delta_f(s, v) \leqslant \delta_f(s, u) + 1 \leqslant \delta_{f'}(s, u) + 1 = \delta_{f'}(s, v),$$

ami ellentmond a $\delta_f(s, v) > \delta_{f'}(s, v)$ feltevésnek.

Kaptuk tehát, hogy $(u, v) \notin E_f$ és $(u, v) \in E_{f'}$. Ez csak úgy lehetséges, hogy a kérdéses iterációs lépésben az $f(u, v)$ folyamérték szigorúan monoton csökken, következésképpen az $f(v, u)$ folyamérték szigorúan monoton nő, tehát (v, u) rajta van a javító úton. Mivel az Edmonds-Karp algoritmus mindig egy legrövidebb út mentén növeli a folyamot, ezért

$$\delta_f(s, v) = \delta_f(s, u) - 1 \leqslant \delta_{f'}(s, u) - 1 = \delta_{f'}(s, v) - 2,$$

ami ellentmond a $\delta_f(s, v) > \delta_{f'}(s, v)$ feltevésnek. Így az adott tulajdonságoknak megfelelő v csúcs nem létezhet.

Állítás. Ha a (G, s, t, c) hálózatban az Edmonds-Karp algoritmussal keresünk maximális folyamot, akkor az iterációs lépések száma $O(|V||E|)$.

Bizonyítás. Kezdjük egy elnevezéssel: a G_f reziduális hálózat P javító útján egy (u, v) élt kritikusnak nevezünk, ha $c_f(u, v) = c_f(P)$. Világos, hogy miután növeljük a folyamot a P javító út mentén, a javító út minden kritikus éle kikerül a reziduális hálózatból. Az is világos, hogy minden javító úton van legalább egy kritikus él.

Tekintsük a $\bar{G} = (V, \bar{E})$ irányított gráfot, ahol

$$\bar{E} = \{(u, v) \mid (u, v) \in E \text{ vagy } (v, u) \in E\}.$$

Vizsgáljuk meg, hogy egy $(u, v) \in \bar{E}$ él hányszor lehet kritikus az Edmonds-Karp algoritmus lefutása során! Ne feledjük, a reziduális hálózatok élei mindig $\bar{G}$ élei közül valók.

Először is jegyezzük meg, hogy amikor (u, v) kritikus él, akkor

$$\delta_f(s, v) = \delta_f(s, u) + 1,$$

hiszen az Edmonds-Karp algoritmus mindig egy legrövidebb út mentén növeli a folyamot. A folyamnövelés után (u, v) kikerül a reziduális hálózatból. Ezután (u, v) csak akkor jelenhet meg ismét egy reziduális hálózatban, ha a folyamérték (u, v)-n szigorúan monoton csökken, ami viszont csak akkor lehetséges, ha (v, u)-n szigorúan monoton nő, vagyis (v, u) megjelenik egy javító úton. Legyen a folyam f' amikor ez legközelebb bekövetkezik. Ekkor

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1.$$

Az előző állítás szerint $\delta_f(s,v) \leqslant \delta_{f'}(s,v)$, ezért

$$\delta_{f'}(s,u) = \delta_{f'}(s,v) + 1 \geqslant \delta_f(s,v) + 1 = \delta_f(s,u) + 2.$$

Ez azt jelenti, hogy az (u,v) él kétszeri kritikussá válása között az s-ből u-ba vezető legrövidebb út hossza legalább kettővel nőtt. Kezdetben a legrövidebb út hossza legalább 0, és amíg u elérhetetlenné nem válik s-ből, a legrövidebb út hossza soha nem lehet nagyobb, mint $|V|-2$. Ezért (u,v) legfeljebb $[|V|/2]$ alkalommal lehet kritikus él.

Mivel $|\bar{E}| \leqslant 2|E|$, azért az Edmonds-Karp algoritmus lefutása során a kritikus élek teljes száma $O(|V||E|)$. Figyelembe véve, hogy minden javító úton van legalább egy kritikus él, az állítás adódik.

Ebből következik, hogy ha egy (G,s,t,c) hálózatban az Edmonds-Karp algoritmussal keresünk maximális folyamot, akkor az eljárás teljes költsége $O(|V||E|^2)$.

Maximális párosítás

Egy $G = (V,E)$ gráfot páros gráfnak nevezünk, ha a V csúcshalmaz felosztható két diszjunkt V_1 és V_2 részre úgy, hogy minden él ezen két halmaz között fut, vagyis tetszőleges $(x,y) \in E$ esetén $x \in V_1$ és $y \in V_2$ vagy fordítva. Páros gráfokra ezzel összhangban használni fogjuk a $G = ((V_1,V_2),E)$ jelölést is.

Egy $G = ((V_1,V_2),E)$ páros gráf E élhalmazának egy E' részhalmazát G egy párosításának nevezzük, ha a $G' = ((V_1,V_2),E')$ gráfban minden csúcs foka legfeljebb 1. Egy $G = ((V_1,V_2),E)$ páros gráf egy E' párosítását maximálisnak nevezzük, ha G bármely E'' párosítására $|E''| \leqslant |E'|$.

A feladat ezek után a következő. Adott egy $G = ((V_1,V_2),E)$ páros gráf. Határozzuk meg G egy maximális párosítását!

Az egyszerűség kedvéért tegyük fel, hogy G-ben nincs izolált csúcs. Készítsünk G-ből hálózatot a következőképpen.

- Vegyünk fel egy új s és egy új t csúcsot, ezek lesznek a hálózat forrása, illetve nyelője.
- Az s csúcsból vezessünk éleket V_1 csúcsaiba, V_2 csúcsaiból pedig vezessünk éleket a t csúcsba.
- Irányítsuk az E-beli éleket V_1-től V_2 felé.
- Minden él kapacitása legyen 1.

Állítás. Az így kapott hálózatban a maximális folyam értéke megegyezik a G-beli maximális párosítások élszámával.

Bizonyítás. Legyen először f egy csupa egész értékű maximális folyam (ilyen létezik, hisz az élkapacitások mind egészek), és legyen $|f| = k$. Bármely $u \in V_1$ csúcsba legfeljebb egységnyi folyam folyhat be, ezért legfeljebb egy olyan u-ból induló $(u, v) \in E$ él lehet, amelyen $f(u, v) = 1$; a többi u-ból induló élen $f(u, v) = 0$. Hasonlóan, bármely $v \in V_2$ csúcsból legfeljebb egységnyi folyam folyhat ki, ezért legfeljebb egy olyan v-be érkező $(u, v) \in E$ él lehet, amelyen $f(u, v) = 1$; a többi v-be érkező élen $f(u, v) = 0$. Ezek szerint azok az $(u, v) \in E$ élek, amelyekre $f(u, v) = 1$, egy párosítást alkotnak G-ben. A rajtuk átmenő összes folyam értéke egyrészt nyilván megegyezik az élek számával, másrészt k-val is, hiszen ez a mennyiség az $(\{s\} \cup V_1, V_2 \cup \{t\})$ vágáson áthaladó folyam $f(\{s\} \cup V_1, V_2 \cup \{t\})$ értéke. Így van a G gráfban egy k élből álló párosítás.

Másodszor tekintsük G egy maximális párosítását, legyen ebben az élek száma k. Álljon a párosítás az $(u_1, v_1), (u_2, v_2), \ldots, (u_k, v_k) \in E$ élekből, ahol $u_1, u_2, \ldots, u_k \in V_1$ és $v_1, v_2, \ldots, v_k \in V_2$. Most minden $1 \leqslant i \leqslant k$ esetén az (s, u_i, v_i, t) út mentén egységnyi értékű folyamot vihetünk a hálózatban s-ből t-be; ezek összesen egy k értékű folyamot adnak.

Innen az állítás adódik.

Ezek szerint ha a G páros gráfhoz elkészítjük a fenti hálózatot, és abban a Ford-Fulkerson algoritmussal keresünk maximális folyamot, akkor a kapott f egészértékű maximális folyamból azonnal meg tudjuk határozni G egy maximális párosítását. Egy maximális párosítás mérete nyilván $O(|V|)$, ezért $|f| = O(|V|)$, így az eljárás teljes költsége $O(|V||E|)$.

König-Hall tétel

Tekintsünk egy $G = ((V_1, V_2), E)$ páros gráfot, amelyre $|V_1| = |V_2|$. A gráf E élhalmazának egy E' részhalmazát G egy teljes párosításának nevezzük, ha a $G' = ((V_1, V_2), E')$ gráfban minden csúcs foka pontosan 1.

A G gráf V_1-beli csúcsainak tetszőleges X részhalmazára jelölje $\Gamma(X)$ azon V_2-beli csúcsok halmazát, amelyek szomszédosak legalább egy X-beli csúccsal.

König-Hall tétel. A G gráfban akkor és csak akkor van teljes párosítás, ha bármely $X \subseteq V_1$ esetén $|X| \leqslant |\Gamma(X)|$.

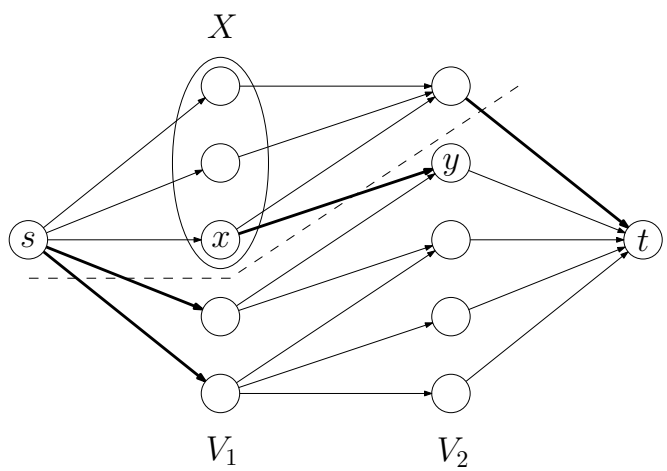
Bizonyítás. A feltétel szükségessége magától értetődik. Az elégségesség bizonyításához tegyük fel, hogy a G gráfban nincs teljes párosítás. Megmutatjuk, hogy ekkor létezik olyan $X \subseteq V_1$ csúcshalmaz, amelyre $|X| > |\Gamma(X)|$. Legyen $|V_1| = |V_2| = n$.

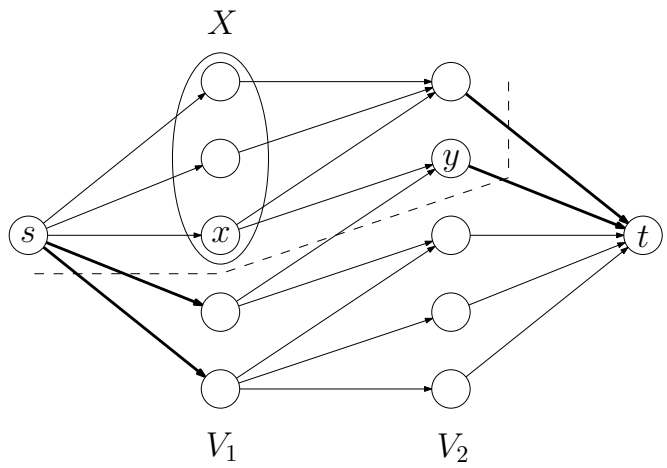
Készítsünk G-ből hálózatot a következőképpen.

- Vegyünk fel egy új s és egy új t csúcsot, ezek lesznek a hálózat forrása, illetve nyelője.
- Az s csúcsból vezessünk éleket V_1 csúcsaiba, V_2 csúcsaiból pedig vezessünk éleket a t csúcsba.
- Irányítsuk az E-beli éleket V_1-től V_2 felé.
- Minden él kapacitása legyen 1.

Idézzük fel, hogy ebben a hálózatban a maximális folyamérték megegyezik a G-beli maximális párosítások élszámával. Feltételünk szerint G-ben nincs teljes párosítás, következésképpen a hálózatban a maximális folyamérték kisebb, mint n. Így a maximális folyam – minimális vágás tétellel összhangban van olyan (S, T) vágás a hálózatban, amelynek kapacitása kisebb, mint n. Állítjuk, hogy $X = S \cap V_1$ olyan csúcshalmaz, amelyre $|X| > |\Gamma(X)|$.

Először is vegyük észre, hogy ha valamely $y \in \Gamma(X)$ csúcs T-ben van, akkor ennek áthelyezése S-be nem növeli a vágás kapacitását. Valóban, a vágás kapacitását ugyan eggyel növeli az (y, t) él, azonban minden olyan (x, y) él, amelyre $x \in X$ a kapacitást eggyel csökkenti, és ilyen élből legalább egy mindenképpen van.





Helyezzük át az összes $y \in \Gamma(X) \cap T$ csúcsot S-be. Legyen az így kialakult vágás (S', T'). Az előbbiek szerint az (S', T') vágás kapacitása is kisebb, mint n. A hálózat V_1-ből V_2-be menő élei nyilván nem keresztezik az (S', T') vágást. Az s csúcsból induló élek közül azok keresztezik a vágást, amelyek T'-beli csúcsokhoz vezetnek, a t csúcsba érkezők közül pedig azok, amelyek S'-beli csúcsokból indulnak. Ennélfogva

$$c(S', T') = |V_1 \cap T'| + |V_2 \cap S'|,$$

Mivel $|V_1 \cap T'| = n - |X|$ és $|V_2 \cap S'| \geqslant |\Gamma(X)|$, így

$$n > c(S', T') = |V_1 \cap T'| + |V_2 \cap S'| \geqslant n - |X| + |\Gamma(X)|,$$

ahonnan $|X| > |\Gamma(X)|$ adódik.

További alkalmazások

Étterem

Egy étterem a város közösségi életének felpezsdítésére vasárnaponként az ebédelni érkező társaságokat úgy ülteti le, hogy egy társaság minden tagja más asztalhoz kerüljön. Az étteremben n asztal van, ezek $k_1, k_2, \ldots, k_n$ személyesek. A következő vasárnapra m társaság jelezte a jövetelét, ezek $l_1, l_2, \ldots, l_m$ tagúak. Adjunk hatékony algoritmust annak eldöntésére, hogy van-e az adott feltételnek eleget tevő ültetés, és ha igen, az algoritmus adjon is meg egy ilyet!

Egy $\mathcal{H}$ hálózatot készítünk, amelyben minden $1 \leqslant i \leqslant m$ esetén a T_i társaságnak megfelel egy u_i csúcs és minden $1 \leqslant j \leqslant n$ esetén az A_j asztalnak

egy v_j csúcs. Minden $1 \leqslant i \leqslant m$ és $1 \leqslant j \leqslant n$ esetén menjen egy irányított él az u_i csúcsból a v_j csúcsba. Vegyünk fel még egy s forrást és egy t nyelőt. Minden $1 \leqslant i \leqslant m$ esetén menjen egy irányított él s-ből u_i-be, valamint minden $1 \leqslant j \leqslant n$ esetén menjen egy irányított él v_j-ből t-be. Legyen az s csúcsból az u_i csúcsba vezető él kapacitása l_i minden $1 \leqslant i \leqslant m$ esetén, legyen a v_j csúcsból a t csúcsba vezető él kapacitása k_j minden $1 \leqslant j \leqslant n$ esetén, a többi él kapacitása pedig legyen 1.

Állítás. A társaságokat akkor és csak akkor lehet a feltételeknek megfelelően leültetni, ha a $\mathcal{H}$ hálózatban a maximális folyamérték $l = l_1 + l_2 + \cdots + l_m$.

Jegyezzük meg, hogy a $\mathcal{H}$ hálózatban a maximális folyamérték nem lehet nagyobb, mint l, hiszen a hálózat $(\{s\}, V \setminus \{s\})$ vágásának a kapacitása éppen ennyi.

Bizonyítás. Tegyük fel először, hogy a társaságokat le lehet ültetni a feltételeknek megfelelő módon. Ha a T_i társaság egy tagját az A_j asztalhoz ültettük, akkor küldjünk egységnyi folyamot az (s, u_i, v_j, t) úton s-ből t-be. Ezt minden társaság minden tagjára elvégezve világos módon egy l értékű folyamot kapunk, amely az összes kapacitásfeltételt kielégíti.

Megfordítva, tegyük fel, hogy a hálózaton a maximális folyamérték l. Ekkor létezik olyan egészértékű folyam a hálózaton, amelynek értéke l. Ha most az (u_i, v_j) élen egységnyi folyam halad át, akkor a T_i társaság egy tagját ültessük az A_j asztalhoz. A kapacitásfeltételek miatt így minden asztalhoz legfeljebb annyi embert ültetünk, ahány személyes az asztal, és egy társaság semelyik két tagját nem ültetjük ugyanahhoz az asztalhoz.

Maximális folyamot a Ford-Fulkerson algoritmussal kereshetünk. Ennek költsége $O(|f^*||E|)$, ahol $|f^*|$ a maximális folyamérték, $|E|$ pedig a hálózat élszáma. Itt $|E| = O(mn)$ nyilvánvaló módon. Másrészt a maximális folyamérték nem lehet nagyobb, mint az $(\{s\}, V \setminus \{s\})$ vágás kapacitása, ezért $|f^*| \leqslant l$. Így az eljárás költsége $O(lmn)$.

Ha a maximális folyamot az Edmonds-Karp algoritmussal keressük, akkor a költség $O(|V||E|^2)$, ahol $|V|$ és $|E|$ a hálózat csúcs- illetve élszáma. Itt $|V| = O(m+n)$ és $|E| = O(mn)$, így az eljárás költségére $O((m+n)m^2n^2)$ adódik.

Páros gráfok fokszámsorozatai

Legyenek $p_1 \geqslant p_2 \geqslant \cdots \geqslant p_m$ és $q_1 \geqslant q_2 \geqslant \cdots \geqslant q_n$ olyan nem negatív egész számok, amelyekre $p_1 + p_2 + \cdots + p_m = q_1 + q_2 + \cdots + q_n$ teljesül. Adjunk hatékony algoritmust annak eldöntésére, hogy létezik-e olyan $G = ((V, W), E)$

páros gráf, amelyben $V = \{v_1, v_2, \ldots, v_m\}$ és $W = \{w_1, w_2, \ldots, w_n\}$, továbbá $\deg(v_i) = p_i$ és $\deg(w_j) = q_j$ minden $1 \leqslant i \leqslant m$, illetve minden $1 \leqslant j \leqslant n$ esetén! Mutassuk meg azt is, hogy akkor és csak akkor létezik ilyen $G = ((V, W), E)$ páros gráf ha

$$\sum_{i=1}^{m} \min\{p_i, k\} \geqslant \sum_{j=1}^{k} q_j$$

fennáll minden $1 \leqslant k \leqslant n$ esetén!

Készítsük el a következő $\mathcal{H}$ hálózatot.

- A hálózat csúcsai legyenek $s, v_1, v_2, \ldots, v_m, w_1, w_2, \ldots, w_n, t$, ahol s a forrás és t a nyelő.
- Az s csúcsból vezessünk irányított éleket a $v_1, v_2, \ldots, v_m$ csúcsokba; a v_i csúcsba vezető él kapacitása legyen p_i minden $1 \leqslant i \leqslant m$ esetén.
- A $w_1, w_2, \ldots, w_n$ csúcsokból vezessünk irányított éleket a t csúcsba; a w_j csúcsból induló él kapacitása legyen q_j minden $1 \leqslant j \leqslant n$ esetén.
- Minden v_i csúcsból vezessünk irányított élt minden w_j csúcsba. Ezeknek az éleknek a kapacitása legyen 1.

Legyen $V = \{v_1, v_2, \ldots, v_m\}$ és $W = \{w_1, w_2, \ldots, w_n\}$.

Állítás. Akkor és csak akkor létezik olyan $G = ((V, W), E)$ páros gráf, amelyben a fokszámok az előírt értékek, ha a $\mathcal{H}$ hálózatban a maximális folyamérték $p_1 + p_2 + \cdots + p_m$.

Jegyezzük meg, hogy a hálózatban a maximális folyamérték nem lehet nagyobb, mint $p_1 + p_2 + \cdots + p_m$, hiszen a hálózat $(\{s\}, V \cup W \cup \{t\})$ vágásának a kapacitása éppen ennyi.

Bizonyítás. Tegyük fel először, hogy van olyan $G = ((V, W), E)$ páros gráf, amely megfelel a feltételeknek. Ha most $(v_i, w_j) \in E$, akkor küldjünk egységnyi folyamot az (s, v_i, w_j, t) úton s-ből t-be. Ezt a G gráf minden (v_i, w_j) élére végrehajtva világos módon egy $p_1 + p_2 + \cdots + p_m$ értékű folyamot kapunk, amely az összes kapacitásfeltételt kielégíti.

Megfordítva, tegyük fel, hogy a hálózaton a maximális folyamérték $p_1 + p_2 + \cdots + p_m$. Ekkor létezik olyan egészértékű folyam a hálózaton, amelynek értéke $p_1 + p_2 + \cdots + p_m$. Ha most a hálózat (v_i, w_j) élen egységnyi folyam halad át, akkor legyen $(v_i, w_j) \in E$ a G gráfban. A kapacitásfeltételek miatt így egy olyan $G = ((V, W), E)$ páros gráfhoz jutunk, amelyben a fokszámok az előírt értékek.

Mivel a $\mathcal{H}$ hálózat $(\{s\}, V \cup W \cup \{t\})$ vágásának a kapacitása $p_1 + p_2 + \cdots + p_m$, ezért a maximális folyam – minimális vágás tétellel összhangban ezek után elég megmutatni, hogy a feladatban megfogalmazott

$$\sum_{i=1}^{m} \min\{p_i, k\} \geqslant \sum_{j=1}^{k} q_j$$

minden $1 \leqslant k \leqslant n$ esetén feltétel ekvivalens azzal, hogy a $\mathcal{H}$ hálózat tetszőleges vágásának a kapacitása legalább $p_1 + p_2 + \cdots + p_m$.

Tegyük fel először, hogy a $\mathcal{H}$ hálózat tetszőleges vágásának a kapacitása legalább $p_1 + p_2 + \cdots + p_m$. Legyen $1 \leqslant k \leqslant n$ és tekintsük a hálózatnak azt az (S, T) vágását, ahol

$$S = \{s\} \cup \{v_i \mid p_i > k\} \cup \{w_{k+1}, w_{k+2}, \ldots, w_n\}$$

és

$$T = \{v_i \mid p_i \leqslant k\} \cup \{w_1, w_2, \ldots, w_k\} \cup \{t\}.$$

Ekkor

$$\sum_{i=1}^{m} p_i \leqslant c(S, T) = \sum_{p_i \leqslant k} p_i + |\{i \mid p_i > k\}| \times k + \sum_{j=k+1}^{n} q_j,$$

ahonnan

$$\sum_{j=1}^{k} q_j \leqslant \sum_{p_i \leqslant k} p_i + |\{i \mid p_i > k\}| \times k = \sum_{i=1}^{m} \min\{p_i, k\}$$

(felhasználtuk, hogy $p_1 + p_2 + \cdots + p_m = q_1 + q_2 + \cdots + q_n$).

Megfordítva, tegyük fel, hogy

$$\sum_{i=1}^{m} \min\{p_i, k\} \geqslant \sum_{j=1}^{k} q_j$$

fennáll minden $1 \leqslant k \leqslant n$ esetén. Tekintsük a hálózat egy tetszőleges (S, T) vágását és legyen $k = |T \cap W|$. Ha $k = 0$, akkor

$$c(S, T) \geqslant q_1 + q_2 + \cdots + q_n = p_1 + p_2 + \ldots + p_m$$

triviálisan igaz, ezért tegyük fel, hogy $k \geqslant 1$. Most

$$
\begin{aligned}
c(S,T) &= \sum_{\{i|v_i \in T\cap V\}} p_i + |\{i \mid v_i \in S\cap V\}| \times k + \sum_{\{j|w_j \in S\cap W\}} q_j \\
&\geqslant \sum_{\{i|v_i \in T\cap V\}} p_i + \sum_{\{i|v_i \in S\cap V\}} \min\{p_i, k\} + \sum_{\{j|w_j \in S\cap W\}} q_j \\
&\geqslant \sum_{\{i|v_i \in T\cap V\}} p_i + \sum_{j=1}^{k} q_j - \sum_{\{i|v_i \in T\cap V\}} \min\{p_i, k\} + \sum_{\{j|w_j \in S\cap W\}} q_j \\
&\geqslant \sum_{\{i|v_i \in T\cap V\}} p_i + \sum_{\{j|w_j \in T\cap W\}} q_j - \sum_{\{i|v_i \in T\cap V\}} p_i + \sum_{\{j|w_j \in S\cap W\}} q_j \\
&= \sum_{j=1}^{n} q_j = \sum_{i=1}^{m} p_i
\end{aligned}
$$

(a második egyenlőtlenségnél használtuk ki a feltételt, a harmadiknál pedig azt, hogy $q_1 \geqslant q_2 \geqslant \cdots \geqslant q_n$).

Innen az állítás adódik.

Élidegen utak irányított gráfokban

Legyen $G = (V, E)$ izolált csúcsok nélküli irányított gráf, és $s, t \in V$ különböző csúcsok. Adjunk hatékony algoritmust maximális számú, páronként élidegen (közös él nélküli) s-ből t-be menő irányított út megkeresésére!

Tekintsük a (G, s, t, c) hálózatot, ahol minden él kapacitása 1. A kapacitások egészek, így van olyan f maximális folyam, amelyre $f(u, v)$ egész minden (u, v) csúcspárra. Ebből azonnal következik, hogy ha $(u, v) \in E$, akkor $f(u, v)$ vagy -1 vagy 0 vagy 1. Legyen $|f| = k$. Állítjuk, hogy ekkor van G-ben k darab páronként élidegen s-ből t-be menő irányított út.

Először azt mutatjuk meg, hogy ha $k > 0$, akkor van olyan P út s-ből t-be, amelynek élein az f folyam 1 értéket vesz fel. Induljunk el s-ből egy olyan $(s, u) \in E$ élen, amelyre $f(s, u) = 1$. Az u csúcsba érve töröljük ezt az élt, majd lépjünk tovább egy olyan $(u, v) \in E$ élen, amelyre $f(u, v) = 1$, és töröljük ezt az élt is. Így folytatva sosem akadhatunk el egy t-től különböző csúcsban, hiszen a hálózat s-től és t-től különböző csúcsaiból ugyanannyi egységnyi folyamot szállító él indul ki, mint amennyi befut oda, s-ből pedig több. Előbb-utóbb t-be érünk, hiszen az élek fogynak.

Ha tehát $k > 0$, akkor nézzünk egy az előbbiek szerinti P utat. Töröljük G-ből a P éleit, és feledkezzünk meg a rajtuk átmenő egységnyi folyamról. A kapott gráfban egy $k - 1$ értékű folyam marad, amire $k - 1 > 0$ esetén

ismételjük meg a fenti útkeresést. Így folytatva végül k darab páronként élidegen s-ből t-be menő irányított utat kapunk. A maximális folyam értéke tehát legfeljebb akkora, mint a páronként élidegen s-ből t-be menő irányított utak maximális elemszámú rendszereinek elemszáma.

Megfordítva, legyenek $P_1, P_2, \ldots, P_k$ maximális számú páronként élidegen s-ből t-be menő irányított utak G-ben. Az általánosság megszorítása nélkül feltehetjük, hogy ezek az utak egyszerűek. Ha G valamely két u és v csúcsa között mindkét irányban megy él, és mindkét élt használják a fenti élidegen utak, mondjuk (u, v)-t a P_i út, míg (v, u)-t a P_j út, akkor P_i-ben helyettesítsük az u utáni részt a P_j út u utáni részével, valamint P_j-ben helyettesítsük a v utáni részt a P_i út v utáni részével. Ezt megismételve minden ilyen ellentétes élpárra kapjuk, hogy szükségképpen vannak olyan $P_1, P_2, \ldots, P_k$ maximális számú páronként élidegen s-ből t-be menő irányított utak G-ben, amelyek az ellentétes élpárokból legfeljebb az egyiket használják. Ezek mindegyikén egységnyi folyamot küldhetünk a (G, s, t, c) hálózatban a forrásból a nyelőbe, ami összesen egy k értékű folyamot jelent. A maximális folyam értéke tehát legalább akkora, mint a páronként élidegen s-ből t-be menő irányított utak maximális elemszámú rendszereinek elemszáma.

Ebből következik, hogy a maximális folyam értéke megegyezik a páronként élidegen s-ből t-be menő irányított utak maximális elemszámú rendszereinek elemszámával.

A hálózatban a Ford-Fulkerson algoritmussal kereshetünk egy egészértékű maximális folyamot. Ennek költsége $O(|V||E|)$, hiszen a maximális folyam értéke nem haladhatja meg az s-ből kimenő élek számát, ami $O(|V|)$. A maximális folyam ismeretében ezek után az élidegen utak a fenti gondolatmenetet követve szélességi vagy mélységi keresések alkalmazásával megkonstruálhatók. A költség itt is $O(|V||E|)$. Így az algoritmus összköltsége $O(|V||E|)$.

Orvosi ügyelet

Egy kórház a hétvégi orvosi ügyeleteket n orvossal akarja ellátni.

- Minden orvosról tudjuk, hogy mely napokon nem tud ügyeletet vállalni.
- Minden orvos legfeljebb k napot ügyelhet összesen.
- Egy orvos egy hétvégén legfeljebb egy napot ügyelhet.
- Minden hétvége minden napjára be kell osztani egy orvost.

Adjunk hatékony algoritmust annak eldöntésére, hogy van-e az adott feltételeknek eleget tevő beosztás, és ha igen, az algoritmus adjon is meg egy ilyet!

Egy $\mathcal{H}$ hálózatot készítünk, amelyben az orvosoknak feleljenek meg az $u_1, u_2, \ldots, u_n$, az ellátandó napoknak pedig a $w_1, w_2, \ldots, w_m$ csúcsok. Vegyünk fel továbbá egy v_{ij} csúcsot minden olyan esetben, amikor az i-edik orvos a j-edik hétvége legalább egy napján rendelkezésre áll. Az u_i csúcsból menjen egy 1 kapacitású él a v_{ij} csúcsba, míg a v_{ij} csúcsból induljanak 1 kapacitású élek a j-edik hétvége azon napjaihoz, amelyeken az orvos rendelkezésre áll. Vegyünk fel végül egy s forrást és egy t nyelőt. Az s forrásból vezessenek k kapacitású élek az orvosokhoz, a t forrásba pedig vezessenek 1 kapacitású élek az ellátandó napokból.

Állítás. Akkor és csak akkor létezik olyan ügyeleti beosztás, amely az összes feltételt kielégíti, ha a $\mathcal{H}$ hálózatban a maximális folyamérték m.

Jegyezzük meg, hogy a hálózatban a maximális folyamérték nem lehet nagyobb, mint m, hiszen a hálózat $(V \setminus \{t\}, \{t\})$ vágásának a kapacitása éppen ennyi.

Bizonyítás. Tegyük fel először, hogy van olyan ügyeleti beosztás, amely az összes feltételt kielégíti. Ha az i-edik orvos ügyel a j-edik hétvége valamely l napján, akkor küldjünk egységnyi folyamot az (s, u_i, v_{ij}, w_l, t) úton s-ből t-be. Ezt minden ilyen (i, l) párra elvégezve világos módon egy m értékű folyamot kapunk, amely az összes kapacitásfeltételt kielégíti.

Megfordítva, tegyük fel, hogy a hálózaton a maximális folyamérték m. Ekkor létezik olyan egészértékű folyam a hálózaton, amelynek értéke m. Ha most a (v_{ij}, w_l) élen egységnyi folyam halad át, akkor osszuk be ügyelni az i-edik orvost a j-edik hétvége l napjára. A kapacitásfeltételek miatt így minden orvos legfeljebb k napot ügyel összesen, egy hétvégén legfeljebb egy napot, és minden hétvége minden napjára be van osztva egy orvos.

Maximális folyamot a Ford-Fulkerson algoritmussal kereshetünk. Ennek költsége $O(|f^*||E|)$, ahol $|f^*|$ a maximális folyamérték, $|E|$ pedig a hálózat élszáma. Itt $|E| = O(nm)$ nyilvánvaló módon. Másrészt a maximális folyamérték nem lehet nagyobb, mint a $(V \setminus \{t\}, \{t\})$ vágás kapacitása, ezért $|f^*| \leqslant m$. Így az eljárás költsége $O(m^2 n)$.

Projektek és beruházások

Egy cég a $P_1, P_2, \ldots, P_n$ projektek végrehajtására kap megbízási ajánlatot. A megbízó a P_i projekt végrehajtásáért p_i forintot fizetne. A projektek végrehajtásához különböző beruházások szükségesek, jelölje ezeket $B_1, B_2, \ldots, B_m$, a P_i projekthez pontosan az $R_i \subseteq \{B_1, B_2, \ldots, B_m\}$ beruházások. A cégnek b_j forintjába kerül a B_j beruházás elvégzése. Adjunk hatékony algo-

ritmust annak eldöntésére, hogy a cég mely projektek végrehajtását vállalja el, ha a bevételt maximalizálni akarja!

Az egyszerűség kedvéért tegyük fel, hogy minden projekthez végre kell hajtani legalább egy beruházást, és nincs olyan beruházás, amely egyetlen projekthez sem szükséges. Készítsük el a következő hálózatot.

- A hálózat csúcsai legyenek $s, B_1, B_2, \ldots, B_m, P_1, P_2, \ldots, P_n, t$, ahol s a forrás és t a nyelő.
- Az s csúcsból vezessünk irányított éleket a $B_1, B_2, \ldots, B_m$ csúcsokba; a B_i csúcsba vezető él kapacitása legyen b_i minden $1 \leqslant i \leqslant m$ esetén.
- A $P_1, P_2, \ldots, P_n$ csúcsokból vezessünk irányított éleket a t csúcsba; a P_i csúcsból induló él kapacitása legyen p_i minden $1 \leqslant i \leqslant n$ esetén.
- Egy B_i csúcsból vezessünk irányított élt egy P_j csúcsba akkor és csak akkor, ha $B_i \in R_j$. Egy ilyen él kapacitása legyen $C + 1$, ahol

$$C = \sum_{k=1}^{n} p_k.$$

Az algoritmus ezek után egyszerű. Határozzuk meg a hálózat egy (S, T) minimális vágását, és vállaljuk el azokat a projekteket, amelyek T-ben vannak. Egy (S, T) minimális vágást a következőképpen találhatunk. Keressük meg a hálózat egy maximális f folyamát, majd a hálózat csúcsait aszerint osszuk S-be, illetve T-be, hogy a G_f reziduális hálózatban elérhetők-e s-ből, vagy nem. Ha a maximális folyamot az Edmonds-Karp algoritmussal keressük, akkor az algoritmus költsége $O((m + n)(m + n + r)^2)$, ahol

$$r = \sum_{k=1}^{n} |R_k|.$$

Lássuk az algoritmus helyességének a bizonyítását. Tekintsük először projekteknek és beruházásoknak egy olyan

$$\{P_{i_1}, P_{i_2}, \ldots, P_{i_h}, B_{j_1}, B_{j_2}, \ldots, B_{j_l}\}$$

halmazát, amelyben minden projekt mellett az ahhoz szükséges beruházások is szerepelnek. Tekintsük továbbá azt az (S', T') vágást, ahol $T' = \{B_{j_1}, B_{j_2}, \ldots, B_{j_l}, P_{i_1}, P_{i_2}, \ldots, P_{i_h}, t\}$.

A hálózat azon élei, amelyek beruházásokból vezetnek projektekbe, nem keresztezik a vágást, hiszen T'-ben az összes olyan beruházás benne van, amely valamelyik T'-beli projekthez szükséges.

Az s csúcsból induló élek közül azok keresztezik a vágást, amelyek T'-ben lévő beruházásokhoz vezetnek. Ezek a

$$\sum_{k=1}^{l} b_{j_k}$$

mennyiséggel járulnak hozzá a vágás kapacitásához.

A t csúcsba érkező élek közül azok keresztezik a vágást, amelyek nem T'-ben lévő projektekből indulnak. Ezek a

$$C - \sum_{k=1}^{h} p_{i_k}$$

mennyiséggel járulnak hozzá a vágás kapacitásához.

Az (S', T') vágás kapacitása ezen két mennyiség összege:

$$\sum_{k=1}^{l} b_{j_k} + C - \sum_{k=1}^{h} p_{i_k} = C - \left(\sum_{k=1}^{h} p_{i_k} - \sum_{k=1}^{l} b_{j_k} \right).$$

Vegyük észre, hogy a jobb oldalon a zárójeles mennyiség a cég bevétele abban az esetben, amikor a $P_{i_1}, P_{i_2}, \ldots, P_{i_h}$ projekteket teljesíti és a $B_{j_1}, B_{j_2}, \ldots, B_{j_l}$ beruházásokat hajtja végre. Így a bevétel akkor lesz maximális, amikor az (S', T') vágás kapacitása a lehető legkisebb.

A bizonyítás teljessé tételéhez ezek után elég megmutatni, hogy a hálózat egy (S, T) minimális vágására is teljesül, hogy T a benne levő projektek mellett az azokhoz szükséges beruházásokat is tartalmazza. Legyen (S, T) a hálózat egy minimális vágása. Mivel az

$$(\{s, B_1, B_2, \ldots, B_m, P_1, P_2, \ldots, P_n\}, \{t\})$$

vágás kapacitása C, ezért az (S, T) minimális vágás kapacitása sem lehet ennél nagyobb. Ebből viszont következik, hogy az (S, T) vágást nem keresztezheti egyetlen beruházásból projektbe vezető él sem, hiszen ezek kapacitása $C + 1$. Így ha valamelyik projekt benne van T-ben, akkor az ehhez szükséges beruházások is mind benne vannak T-ben.

Legnagyobb sűrűségű részgráf

Tekintsünk egy $G = (V, E)$ irányítatlan gráfot. A V csúcshalmaz egy tetszőleges, nem üres S részhalmazára jelölje $G(S) = (S, E(S))$ a G gráfnak az S csúcshalmaz által feszített részgráfját. Itt $E(S) \subseteq E$ a G gráf azon éleinek

halmaza, amelyek két S-beli csúcsot kötnek össze. Az S csúcshalmaz által feszített $G(S)$ gráf sűrűségét definiáljuk a $G(S)$ gráf élszámának és csúcsszámának a hányadosaként, vagyis a sűrűség $|E(S)|/|S|$. Adjunk hatékony algoritmust egy olyan feszített részgráf meghatározására, amelynek sűrűsége maximális!

Ha a gráfnak egyetlen éle sincs, akkor bármely feszített részgáfjának 0 a sűrűsége. Tegyük fel ezért, hogy a G gráfnak van legalább egy éle. Jelölje a G gráf csúcsainak számát n, éleinek számát pedig m. Jelölje továbbá a G gráf legnagyobb sűrűségű feszített részgráfjának sűrűségét D^*. Nyilvánvaló módon $0 < D^* \leqslant m$.

Legyen ezek után $0 < \gamma \leqslant m$, és készítsük el a következő $\mathcal{H}_\gamma$ hálózatot:

- A hálózat csúcshalmaza legyen $V \cup \{s, t\}$, ahol az új s csúcs a forrás és az új t csúcs a nyelő.
- Minden $v \in V$ esetén az s csúcsból vezessünk m kapacitású irányított élt a v csúcsba.
- Minden $v \in V$ esetén a v csúcsból vezessünk $m + 2\gamma - d(v)$ kapacitású irányított élt a t csúcsba, ahol $d(v)$ a v csúcs G-beli fokszáma.
- Minden olyan $u, v \in V$ csúcspárra, amelyeket él köt össze G-ben, vezessen 1 kapacitású él u-ból v-be, és szintén 1 kapacitású él v-ből u-ba.

Mivel a $\mathcal{H}_\gamma$ hálózat $(\{s\}, V \cup \{t\})$ vágásának kapacitása nm, ezért a hálózat maximális folyamának értéke nem haladhatja meg az nm értéket.

Állítás. A $\mathcal{H}_\gamma$ hálózatban a maximális folyamérték akkor és csak akkor nm, ha $\gamma \geqslant D^*$.

Mielőtt rátérnénk a bizonyításra alaposabban szemügyre vesszük a $\mathcal{H}_\gamma$ hálózat többi vágását is. Legyen $S \subseteq V$ tetszőleges, nem üres halmaz, és tekintsük az $(\{s\} \cup S, (V \setminus S) \cup \{t\})$ vágást. Ennek a vágásnak a kapacitása az s csúcsból a $V \setminus S$-beli csúcsokba vezető élek, az S-beli csúcsokból a t csúcsba vezető élek, valamint az S-beli csúcsokból a $V \setminus S$-beli csúcsokba vezető élek kapacitásainak összege. Jelölje $\delta(S)$ a G gráf azon éleinek halmazát, amelyek egy S-beli és $V \setminus S$-beli csúcsot kötnek össze G-ben. Egyszerű számolással adódik, hogy

$$\sum_{v \in S} d(v) = 2|E(S)| + |\delta(S)|,$$

hiszen a bal oldali összegben a $G(S)$ feszített részgráf éleit kétszer, a $\delta(S)$-beli éleket egyszer vesszük figyelembe. Mindezek alapján

$$\begin{aligned} c(\{s\} \cup S, (V \setminus S) \cup \{t\}) &= m|V \setminus S| + |\delta(S)| + \sum_{v \in S}(m + 2\gamma - d(v)) \\ &= nm - m|S| + |\delta(S)| + m|S| + 2\gamma|S| - \sum_{v \in S} d(v) \\ &= nm + |\delta(S)| + 2\gamma|S| - (2|E(S)| + |\delta(S)|) \\ &= nm + 2|S| \left(\gamma - \frac{|E(S)|}{|S|}\right). \end{aligned}$$

Ezt a formulát a későbbiekben többször is használni fogjuk.

Bizonyítás. Először tegyük fel, hogy $\gamma < D^*$. Legyen S^* egy olyan részhalmaza V-nek, amely által feszített részgráf sűrűsége D^*. Ekkor a $\mathcal{H}_\gamma$ hálózatban a fenti képlet szerint

$$\begin{aligned} c(\{s\} \cup S^*, (V \setminus S^*) \cup \{t\}) &= nm + 2|S^*| \left(\gamma - \frac{|E(S^*)|}{|S^*|}\right) \\ &= nm + 2|S^*|(\gamma - D^*) \\ &< nm, \end{aligned}$$

így a hálózatban a maximális folyamérték is szükségképpen kisebb, mint nm.

Tegyük fel ezek után, hogy a hálózatban a maximális folyamérték kisebb, mint nm. Ekkor a maximális folyam, minimális vágás tétellel összhangban van olyan $(\{s\} \cup S, (V \setminus S) \cup \{t\})$ vágása a hálózatnak, amelynek kapacitása szintén kisebb, mint nm. Egy ilyen $S \subseteq V$ részhalmaz nyilván nem üres, hiszen $S = \emptyset$ esetén a vágás kapacitása nm. Így a fenti képlet szerint

$$c(\{s\} \cup S, (V \setminus S) \cup \{t\}) = nm + 2|S| \left(\gamma - \frac{|E(S)|}{|S|}\right) < nm,$$

ami csak akkor lehetséges, ha

$$\gamma < \frac{|E(S)|}{|S|} \leqslant D^*.$$

Ezek után az algoritmus a következő. Tudjuk, hogy $0 < D^* \leqslant m$, határozzuk meg ezért először a maximális folyamértéket a $\mathcal{H}_\gamma$ hálózatban $\gamma = m/2$ választással. Ha a maximális folyamérték nm, akkor $0 < D^* \leqslant \gamma$, ha kisebb, akkor $\gamma < D^* \leqslant m$, az állítással összhangban. Általában is, ha már tudjuk,

hogy $b < D^* \leqslant c$ valamilyen b és c konstansokkal, akkor a következő lépésben határozzuk meg a maximális folyamértéket a $\mathcal{H}_\gamma$ hálózatban $\gamma = (b+c)/2$ választással. Ha a maximális folyamérték nm, akkor $b < D^* \leqslant \gamma$, ha kisebb, akkor $\gamma < D^* \leqslant c$, az előzőekhez hasonlóan. Ezt ismételve a D^* értéket tetszőleges pontossággal meg tudjuk határozni.

De hogyan határozzunk meg egy D^* sűrűségű feszített részgráfot? Ehhez szükségünk lesz a második legnagyobbra is azon sűrűségek közül, amiket a részgráfokhoz rendeltünk. Jelölje ezt D'. Mivel $D^* = m^*/n^*$ és $D' = m'/n'$ alkalmas $1 \leqslant m^* \leqslant m$ és $1 \leqslant n^* \leqslant n$, valamint $1 \leqslant m' \leqslant m$ és $1 \leqslant n' \leqslant n$ egészekkel, ezért

$$D^* - D' = \frac{m^*}{n^*} - \frac{m'}{n'} = \frac{m^*n' - m'n^*}{n^*n'} \geqslant \frac{1}{n^2},$$

hiszen $m^*n' - m'n^* \geqslant 1$ és $n^*n' \leqslant n^2$. Így ha a fenti algoritmus során olyan b alsó és c felső korlátaink vannak D^*-ra, amelyekre $c - b < 1/n^2$, akkor $\gamma = b$ választással $D' \leqslant \gamma < D^*$.

Mivel a D^*-ot tartalmazó intervallum hossza minden lépésben a felére csökken, $O(\log(mn^2)) = O(\log n^4) = O(\log n)$ intervallumfelezés (maximális folyam keresés a megfelelő $\mathcal{H}_\gamma$ hálózatokban) után $D' \leqslant \gamma < D^*$ a rendelkezésünkre áll.

Nézzük mi történik, ha ilyen γ választással keresünk maximális folyamot a $\mathcal{H}_\gamma$ hálózatban? Mivel $\gamma < D^*$, ezért a hálózatban a maximális folyamérték kisebb lesz, mint nm. Tekintsük most a hálózat egy olyan $(\{s\} \cup S, (V \setminus S) \cup \{t\})$ vágását, amelynek kapacitása megegyezik a maximális folyamértékkel. Nyilván S nem üres, hiszen a maximális folyaamérték, és így a vágás kapacitása is kisebb, mint nm. A vágások kapacitására levezetett formula szerint

$$c(\{s\} \cup S, (V \setminus S) \cup \{t\}) = nm + 2|S| \left(\gamma - \frac{|E(S)|}{|S|}\right) < nm,$$

ami csak akkor lehetséges, ha $\gamma < |E(S)|/|S|$. Figyelembe véve, hogy γ nagyobb, mint a második legnagyobb sűrűség, ezért $G(S)$ szükségképpen egy legnagyobb sűrűségű feszített részgráf.

Hálózatok további korlátokkal

Tekintsünk egy (G, s, t, c) hálózatot. Az egyszerűség kedvéért tegyük fel, hogy a hálózat bármely u és v csúcsa között legfeljebb csak az egyik irányba megy él. Speciálisan az s forrásba nem érkezik és a t nyelőből nem indul egyetlen él sem. Tegyük fel továbbá, hogy s-ből t-be nem megy közvetlen él.

Jegyezzük meg, hogy az a feltétel, mely szerint két csúcs között csak az egyik irányban mehet él, igazából nem jelent megszorítást. Ha (u, v) és (v, u) is éle a hálózatnak, akkor az utóbbit helyettesíthetjük egy (v, v^*), (v^*, u) élpárral, ahol v^* egy új csúcs. A (v, v^*) és (v^*, u) élekhez ugyanazt a kapacitást rendeljük, mint ami a (v, u) élnél szerepelt.

Tegyük fel, hogy a $c(u, v)$ kapacitások mellett, amelyek felülről korlátozzák az éleken áthaladó folyamértékeket, alsó korlátjaink is vannak az $f(u, v)$ mennyiségekre. Egy ilyen hálózatot formálisan egy (G, s, t, c, k) ötössel írhatunk le. Ebben az első négy összetevő értelmezése a régi. Az utolsó tag, a k egy a G élein értelmezett, nem negatív értékű függvény, amely az élekhez rendelt alsó korlátokat mondja meg. A korábbi folyam definíció kiegészítéseként megköveteljük, hogy G minden (u, v) élére $k(u, v) \leqslant f(u, v)$ is teljesüljön.

Adjunk hatékony algoritmust annak eldöntésére, hogy létezik-e legfeljebb d értékű folyam egy $\mathcal{H} = (G, s, t, c, k)$ hálózathoz, ahol d adott pozitív valós szám!

Tekintsünk egy $\mathcal{H} = (G, s, t, c, k)$ hálózatot. Ehhez készítünk egy alsó korlátok nélküli $\mathcal{H}'$ hálózatot a következőképpen. Vegyünk fel a $G = (V, E)$ gráf csúcsai mellé egy S új forrást és egy T új nyelőt. A $\mathcal{H}'$ hálózatot csúcshalmaza $V \cup \{S, T\}$ lesz. A G éleit megtartjuk, de új kapacitásokat rendelünk hozzájuk. Az $(u, v) \in E$ él kapacitása a $\mathcal{H}'$ hálózatban legyen

$$c'(u, v) = c(u, v) - k(u, v).$$

Feltehetjük, hogy $c'(u, v) \geqslant 0$, ellenkező esetben nyilván nem létezhet folyam a $\mathcal{H}$ hálózatban. Adjunk még $\mathcal{H}'$-höz minden $v \in V$ csúcsra egy (S, v) és egy (v, T) élt, amelyek kapacitása legyen

$$c'(S, v) = \sum_{(u,v) \in E} k(u, v) \quad \text{és} \quad c'(v, T) = \sum_{(v,w) \in E} k(v, w),$$

vagyis a v-be érkező, illetve a v-ből kimenő G-beli élek alsó korlátjainak összege. Jegyezzük meg, hogy $c'(S, s) = 0$ és $c'(t, T) = 0$. Végül adjuk még $\mathcal{H}'$-höz a (t, s) élt, amelynek kapacitása legyen d.

Állítás. A $\mathcal{H} = (G, s, t, c, k)$ hálózatban akkor és csak akkor létezik legfeljebb d értékű folyam, ha a $\mathcal{H}'$ hálózatban a maximális folyamérték

$$\sum_{(u,v) \in E} k(u, v).$$

Megjegyezzük, hogy a $\mathcal{H}'$-re vonatkozó állítás egyenértékű azzal, hogy egy maximális folyam esetén az (S, v) és (v, T) éleken áthaladó folyamértékek megegyeznek az élek kapacitásaival minden $v \in V$ csúcsra, ugyanis az

$(\{S\}, V \cup \{T\})$ és az $(\{S\} \cup V, \{T\})$ vágások kapacitása éppen

$$\sum_{(u,v)\in E} k(u,v).$$

Azt is jegyezzük meg, hogy az Edmonds-Karp algoritmus felhasználásával $O(|V||E|^2)$ költséggel meghatározható a maximális folyamérték a $\mathcal{H}'$ hálózaton.

Bizonyítás. Tegyük fel először, hogy $\mathcal{H}'$-ben a maximális folyamérték

$$\sum_{(u,v)\in E} k(u,v).$$

Legyen f egy maximális folyam $\mathcal{H}'$-ben. Ebből egy g folyamot konstruálunk $\mathcal{H}$-ban. Módosítjuk f-t. Egymás után véve az $(u,v) \in E$ éleket

- vonjunk le $k(u,v)$-t $f(u,T)$-ből és $f(S,v)$-ből,
- adjunk hozzá $k(u,v)$-t $f(u,v)$-hez.

A módosítások során a folyamokra vonatkozó "megmaradási" feltételt a G összes csúcsában megtartottuk: egy csúcsból kimenő, illetve oda beérkező összes folyamból mindig ugyanannyit vontunk le az egyik élen, mint amennyit hozzáadtunk egy másikon.

A módosítások után az S-hez és T-hez illeszkedő éleken áramló folyamérték 0, így ezeket figyelmen kívül hagyhatjuk. Ebből következik, hogy a G élein értelmezett

$$g(u,v) = f(u,v) + k(u,v)$$

függvény kielégíti a $\mathcal{H}$-beli folyamokra vonatkozó "megmaradási" feltételt.

Tekintsük most G egy tetszőleges (u,v) élét. A

$$0 \leqslant f(u,v) \leqslant c'(u,v) = c(u,v) - k(u,v)$$

egyenlőtlenségekből adódik, hogy

$$k(u,v) \leqslant f(u,v) + k(u,v) \leqslant c(u,v).$$

Ám itt a középen álló mennyiség nem más, mint $g(u,v)$.

Így g tényleg egy folyam a $\mathcal{H} = (G,s,t,c,k)$ hálózaton. A g folyam értéke világos módon megegyezik a $\mathcal{H}'$ hálózat f folyamának a (t,s) élen átfolyó értékével. Mivel $c'(t,s) = d$, ezért $|g| \leqslant d$.

A megfordítást úgy bizonyítjuk, hogy egy legfeljebb d értékű $\mathcal{H}$-beli g folyamból egy alkalmas f-et konstruálunk. Ezt lényegében az előző eljárás fordítottjával tehetjük meg. Először egy f' függvényt definiálunk a $\mathcal{H}'$ élein. Legyen

- $f'(t,s) = |g|$,
- $f'(u,v) = g(u,v)$ minden $(u,v) \in E$ esetén,
- $f'(S,v) = f'(v,T) = 0$ minden $v \in V$ esetén.

Az így kapott f' függvény nyilván kielégíti a $\mathcal{H}'$-beli folyamokra vonatkozó "megmaradási" feltételt. Ezután módosítjuk f'-t. Egymás után véve az $(u,v) \in E$ éleket

- adjunk $k(u,v)$-t $f'(u,T)$-hez és $f'(S,v)$-hez,
- vonjunk le $k(u,v)$-t $f'(u,v)$-ből.

Az előzőekhez hasonló módon látható, hogy az eredményül kapott f függvény egy folyam a $\mathcal{H}'$ hálózatban, amelyre

$$|f| = \sum_{(u,v)\in E} k(u,v),$$

így maximális. Ezzel a bizonyítás teljes.

Jegyezzük meg, hogy a $\mathcal{H}$ hálózatban

$$d = \sum_{(v,t)\in E} c(v,t)$$

felső korlát bármely folyam értékére, így ezzel a d-vel futtatva az algoritmust eldönthetjük, hogy létezik-e folyam $\mathcal{H}$-ban.

És még egy fontos észrevétel. Tegyük fel, hogy a $\mathcal{H} = (G,s,t,c,k)$ hálózatban a kapacitások és az alsó korlátok mind nem negatív egész számok. Legyen továbbá d szintén nem negatív egész. Ha most $\mathcal{H}$-ban létezik olyan g folyam, amelyre $|g| \leqslant d$, akkor létezik olyan g^* folyam is, amelyre $|g^*| \leqslant d$ szintén teljesül, ezen felül a $g^*(u,v)$ értékek mind egészek.

Esetünkben a $\mathcal{H}$ hálózathoz konstruált $\mathcal{H}'$ hálózat kapacitásai mind egészek lesznek. Ebben létezik egészértékű maximális folyam, amelynek értéke a feltétel miatt

$$\sum_{(u,v)\in E} k(u,v).$$

Egy ilyen egészértékű maximális folyamból konstruált g^* folyam pedig szintén egészértékű lesz.

Piackutatás

Egy vállalat k féle terméket állít elő, jelölje ezeket $P_1, P_2, \ldots, P_k$, amelyeket saját maga értékesít. A vállalatnak hosszú időre visszamenőleg rendelkezésére áll, hogy a vásárlóik mikor, mely termékeket vásárolták meg (törzsvásárlói kártya). A vállalat felmérést szeretne végezni, hogy mely termékekkel vannak a vásárlóik leginkább megelégedve. Ebből a célból kiválasztanak n vásárlót, jelölje őket $V_1, V_2, \ldots, V_n$, akiknek személyre szóló kérdőíveket küldenek. A kérdőívek összeállításának szempontjai a következők:

- A kérdőívek minden vásárlótól csak az általa megvásárolt termékek felől érdeklődnek.
- Minden $1 \leqslant i \leqslant n$ esetén a V_i vásárlónak küldött kérdőív legalább c_i és legfeljebb c'_i termék felől érdeklődik.
- Minden $1 \leqslant j \leqslant k$ esetén a P_j termékről a kérdőívek legalább r_j és legfeljebb r'_j különböző vásárlótól érdeklődnek.

Adjunk hatékony algoritmust annak eldöntésére, hogy meg lehet-e szerkeszteni a kérdőíveket az adott szempontok szerint, és ha igen, az algoritmus szerkessze is meg a ezeket!

Egy $\mathcal{H}$ hálózatot készítünk, amelyben minden $1 \leqslant i \leqslant n$ esetén a V_i vásárlónak megfelel egy v_i csúcs és minden $1 \leqslant j \leqslant k$ esetén a P_j terméknek egy p_j csúcs. Ha a V_i vásárló vásárolt a P_j termékből, akkor a v_i csúcsból menjen egy irányított él a p_j csúcsba. Ezeknek az éleknek a kapacitása legyen 1, az alsó korlátjuk pedig 0. Vegyünk fel még egy s forrást és egy t nyelőt. Minden $1 \leqslant i \leqslant n$ esetén menjen egy irányított él s-ből v_i-be, amelynek kapacitása c'_i, alsó korlátja pedig c_i, valamint minden $1 \leqslant j \leqslant k$ esetén menjen egy irányított él p_j-ből t-be, amelynek kapacitása r'_j, alsó korlátja pedig r_j.

Állítás. A kérdőíveket akkor és csak akkor lehet az adott szempontok szerint megszerkeszteni, ha a $\mathcal{H}$ hálózatban létezik folyam.

Bizonyítás. Tegyük fel először, hogy a kérdőíveket meg lehet szerkeszteni az adott szempontok szerint. Ha a V_i vásárlótól érdeklődünk a P_j termék felől, akkor küldjünk egységnyi folyamot az (s, v_i, p_j, t) úton s-ből t-be. Ezt minden ilyen (V_i, P_j) párra elvégezve világos módon egy olyan folyamot kapunk, amely az összes kapacitás és alsó korlát feltételt kielégíti.

Megfordítva, tegyük fel, hogy a hálózatban létezik folyam. Ekkor létezik egészértékű folyam is a hálózaton. Ha most a (v_i, p_j) élen egységnyi folyam halad át, akkor a P_j termék felől érdeklődjünk a V_i vásárlótól. A kapacitás és

alsó korlát feltételek miatt így az összes szempontnak megfelelő kérdőívekhez jutunk.

Repülőgépek ütemezése

Tegyük fel, hogy egy légitársaság a $J_1, J_2, \ldots, J_m$ járatokat szeretné üzemeltetni, és d darab azonos típusú repülőgépe van erre a célra. Minden (J_i, J_k) járatpárra ismert, hogy van-e elég idő arra, hogy J_i teljesítése után egy gép felkészíthető a J_k repülésre. Ha egy (J_i, J_k) párra igen a válasz, akkor röviden azt fogjuk mondani, hogy J_k követheti J_i-t.

Adjunk hatékony algoritmust annak eldöntésére, hogy üzemeltethető-e az összes járat legfeljebb d repülőgéppel!

Egy $\mathcal{H}$ hálózatot készítünk, amelyben minden $1 \leqslant i \leqslant m$ esetén a J_i járatnak két csúcs, v_i és v_i', valamint a (v_i, v_i') él felel meg. Ha J_k követheti J_i-t, akkor menjen egy irányított él v_i'-ből v_k-ba. Vegyünk fel még egy s forrást és egy t nyelőt, és minden $1 \leqslant i \leqslant m$ esetén menjen egy-egy irányított él s-ből v_i-be, illetve v_i'-ből t-be. Az összes él kapacitása legyen 1, a (v_i, v_i') élek alsó korlátja szintén 1, a többi él alsó korlátja pedig 0.

Állítás. A $J_1, J_2, \ldots, J_m$ járatok akkor és csak akkor üzemeltethetők d repülőgéppel, ha a $\mathcal{H}$ hálózatban létezik olyan g folyam, amelyre $|g| \leqslant d$.

Bizonyítás. Tegyük fel először, hogy létezik olyan folyam a hálózaton, amelynek értéke kisebb vagy egyenlő, mint d. A kapacitások és korlátok egész volta miatt ezen folyamok között van olyan, amely minden élen egész értéket vesz fel. Jelöljön g egy ilyen folyamot. Szintén a kapacitások és korlátok figyelembe vételével adódik, hogy g értéke minden élen 0 vagy 1.

Tekintsük azt a részgráfot, amelynek élei az egységnyi folyamot szállító élek. Ez felbontható legfeljebb d darab

$$(s, v_{i_1}, v_{i_1}', v_{i_2}, v_{i_2}', \ldots, v_{i_l}, v_{i_l}', t)$$

út egyesítésére. Valóban, tekintsünk egy olyan (s, v_i) élt, amelyen elindul egy egység folyam. Ezt a (v_i, v_i') él szállítja tovább, majd a folyam valamelyik v_i'-ből induló élen folytatja útját, míg végül t-be jut. Eközben egy s-ből t-be menő utat járunk be. Ezt megismételve minden olyan (s, v_j) élre, amelyen elindul egy egység folyam, végül a kívánt s-ből t-be menő utakat kapjuk.

Két ilyen útnak s-en és t-n kívül nyilván nincs közös csúcsa. Egy ilyen út létezése azt jelenti, hogy a $J_{i_1}, J_{i_2}, \ldots, J_{i_l}$ járatok egy géppel teljesíthetők. A (v_i, v_i') élekre kiszabott alsó korlát feltételek miatt minden (v_i, v_i') él rajta van valamelyik ilyen úton. Tehát legfeljebb d repülőgéppel tényleg minden járat lebonyolítható.

Megfordítva, tegyük fel, hogy van egy legfeljebb d repülőgépet használó, minden járatot teljesítő ütemezésünk. Nézzük ebben egy gép egymás utáni feladatait. Ha ezek a $J_{i_1}, J_{i_2}, \ldots, J_{i_l}$ járatok (ebben a sorrendben), akkor az

$$(s, v_{i_1}, v'_{i_1}, v_{i_2}, v'_{i_2}, \ldots, v_{i_l}, v'_{i_l}, t)$$

út éleit a hálózat tartalmazza. Ezen az úton egységnyi folyamot küldhetünk s-ből t-be. Ezt minden repülőgépre elvégezve egy legfeljebb d értékű g folyamot kapunk.

A kapacitásfeltételek teljesülnek, mert az így kapott utaknak nincs közös élük, hiszen minden járat egy repülőgéphez tartozik. A (v_i, v'_i) élekre kiszabott alsó korlát feltételek sem sérülnek, mert az ütemezésben minden járat teljesítve van. Így g eleget tesz a követelményeknek.

Stabil párosítás

Tekintsük fiúk egy F és lányok egy L halmazát. Tegyük fel, hogy a két halmaz ugyanolyan elemszámú. Most az $F \times L = \{(f, l) \mid f \in F,\ l \in L\}$ halmaz egy S részhalmazát teljes párosításnak nevezzük, ha minden F-beli fiú és minden L-beli lány pontosan egy S-beli rendezett párban fordul elő.

Minden $f \in F$ fiú rangsorolja az összes L-beli lányt; az f fiú rangsorában egy l lány pontosan akkor előz meg egy l' lányt, ha f-nek jobban tetszik l, mint l' (holtverseny nincs). A lányok ugyanígy rangsorolják a fiúkat.

Legyen S egy teljes párosítást, továbbá legyenek (f, l) és (f', l') olyan S-beli párok, amelyek esetén f-nek jobban tetszik l', mint l, és l'-nek jobban tetszik f, mint f'. Ilyenkor azt mondjuk, az (f, l') pár instabilitást jelent S-re nézve. Célunk olyan S teljes párosítás megadása, amelyre nézve nincs instabilitási tényező; egy ilyen párosítást stabilnak nevezünk.

Gale-Shapley algoritmus

Az algoritmus szemléletesen a következőképpen írható le.

- Kezdetben senkinek nincs párja.
- Amíg van olyan fiú, akinek éppen nincs párja és még nem tett ajánlatot minden lánynak ismételjük a következőt. Vegyünk egy ilyan f fiút. Legyen l az a lány, aki az f fiúnak a legjobban tetszik azok közül, akiknek még nem tett ajánlatot. Ajánlja fel az f fiú az l lánynak, hogy legyenek egy pár. Ha az l lánynak éppen nincs párja, akkor elfogadja az ajánlatot és ettől kezdve egy párt alkotnak. Kicsit bonyolultabb a helyzet ha az l lánynak már van párja, mondjuk az f' fiú. Ha az f' fiú jobban tetszik az l lánynak, mint az f fiú, akkor l kosarat ad f-nek, a párok nem változnak. Ellenkező esetben az l lány az f' fiúnak int búcsút (ekkor is azt fogjuk mondani, hogy a lány kikosarazta a fiút) és új párja az f fiú lesz.

- Ha már nincs olyan fiú, akinek nincs párja és még nem tett ajánlatot az összes lánynak, akkor az algoritmus befejeződik. A kialakult párokat véglegesítjük; akinek ekkor nincs párja, az egyedül marad.

Az egyszerűség kedvéért legyen $F = \{1, 2, \ldots, n\}$ és $L = \{1, 2, \ldots, n\}$. A fiúk preferencialistáit egy FiúPref$[1 : n, 1 : n]$ tömbben tároljuk, itt FiúPref$[f, i]$ az a lány, aki az f fiú preferencialistáján az i-edik helyen áll. Hasonlóan, a lányok preferencialistáit egy LányPref$[1 : n, 1 : n]$ tömbben tároljuk, itt LányPref$[l, i]$ az a fiú, aki az l lány preferencialistáján az i-edik helyen áll.

Használunk még egy InvLányPref$[1 : n, 1 : n]$ tömböt, InvLányPref$[l, f]$ azt mutatja meg, hogy az f fiú hányadik helyen áll az l lány preferencialistáján. Az InvLányPref$[1 : n, 1 : n]$ tömb egyszerűen felépíthető a LányPref$[1 : n, 1 : n]$ tömbből.

Azokat a fiúkat, akiknek éppen nincs párja egy Q sorban tároljuk. Az ajánlat$[1 : n]$ tömb ajánlat$[f]$ eleme azt mutatja meg, hogy az f fiú eddig hány lánynak tett ajánlatot.

Végül a pár$[1 : n]$ tömb pár$[l]$ eleme az a fiú, aki az l lány aktuális párja. Ha az l lánynak éppen nincs párja, akkor pár$[l] = 0$.

```
StabilPárosítás(FiúPref[1:n,1:n],LányPref[1:n,1:n])
for i=1 to n do
  for l=1 to n do
    InvLányPref[l,LányPref[l,i]]=i
for l=1 to n do
  pár[l]=0
Sorinicializálás(Q)
for f=1 to n do
  ajánlat[f]=0
  Sorba(Q,f)
while Q<>EMPTYSET do
  f=Sorból(Q)
  ajánlat[f]=ajánlat[f]+1
  l=FiúPref[f,ajánlat[f]]
  if pár[l]=0
    then
      pár[l]=f
    else
      f'=pár[l]
      if InvLányPref[l,f]>InvLányPref[l,f']
        then
```

```
          pár[l]=f
          if ajánlat[f']<n then Sorba(Q,f')
        else
          if ajánlat[f]<n then Sorba(Q,f)
for l=1 to n do
  if pár[l]<>0 then
    print '(' l ',' pár[l] ')'
```

Mekkora az algoritmus költsége? Az inicializálás költsége nyilván $O(n^2)$. A while ciklus iterációinak száma legfeljebb n^2, hiszen minden iterációban egy fiú ajánlatot tesz egy olyan lánynak, akinek addig még nem tett ajánlatot és a lehetséges (fiú, lány) párok száma n^2. A while ciklus egy iterációja világos módon konstans költségű, ezért a while ciklus költsége is $O(n^2)$. Így az algoritmus teljes költsége $O(n^2)$.

Helyesség

Lássuk ezután az algoritmus helyességének bizonyítását. Néhány egyszerű észrevétellel kezdünk.

(1) Ha egy fiúnak végül lesz párja, akkor ő minden olyan lánynak tett ajánlatot, aki a párjánál jobban tetszik neki.

(2) Ha egy fiúnak végül nem lesz párja, akkor ő az összes lánynak tett ajánlatot.

(3) Ha egy lánynak legalább egy fiú ajánlatot tesz, akkor a lánynak végül lesz párja.

(4) Ha egy lánynak legalább egy fiú ajánlatot tesz, akkor a lánynak az a fiú lesz végül a párja, aki az ajánlattevői közül legjobban tetszik neki.

Állítás. A `StabilPárosítás` algoritmus teljes párosítást szolgáltat.

Bizonyítás. Indirekt módon bizonyítunk. Tegyük fel, hogy az állítással ellentétben valamelyik f fiúnak nem lett párja. A (2) észrevétel szerint ekkor f minden lánynak tett ajánlatot. Ez azt jelenti, hogy minden lánynak tett ajánlatot valaki, így a (3) észrevétel szerint minden lánynak lett végül párja. Ez azonban csak úgy lehet, ha az összes fiúnak is lett végül párja, hiszen a fiúk és a lányok száma ugyanannyi. Ellentmondásra jutottunk, így az állítás igaz.

Állítás. A `StabilPárosítás` algoritmus által szolgáltatott párosításra nézve nincs instabilitási tényező.

Bizonyítás. Ismét indirekt módon bizonyítunk. Tegyük fel, hogy az állítással ellentétben van olyan (f, l) páros, amely instabilitási tényező. Jelölje f-nek és l-nek a `StabilPárosítás` algoritmus által szolgáltatott párját l' és f'.

Az (1) észrevétel szerint f ajánlatot tett l-nek még l' előtt. Mivel végül l' lett f párja, így valamikor l kikosarazta őt. A (4) észrevétel szerint l-nek végül az a fiú lett a párja, aki az ajánlattevői közül a legjobban tetszett neki, így l-nek szükségképpen jobban tetszik f', mint f. Viszont ez azt jelenti, hogy az (f, l) páros nem instabilitási tényező. Ellentmondásra jutottunk, így az állítás igaz.

Optimalitás

Vizsgáljuk meg, hogy a `StabilPárosítás` algoritmus a fiúknak vagy a lányoknak kedvez-e inkább! Mivel a lányoknak a nekik ajánlatot tevő fiúk közül a nekik legjobban tetsző lesz a párja, míg a fiúknak azon lányok közül, akiknek ajánlatot tettek a nekik legkevésbé tetsző lesz a párja, azt gondolhatnánk, hogy a lányok járnak jobban. A valóság ennek éppen az ellenkezője.

Állítás. A `StabilPárosítás` algoritmus minden fiúhoz a számára szóba jövő lányok közül a neki legjobban tetszőt párosítja. (Egy fiú számára egy lány akkor jön szóba, ha van olyan stabil párosítás, amelyben a fiú és a lány egy párt alkotnak.)

Bizonyítás. Indirekt módon bizonyítunk. Tegyük fel, hogy az állítással ellentétben van olyan fiú, akihez a `StabilPárosítás` algoritmus a számára szóba jövő lányok közül nem a neki legjobban tetszőt párosítja.

Tekintsük a legelső olyan iterációt, amikor egy fiút a számára szóba jövő lányok közül a neki legjobban tetsző kikosaraz. Legyen a fiú f, a lány pedig l. Legyen továbbá f' az a fiú, aki miatt l kikosarazta f-et. Ekkor persze l-nek jobban tetszik f', mint f. Mivel a legelső olyan iterációban vagyunk, amikor egy fiút a számára szóba jövő lányok közül a neki legjobban tetsző kikosaraz, f'-t még biztos nem kosarazta ki a számára szóba jövő lányok közül a neki legjobban tetsző. Legyen ez utóbbi lány l^*. Világos, hogy f'-nek legalább annyira tetszik l, mint l^* (itt l és l^* nem feltétlenül különböző lányok).

Legyen S egy olyan stabil párosítás, ahol f és l egy párt alkot. Ilyen stabil párosítás létezik, hisz l az f fiú számára szóba jövő lányok között van. Legyen S-ben f' párja l'. Nyilvánvaló módon f'-nek legalább annyira tetszik

l^*, mint l' (itt sem feltétlenül különböző lányok l^* és l'). Mivel f'-nek legalább annyira tetszik l, mint l^*, és legalább annyira tetszik l^*, mint l', ezért f'-nek legalább annyira tetszik l, mint l' (jegyezzük meg, hogy l és l' különböző lányok).

Így az S stabil párosítás (f, l) és (f', l') párjait tekintve azt látjuk, hogy l-nek jobban tetszik f', mint f, és f'-nek jobban tetszik l, mint l', vagyis az (f', l) páros instabilitást jelent S-re nézve. Ellentmondásra jutottunk, így az állítás igaz.

Állítás. A `StabilPárosítás` algoritmus minden lányhoz a számára szóba jövő fiúk közül a neki legkevésbé tetszőt párosítja. (Egy lány számára egy fiú akkor jön szóba, ha van olyan stabil párosítás, amelyben a lány és a fiú egy párt alkotnak.)

Bizonyítás. Ismét indirekt módon bizonyítunk. Tegyük fel, hogy az állítással ellentétben van olyan S stabil párosítás, ahol valamelyik l lány rosszabbul jár, mint a `StabilPárosítás` algoritmus esetén.

Legyen l-nek a `StabilPárosítás` algoritmus által szolgáltatott párja f, az S-beli párja pedig f'. Feltételünk szerint l-nek jobban tetszik f, mint f'. Legyen továbbá az f fiú S-beli párja az l' lány. Az előző állítás szerint a `StabilPárosítás` algoritmus f-hez a számára szóba jövő lányok közül a neki legjobban tetszőt párosítja. Ebből következik, hogy f-nek jobban tetszik l, mint l'.

Így az S stabil párosítás (f', l) és (f, l') párjait tekintve azt látjuk, hogy l-nek jobban tetszik f, mint f', és f-nek jobban tetszik l, mint l', vagyis az (f, l) páros instabilitást jelent S-re nézve. Ellentmondásra jutottunk, így az állítás igaz.

Közelítő algoritmusok

Minimális összsúlyú éllefogó csúcshalmaz

Tekintsünk egy $G = (V, E)$ irányítatlan gráfot. A G gráf csúcsainak egy U részhalmazát éllefogó halmaznak nevezzük, ha G bármely élének van U-beli végpontja. Tegyük fel, hogy adott a csúcsok halmazán egy $w\colon V \to \mathbb{R}_0^+$ nem negatív súlyfüggvény. Egy U éllefogó halmaz $w(U)$ súlya legyen a benne szereplő csúcsok súlyainak összege. Adjuk meg a G gráf egy minimális összsúlyú éllefogó csúcshalmazát!

Sajnos a feladat megoldására nem ismert hatékony eljárás! Az alábbi algoritmus egy olyan éllefogó csúcshalmazt szolgáltat, amelynek összsúlya legfeljebb kétszerese az optimális éllefogó csúcshalmaz összsúlyának.

A G gráf minden $e \in E$ éléhez rendeljünk hozzá egy $\pi(e)$ nem negatív potenciálértéket. Azt mondjuk, hogy a $\pi\colon E \to \mathbb{R}_0^+$ potenciálfüggvény kompatibilis a w súlyfüggvénnyel, ha tetszőleges $v \in V$ csúcsra

$$\sum_{(u,v)\in E} \pi(u, v) \leqslant w(v),$$

azaz a v csúcsra illeszkedő élek potenciálértékeinek összege nem haladja meg a v csúcs súlyát. Ha a v csúcsra illeszkedő élek potenciálértékeinek összege megegyezik a v csúcs súlyával, akkor v-t telített csúcsnak nevezzük.

Kezdetben minden $e \in E$ élre legyen $\pi(e) = 0$ és legyen $U = \emptyset$. Ezek után amíg van olyan él, amelynek egyik végpontja sem telített csúcs, ismételjük a következőt. Válasszunk egy ilyen e élt, és növeljük a $\pi(e)$ potenciálértéket, amíg e legalább egyik végpontja telített csúccsá nem válik. Amelyik végpont így telített csúccsá vált, azt vegyük hozzá az U halmazhoz (ha e mindkét végpontja telített csúccsá vált, akkor mindkettőt hozzávesszük U-hoz).

Világos, hogy az algoritmus befejeződése után a π potenciálfüggvény kompatibilis a w súlyfüggvénnyel. Megmutatjuk, hogy a kapott U halmaz éllefogó csúcshalmaz. Indirekt tegyük fel, hogy ez nem igaz, vagyis van olyan (u, v)

él, amelynek egyik végpontja sem tartozik hozzá U-hoz. Ekkor u és v nem telített csúcsok. Ez viszont lehetetlen, hiszen az algoritmus nem fejeződik be addig, amíg van olyan él, amelynek egyik végpontja sem telített csúcs.

Mivel minden $v \in U$ csúcs telített, ezért

$$w(U) = \sum_{v \in U} w(v) = \sum_{v \in U} \sum_{(u,v) \in E} \pi(u,v).$$

Itt a jobb oldali kettős összegben minden él legfeljebb kétszer fordulhat elő, így

$$\sum_{v \in U} \sum_{(u,v) \in E} \pi(u,v) \leqslant 2 \sum_{e \in E} \pi(e).$$

Innen

$$w(U) \leqslant 2 \sum_{e \in E} \pi(e)$$

adódik.

Ezek után tekintsünk egy optimális U^* éllefogó csúcshalmazt. A π potenciálfüggvénynek a w súlyfüggvénnyel való kompatibilitása miatt tetszőleges $v \in U^*$ csúcsra

$$\sum_{(u,v) \in E} \pi(u,v) \leqslant w(v).$$

Összegezzük ezeket az egyenlőtlenségeket az összes $v \in U^*$ csúcsra:

$$\sum_{v \in U^*} \sum_{(u,v) \in E} \pi(u,v) \leqslant \sum_{v \in U^*} w(v) = w(U^*).$$

Mivel U^* éllefogó csúcshalmaz, ezért a bal oldalon levő kettős összegben minden él legalább egyszer előfordul, így

$$\sum_{e \in E} \pi(e) \leqslant \sum_{v \in U^*} \sum_{(u,v) \in E} \pi(u,v).$$

Innen

$$\sum_{e \in E} \pi(e) \leqslant w(U^*)$$

adódik.

Összevetve a két végső egyenlőtlenséget

$$w(U) \leqslant 2 \sum_{e \in E} \pi(e) \leqslant 2w(U^*)$$

következik.

Terheléselosztás

Tegyük fel, hogy adottak az $a_1, a_2, \ldots, a_n$ feladatok, amelyek végrehajtásához a $b_1, b_2, \ldots, b_m$ erőforrások állnak rendelkezésre. Egy erőforrás egyszerre egy feladattal tud foglalkozni, és ha valamelyiket elkezdte, akkor azt addig nem hagyja abba, amíg teljesen el nem készült vele. Minden a_i feladathoz adott a t_i végrehajtási ideje.

Minden a_i feladatot rendeljük hozzá valamelyik b_j erőforráshoz, ezen fogjuk a feladatot elvégezni. Egy erőforráshoz természetesen több feladatot is hozzárendelhetünk. Minden $1 \leqslant j \leqslant m$ esetén jelölje A_j azon i indexek halmazát, amelyekre az a_i feladatot a b_j erőforráshoz rendeltük. Jelölje továbbá

$$T_j = \sum_{i \in A_j} t_i$$

a b_j erőforrás terhelését, vagyis azt az időt, amely a b_j erőforráshoz rendelt feladatok elvégzéséhez szükséges. Adjunk meg egy olyan hozzárendelést, amelynél a $T = \max\{T_j \mid 1 \leqslant j \leqslant m\}$ maximális terhelés a lehető legkisebb!

Sajnos a feladat megoldására nem ismert hatékony eljárás! A következő algoritmus egy olyan hozzárendelést szolgáltat, amelynél a maximális terhelés legfeljebb másfélszerese az optimális megoldáshoz tartozó maximális terhelésnek.

Tegyük fel, hogy a feladatok a végrehajtási idejük szerint monoton csökkenően rendezettek (ha ez nem teljesül, akkor először rendezzük őket):

$$t_1 \geqslant t_2 \geqslant \cdots \geqslant t_n.$$

A feladatokat ebben a sorrendben rendeljük hozzá az erőforrásokhoz. Minden $1 \leqslant i \leqslant n$ esetén az a_i feladatot ahhoz a b_j erőforráshoz rendeljük, amelynek addigi terhelése (az addig hozzárendelt feladatok végrehajtási idejének összege) a legkisebb. Jegyezzük meg, hogy kupac adatszerkezettel ez hatékonyan megvalósítható.

Jelölje T az algoritmusunk által előállított hozzárendelésnél a maximális terhelést, T^* pedig egy optimális hozzárendelésnél a maximális terhelést. Megmutatjuk, hogy $T \leqslant \frac{3}{2}T^*$.

Ha $n \leqslant m$, akkor az állítás triviálisan igaz; ebben az esetben algoritmusunk egy optimális hozzárendelést állít elő. Tegyük fel ezért, hogy $n > m$. Legyen b_k az az erőforrás, amelynek az algoritmusunk által előállított hozzárendelésnél a legnagyobb a terhelése, és legyen a_l az a feladat, amelyet az algoritmusunk utoljára rendelt ehhez az erőforráshoz. Ha algoritmusunk a b_k erőforráshoz csak az a_l feladatot rendelte hozzá, akkor az állítás megint

csak triviálisan igaz; ebben az esetben is algoritmusunk egy optimális hozzárendelést állít elő. Tegyük fel ezért, hogy algoritmusunk a b_k erőforráshoz legalább két feladatot rendelt hozzá. Mivel algoritmusunk az első m feladatot különböző erőforrásokhoz rendeli, ezért $l \geqslant m+1$.

Tekintsük azt a pillanatot, amikor algoritmusunk hozzárendeli az a_l feladatot a b_k erőforráshoz. Ekkor a b_k erőforrásnak a legkisebb a terhelése. A b_k erőforrás terhelése ebben a pillanatban $T_k - t_l$, amiből következik, hogy a többi erőforrás terhelése is legalább ennyi. Az erőforrások terhelése a hátralevő időben nyilván nem csökken, így algoritmusunk befejezésekor $T_j \geqslant T_k - t_l$ minden $1 \leqslant j \leqslant m$ esetén. Adjuk össze ezeket az egyenlőtlenségeket:

$$\sum_{j=1}^{m} T_j \geqslant m(T_k - t_l),$$

illetve átrendezve

$$T_k - t_l \leqslant \frac{1}{m} \sum_{j=1}^{m} T_j.$$

Az erőforrások terhelésének összege nyilván megegyezik a feladatok végrehajtási idejének összegével:

$$\sum_{j=1}^{m} T_j = \sum_{i=1}^{n} t_i.$$

Behelyettesítünk az előző egyenlőtlenségbe:

$$T_k - t_l \leqslant \frac{1}{m} \sum_{i=1}^{n} t_i.$$

Vegyük szemügyre a jobb oldalt. Ez a végrehajtási idők összegének az m-ed része. Világos, hogy akármilyen módon is rendeljük hozzá a feladatokat az erőforrásokhoz, mindig lesz olyan erőforrás, amelynek terhelése legalább ennyi. Ebből következik, hogy

$$T^* \geqslant \frac{1}{m} \sum_{i=1}^{n} t_i.$$

Ezt az egyenlőtlenséget az előzővel kombinálva kapjuk, hogy

$$T_k - t_l \leqslant T^*.$$

Tekintsük ezután az első $m+1$ feladatot. Mivel a feladatok a végrehajtási idejük szerint csökkenően rendezettek, ezeknek a feladatoknak a végrehajtási

ideje legalább t_l. Mivel csak m erőforrás áll rendelkezésre, akármilyen módon is rendeljük hozzá a feladatokat az erőforrásokhoz, mindig lesz az első $m+1$ feladat között két olyan, amelyeket ugyanahhoz az erőforráshoz rendeltünk. Ennek az erőforrásnak a terhelése világos módon legalább $2t_l$. Ebből következik, hogy

$$T^* \geqslant 2t_l,$$

illetve átrendezve

$$t_l \leqslant \tfrac{1}{2}T^*.$$

Adjuk össze a kapott két egyenlőtlenséget:

$$(T_k - t_l) + t_l \leqslant T^* + \tfrac{1}{2}T^*,$$

azaz

$$T_k \leqslant \tfrac{3}{2}T^*.$$

Innen $T = T_k$ felhasználásával az állítás adódik.

Kifinomultabb elemzéssel megmutatható, hogy az algoritmusunk által előállított hozzárendelésre $T \leqslant \frac{4}{3}T^*$ is teljesül.

Kiszolgáló egységek telepítése

Adott egy n elemű $P = \{p_1, p_2, \ldots, p_n\}$ pontrendszer a síkon. Tegyük fel, hogy a sík bármely két x és y pontjának ismerjük a $d(x, y)$ távolságát. A d távolságfüggvényről itt annyit teszünk fel, hogy

- a sík tetszőleges x és y pontjára $d(x, y) \geqslant 0$,
- a sík tetszőleges x és y pontjára $d(x, y) = 0$ akkor és csak akkor, ha $x = y$,
- a sík tetszőleges x és y pontjára $d(x, y) = d(y, x)$,
- a sík tetszőleges x, y és z pontjára $d(x, y) + d(y, z) \geqslant d(x, z)$.

A szokásos euklideszi távolságfüggvény megfelel ezeknek a feltételeknek, de szóba jöhet más távolságfüggvény is.

Adott továbbá egy $k < n$ pozitív egész szám. Egy k elemű $C = \{c_1, c_2, \ldots, c_k\}$ síkbeli pontrendszer fedési sugara legyen az

$$r(C) = \max_{1 \leqslant i \leqslant n}(\min_{1 \leqslant j \leqslant k} d(p_i, c_j))$$

mennyiség. Adjunk meg egy olyan k elemű C pontrendszert a síkon, amelynek $r(C)$ fedési sugara a lehető legkisebb!

A feladat szemléletesen a következőképpen fogalmazható meg. Tegyük fel, hogy egy áruházlánc k üzlet nyitását tervezi egy olyan régióban, ahol n kistelepülés található. Az üzletek helyét úgy szeretnék meghatározni, hogy a települések lakóinak a lehető legkevesebbet kelljen autózni a hozzájuk legközelebbi üzletig. Hol legyenek az üzletek?

Sajnos a feladat megoldására nem ismert hatékony eljárás! A következő algoritmus egy olyan k elemű C pontrendszert szolgáltat, amelynek fedési sugara legfeljebb kétszerese az optimális megoldás fedési sugarának.

Bevezetünk egy jelölést. A sík egy x pontjára és egy Y nem üres pontrendszerére legyen

$$d(x, Y) = \min\{d(x, y) \mid y \in Y\}.$$

Az algoritmus ezek után a következő.

```
KiszolgálóEgységekTelepítése(P,k)
Legyen p tetszőleges P-beli pont
C={p}
while |C|<k do
   legyen p olyan P-beli pont, amelyre d(p,C) maximális
   C=C∪{p}
return C
```

Az algoritmus érdekes vonása, hogy C pontjait a P-beli pontok közül választja ki.

Legyen $C^* = \{c_1, c_2, \ldots, c_k\}$ egy optimális megoldása a feladatnak és legyen $r = r(C^*)$. Indirekt tegyük fel, hogy az algoritmus által előállított C pontrendszerre $r(C) > 2r$. Ekkor létezik olyan $p_{i_0} \in P$ pont, amely C minden pontjától nagyobb, mint $2r$ távolságra van, vagyis amelyre $d(p_{i_0}, C) > 2r$.

Minden $1 \leqslant j \leqslant k$ esetén jelölje p_{i_j} azt a P-beli pontot, amelyet az algoritmus a j-edik lépésben vett hozzá a C halmazhoz. Most minden $2 \leqslant j \leqslant k$ esetén

$$d(p_{i_j}, \{p_{i_1}, \ldots, p_{i_{j-1}}\}) \geqslant d(p_{i_0}, \{p_{i_1}, \ldots, p_{i_{j-1}}\}) \geqslant d(p_{i_0}, C) > 2r.$$

Ennélfogva a $\{p_{i_1}, \ldots, p_{i_k}\}$ halmaz bármely két pontja nagyobb, mint $2r$ távolságra van egymástól. Sőt ennél több is igaz, a $\{p_{i_0}, p_{i_1}, \ldots, p_{i_k}\}$ halmaz bármely két pontja nagyobb, mint $2r$ távolságra van egymástól.

Tekintsük ezután a C^* pontrendszert. Mivel $r(C^*) = r$, ezért minden $0 \leqslant j \leqslant k$ esetén létezik olyan C^*-beli pont, amely a p_{i_j} ponttól legfeljebb

r távolságra van. Másrészt semelyik $c_s \in C^*$ pont nem lehet két különböző $\{p_{i_0}, p_{i_1}, \ldots, p_{i_k}\}$-beli, mondjuk p_{i_g} és p_{i_h} ponttól legfeljebb r távolságra, ellenkező esetben

$$2r = r + r \geqslant d(p_{i_g}, c_s) + d(p_{i_h}, c_s) \geqslant d(p_{i_g}, p_{i_h}) > 2r$$

teljesülne, ami nyilván lehetetlen.

Ebből következik, hogy $|C^*| \geqslant |\{p_{i_0}, p_{i_1}, \ldots, p_{i_k}\}| = k + 1$, ellentmondás.

Élidegen utak még egyszer

Adott egy $G = (V, E)$ irányított gráf, és adott a gráfban különböző csúcspároknak egy $X = \{(s_1, t_1), (s_2, t_2), \ldots, (s_k, t_k)\}$ halmaza. Feladatunk a lehető legtöbb X-beli csúcspárt összekötni olyan irányított utakkal, amelyek közül semelyik kettőnek nincs közös éle!

Sajnos a feladat megoldására nem ismert hatékony eljárás! A következő (mohó) algoritmus egy olyan $\mathcal{P}$ úthalmazt szolgáltat, amelyre $|\mathcal{P}^*| \leqslant (2\sqrt{|E|} + 1)|\mathcal{P}|$, ahol $\mathcal{P}^*$ egy optimális megoldása a feladatnak.

Tekintsünk egy az (s_i, t_i) csúcspárt összekötő legrövidebb (legkevesebb élből álló) utat minden $1 \leqslant i \leqslant k$ esetén, és válasszunk ki ezek közül egy legrövidebbet, ez kerüljön $\mathcal{P}$-be először. Most tegyük fel, hogy már kiválasztottunk h éldiszjunkt utat, jelölje ezeket $P_{i_1}, P_{i_2}, \ldots, P_{i_h}$, ahol P_{i_l} az (s_{i_l}, t_{i_l}) csúcspárt köti össze minden $1 \leqslant l \leqslant h$ esetén. Ezek után ha van még olyan út a gráfban, amelyik az $X \setminus \{(s_{i_1}, t_{i_1}), (s_{i_2}, t_{i_2}), \ldots, (s_{i_h}, t_{i_h})\}$-beli csúcspárok valamelyikét egy a $P_{i_1}, P_{i_2}, \ldots, P_{i_h}$ utaktól éldiszjunkt úttal összeköti, akkor válasszunk ki ezek közül egy legrövidebbet, és ezzel bővítsük $\mathcal{P}$-t.

Legyen $\mathcal{P}^* = \{P^*_{j_1}, P^*_{j_2}, \ldots, P^*_{j_r}\}$ egy optimális megoldása a feladatnak, ahol $P^*_{j_l}$ az (s_{j_l}, t_{j_l}) csúcspárt köti össze minden $1 \leqslant l \leqslant r$ esetén. Ha $\mathcal{P}^* = \emptyset$, akkor az állítás triviális, így tegyük fel, hogy $\mathcal{P}^* \neq \emptyset$. Ekkor $\mathcal{P}$ se üres halmaz. Külön fogjuk vizsgálni azokat az utakat, amelyek legalább $\sqrt{|E|}$ élből állnak, ezeket hosszú utaknak nevezzük, és külön azokat, amelyek ennél kevesebb élből, ezeket pedig rövid utaknak.

Nyilvánvaló módon $\mathcal{P}^*$ legfeljebb $\sqrt{|E|}$ hosszú utat tartalmazhat, hiszen minden ilyen út legalább $\sqrt{|E|}$ élből áll.

Tekintsük ezek után $\mathcal{P}^*$ rövid útjait, ezek közül is azokat, amelyek olyan csúcspárokat kötnek össze, amelyeket nem köt össze $\mathcal{P}$-beli út. Legyen (s_{j_l}, t_{j_l}) egy ilyen csúcspár. A mohó algoritmus által egymás után választott utak hossza monoton növekvő sorozatot alkot, így szükségképpen kell lenni olyan $\mathcal{P}$-beli P_{i_g} útnak, amelynek hossza nem haladja meg $P^*_{j_l}$ hosszát, és van legalább egy közös éle $P^*_{j_l}$-lel, különben magát $P^*_{j_l}$-t is beválasztotta volna a

mohó algoritmus $\mathcal{P}$-be. Azt fogjuk mondani, hogy a P_{i_g} út blokkolja a $P^*_{j_l}$ utat. Mivel P_{i_g} hossza nem haladja meg $P^*_{j_l}$ hosszát, és $P^*_{j_l}$ rövid út, ezért P_{i_g} is rövid út. Hány $\mathcal{P}^*$-beli rövid utat blokkolhat P_{i_g}? Legfeljebb $\sqrt{|E|}$-t, hiszen legfeljebb ennyi élből áll, és a $\mathcal{P}^*$-beli utak éldiszjunktak. Ennélfogva legfeljebb $\sqrt{|E|}|\mathcal{P}|$ olyan csúcspár lehet, amelyet összeköt egy $\mathcal{P}^*$-beli rövid út, de nem köt össze $\mathcal{P}$-beli út.

Összegezve az eddigieket, $\mathcal{P}^*$ a következőkből áll:

- hosszú utak, ezek száma legfeljebb $\sqrt{|E|}$,
- rövid utak, amelyek $\mathcal{P}$-ben is ott vannak, ezek száma legfeljebb $|\mathcal{P}|$,
- rövid utak, amelyek nincsenek $\mathcal{P}$-ben, ezek száma legfeljebb $\sqrt{|E|}|\mathcal{P}|$.

Így

$$|\mathcal{P}^*| \leqslant \sqrt{|E|} + |\mathcal{P}| + \sqrt{|E|}|\mathcal{P}| \leqslant (2\sqrt{|E|} + 1)|\mathcal{P}|.$$

Bizonyítható, hogy ennél lényegesen jobb közelítő algoritmus csak akkor létezik, ha $P = NP$.

Lokális keresés

Maximális vágás

Legyen $G = (V, E)$ egy $n \geqslant 2$ csúcsú, irányítatlan, súlyozott élű gráf a $w : E \to \mathbb{R}^+$ pozitív súlyfüggvénnyel. A gráf csúcsainak egy (A, B) partícióját, ahol A és B egyike sem üres halmaz, a gráf egy vágásának nevezzük. Egy (A, B) vágás $w(A, B)$ súlyán a gráf azon éleinek az összsúlyát értjük, amelyek egyik végpontja A-ban, a másik pedig B-ben van. Egy (A, B) vágást maximálisnak nevezünk, ha súlyát nem haladja meg a gráf egyetlen más vágásának a súlya sem. Feladat a gráf egy maximális súlyú vágásának a megtalálása.

Sajnos a feladat megoldására nem ismert hatékony eljárás! A következő lokális keresés algoritmus egy olyan vágást szolgáltat, amelynek súlya legalább a fele egy maximális vágás súlyának.

Induljunk ki G egy tetszőleges vágásából, és ha van olyan csúcs, amelyet a vágás egyik csúcshalmazából áthelyezve a másikba a vágás súlya nő, akkor helyezzünk át egy ilyen csúcsot. Ismételjük ezt, amíg található ilyen csúcs. Mivel G-nek csak véges sok vágása létezik, az eljárás véges sok lépésben befejeződik, ellenkező esetben a véges sok vágás súlyból egy szigorúan monoton növekvő végtelen sorozat állna elő.

Mit mondhatunk a kapott vágás súlyáról? Legyen

$$W = \sum_{(u,v) \in E} w(u, v).$$

Az algoritmus megállása után tetszőleges $u \in A$ csúcsra

$$\sum_{\substack{(u,v) \in E \\ v \in A}} w(u, v) \leqslant \sum_{\substack{(u,v) \in E \\ v \in B}} w(u, v).$$

Összegezve ezeket az egyenlőtlenségeket az összes $u \in A$ csúcsra

$$2 \sum_{\substack{(u,v) \in E \\ u,v \in A}} w(u, v) \leqslant \sum_{\substack{(u,v) \in E \\ u \in A,\, v \in B}} w(u, v) = w(A, B).$$

Hasonlóan, tetszőleges $u \in B$ csúcsra

$$\sum_{\substack{(u,v)\in E\\ v\in B}} w(u,v) \leqslant \sum_{\substack{(u,v)\in E\\ v\in A}} w(u,v).$$

Összegezve ezeket az egyenlőtlenségeket az összes $u \in B$ csúcsra

$$2\sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) \leqslant \sum_{\substack{(u,v)\in E\\ u\in A,\, v\in B}} w(u,v) = w(A,B).$$

Összeadva a két kapott egyenlőtlenséget

$$\sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) \leqslant w(A,B),$$

majd mindkét oldalhoz $w(A,B)$-t adva

$$\sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + w(A,B) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) \leqslant 2w(A,B)$$

adódik. A bal oldal itt

$$\sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u\in A,\, v\in B}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) = \sum_{(u,v)\in E} w(u,v) = W,$$

így

$$2w(A,B) \geqslant W.$$

Nyilván egyetlen vágás súlya se lehet nagyobb, mint W, így a kapott vágás súlya legalább a fele egy maximális vágás súlyának.

Mint láttuk, az algoritmus véges sok áthelyezés után befejeződik, de az áthelyezések száma még egész súlyfüggvény esetén is W lehet, ami exponenciális költség. A következő trükkel, a közelítés mértékén alig változtatva az algoritmus polinomiálissá tehető.

Legyen a súlyfüggvény továbbra is egész, és legyen ε tetszőleges pozitív szám, amelyről az egyszerűség kedvéért tegyük fel, hogy egy páros egész szám reciproka. Módosítsuk az algoritmust úgy, hogy egy csúcsot akkor helyezünk csak át egy (A,B) vágás egyik csúcshalmazából a másikba, ha ezzel a vágás súlya nagyobb, mint $\frac{2\varepsilon}{|V|}w(A,B)$ értékkel növekszik.

Az algoritmus megállása után tetszőleges $u \in A$ csúcsra

$$\sum_{\substack{(u,v)\in E\\ v\in A}} w(u,v) \leqslant \sum_{\substack{(u,v)\in E\\ v\in B}} w(u,v) + \frac{2\varepsilon}{|V|}\, w(A,B).$$

Összegezve ezeket az egyenlőtlenségeket az összes $u \in A$ csúcsra

$$\begin{aligned} 2 \sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) &\leqslant \sum_{\substack{(u,v)\in E\\ u\in A,\, v\in B}} w(u,v) + |A|\, \frac{2\varepsilon}{|V|}\, w(A,B) \\ &= \left(1 + |A|\, \frac{2\varepsilon}{|V|}\right) w(A,B). \end{aligned}$$

Hasonlóan, tetszőleges $u \in B$ csúcsra

$$\sum_{\substack{(u,v)\in E\\ v\in B}} w(u,v) \leqslant \sum_{\substack{(u,v)\in E\\ v\in A}} w(u,v) + \frac{2\varepsilon}{|V|}\, w(A,B).$$

Összegezve ezeket az egyenlőtlenségeket az összes $u \in B$ csúcsra

$$\begin{aligned} 2 \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) &\leqslant \sum_{\substack{(u,v)\in E\\ u\in A,\, v\in B}} w(u,v) + |B|\, \frac{2\varepsilon}{|V|}\, w(A,B) \\ &= \left(1 + |B|\, \frac{2\varepsilon}{|V|}\right) w(A,B). \end{aligned}$$

Összeadva a két kapott egyenlőtlenséget

$$\begin{aligned} \sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) &\leqslant \left(1 + (|A| + |B|)\, \frac{\varepsilon}{|V|}\right) w(A,B) \\ &= (1+\varepsilon) w(A,B) \end{aligned}$$

(hiszen $|A| + |B| = |V|$), majd mindkét oldalhoz $w(A,B)$-t adva

$$\sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + w(A,B) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) \leqslant (2+\varepsilon) w(A,B)$$

adódik. A bal oldal itt

$$\sum_{\substack{(u,v)\in E\\ u,v\in A}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u\in A,\, v\in B}} w(u,v) + \sum_{\substack{(u,v)\in E\\ u,v\in B}} w(u,v) = \sum_{(u,v)\in E} w(u,v) = W,$$

így

$$(2+\varepsilon)w(A,B) \geqslant W.$$

Nyilván egyetlen vágás súlya se lehet nagyobb, mint W, így az egyenlőtlenség igaz lesz akkor is, ha a jobb oldalon egy maximális vágás súlya szerepel.

Mit mondhatunk a csúcs áthelyezések számáról? Minden egyes áthelyezéssel a vágás súlya legalább az $(1+2\varepsilon/|V|)$-szeresére nő. Mivel tetszőleges $x \geqslant 1$ esetén $(1+1/x)^x \geqslant 2$, ezért

$$\left(1+\frac{2\varepsilon}{|V|}\right)^{\frac{|V|}{2\varepsilon}} \geqslant 2,$$

ami azt jelenti, hogy minden $\frac{|V|}{2\varepsilon}$ csúcs áthelyezése után a vágás súlya legalább a kétszeresére nő. Figyelembe véve, hogy a vágás súlya nem haladhatja meg a W értéket, ezért az algoritmus $O(\varepsilon^{-1}|V|\log W)$ csúcs áthelyezése után szükségképpen befejeződik.

Randomizált algoritmusok

Gyorsrendezés

Adott különböző számoknak egy $A[1:n]$ tömbje. A tömb rendezésére egy lehetséges módszer a következő. Vegyük a tömb egy véletlen elemét. Mozgassuk a tömb elejére az ennél kisebb, a végére pedig az ennél nagyobb elemeket, majd a kiválasztott elemet helyezzük el a nála kisebbek, illetve a nála nagyobbak közé. Ezzel a kiválasztott elem a rendezés szerinti helyére kerül. Rendezzük ezután a kiválasztott elem előtti és mögötti résztömböket különkülön rekurzív módon. A módszer neve gyorsrendezés.

Lássuk először a partícionáló eljárást, amely az $A[p:r]$ résztömböt rendezi át helyben. Az eljárás a véletlen partícionáló elem (átrendezés utáni) indexével tér vissza. Az összehasonlítások száma $r-p$.

```
Partícionálás(A,p,r)
i=Random(p,r)
csere(A[i],A[r])
i=p-1
for j=p to r-1 do
  if A[j]<A[r] then
    i=i+1
    csere(A[i],A[j])
csere(A[i+1],A[r])
return i+1
```

Ezután jöjjön a rendező algoritmus. Az algoritmust az $(A,1,n)$ paraméterekkel kell meghívni.

```
Gyorsrendezés(A,p,r)
if p<r then
  q=Partícionálás(A,p,r)
  Gyorsrendezés(A,p,q-1)
  Gyorsrendezés(A,q+1,r)
```

Vegyük észre, hogy a rekurzió mélysége a legrosszabb esetben itt a tömb méretében lineáris. A következő változatban a rekurzió mélysége a tömb méretében logaritmikus a legrosszabb esetben is. Az algoritmust az $(A, 1, n)$ paraméterekkel kell meghívni.

```
MódosítottGyorsrendezés(A,p,r)
while p<r do
  q=Partícionálás(A,p,r)
  if q-p<r-q
    then
      MódosítottGyorsrendezés(A,p,q-1)
      p=q+1
    else
      MódosítottGyorsrendezés(A,q+1,r)
      r=q-1
```

Belátjuk, hogy az algoritmus összehasonlítások számában mért költségének várható értéke $O(n \log n)$. Nem megy az általánosság rovására, ha azt gondoljuk, hogy a rendezendő elemek az $1, 2, \ldots, n$ egész számok. Jelölje $T(n)$ az összehasonlítások számának várható értékét. Nyilván $T(0) = T(1) = 0$.

Legyen $T(n, j)$ az összehasonlítások számának várható értéke abban az esetben, amikor a legelőször választott partícionáló elem éppen j. Világos, hogy

$$T(n, j) = n - 1 + T(j - 1) + T(n - j).$$

A partícionáló elem véletlen választásának köszönhetően

$$T(n) = \frac{1}{n} \left(T(n, 1) + T(n, 2) + \cdots + T(n, n - 1) + T(n, n) \right).$$

Innen az előző összefüggéseket felhasználva kiküszöbölhetjük a $T(n, j)$ mennyiségeket:

$$T(n) = n - 1 + \frac{2}{n} \left(T(0) + T(1) + T(2) + \cdots + T(n - 1) \right).$$

Szorozzuk be mindkét oldalt n-nel:

$$nT(n) = n(n - 1) + 2(T(0) + T(1) + T(2) + \cdots + T(n - 1)),$$

majd írjuk fel ugyanezt a formulát $n - 1$ esetén is:

$$(n - 1)T(n - 1) = (n - 1)(n - 2) + 2(T(0) + T(1) + T(2) + \cdots + T(n - 2)).$$

A két formulát egymásból kivonva kapjuk, hogy

$$nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1),$$

illetve átrendezve

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)}.$$

Növeljük a jobb oldal második tagját:

$$\frac{T(n)}{n+1} < \frac{T(n-1)}{n} + \frac{2}{n}.$$

Ezt ismételten önmagába helyettesítve azt nyerjük, hogy

$$\begin{aligned}
\frac{T(n)}{n+1} &< \frac{2}{n} + \frac{T(n-1)}{n} \\
&< \frac{2}{n} + \frac{2}{n-1} + \frac{T(n-2)}{n-1} \\
&< \frac{2}{n} + \frac{2}{n-1} + \frac{2}{n-2} + \frac{T(n-3)}{n-2} < \cdots \\
&< \frac{2}{n} + \frac{2}{n-1} + \frac{2}{n-2} + \cdots + \frac{2}{2} + \frac{T(1)}{2} \\
&= \frac{2}{n} + \frac{2}{n-1} + \frac{2}{n-2} + \cdots + \frac{2}{2}.
\end{aligned}$$

Innen

$$\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-1} + \frac{1}{n} < \int_1^n \frac{dx}{x} = \ln n$$

(Riemann alsó közelítő összeg) felhasználásával

$$T(n) = O(n \log n)$$

adódik.

Megjegyezzük, hogy összehasonlítások számának várható értékére $T(n) \leqslant 1.39\, n \log n$ is igazolható.

Csavarok és anyák

Adott egy dobozban n különböző méretű csavar, egy másik dobozban pedig a hozzájuk illő anyák. Sajnos sem a csavarokat nem tudjuk egymással összehasonlítani, sem az anyákat. Azt tudjuk csak kipróbálni, hogy egy csavar külső

átmérője kisebb, nagyobb vagy egyenlő egy anya belső átmérőjénél (megpróbáljuk az anyát rácsavarni a csavarra). Egy hatékony randomizált algoritmus az egymásnak megfelelő csavarok és anyák összepárosítására a következő.

Válasszunk véletlenszerűen egy c csavart. Keressük meg a c csavarhoz illő a anyát, eközben válogassuk szét a többi anyát az a anyánál kisebbekre és nagyobbakra (a gyorsrendezés partícionáló eljárásához hasonló módon). Ez n darab összehasonlítás. Az a anya segítségével válogassuk szét a csavarokat is a c csavarnál kisebbekre és nagyobbakra. Ez további $n-1$ darab összehasonlítás. Így az összehasonlítások száma összesen $2n-1$. Ezután rekurzívan párosítsuk a c-nél kisebb csavarokat az a-nál kisebb anyákkal, illetve a c-nél nagyobb csavarokat az a-nál nagyobb anyákkal.

Az algoritmus összehasonlításokban mért költségének várható értéke itt is $O(n \log n)$.

A k-adik elem kiválasztása

Adott különböző valós számok egy $A[1:n]$ tömbje és egy $1 \leq k \leq n$ egész. Határozzuk meg a tömb k-adik elemét!

Egy olyan módszert ismertetünk, amelynél az összehasonlítások számának várható értéke $O(n)$. Hívjuk meg az $A[1:n]$ tömbre a `Partícionálás` eljárást, amelyet a gyorsrendezés is használ. Legyen j a véletlen partícionáló elem (átrendezés utáni) indexe, ezzel tér vissza a `Partícionálás` eljárás.

- Ha $j = k$, akkor kész vagyunk, a k-adik elem $A[j]$.
- Ha $j > k$, akkor a k-adik elemet az $A[1:j-1]$ résztömbben keressük tovább rekurzívan, szintén mint k-adik elemet.
- Ha $j < k$, akkor a k-adik elemet az $A[j+1:n]$ résztömbben keressük tovább rekurzívan, de most mint $(k-j)$-edik elemet, hiszen tudjuk, hogy az $A[1:j]$ résztömb minden eleme megelőzi a k-adik elemet.

Lássuk az algoritmust ezek után. Az algoritmust az $(A, 1, n, k)$ paraméterekkel kell meghívni, és a tömb k-adik elemét adja vissza.

```
Kiválaszt(A,p,r,i)
if p=r then return A[p]
q=Partícionálás(A,p,r)
s=q-p+1
if i=s
```

```
then
  return A[q]
else
  if i<s
    then return Kiválaszt(A,p,q-1,i)
    else return Kiválaszt(A,q+1,r,i-s)
```

Jelölje $T(n,k)$ az algoritmus összehasonlítások számában mért költségének várható értékét. Nyilván $T(1,1)=0$. Ha pedig $n>1$, akkor a partícionáló elem véletlen választásának köszönhetően fennáll a következő rekurzív összefüggés:

$$T(n,k) = n-1+\frac{1}{n}\left(\sum_{j=1}^{k-1} T(n-j,k-j) + \sum_{j=k+1}^{n} T(j-1,k)\right).$$

Teljes indukcióval belátjuk, hogy $T(n,k) \leqslant 4n$. Ez nyilván igaz, ha $n=1$. Nagyobb n-ekre pedig az indukciós feltevést használva

$$\begin{aligned}
T(n,k) &\leqslant n-1+\frac{1}{n}\left(\sum_{j=1}^{k-1} 4(n-j) + \sum_{j=k+1}^{n} 4(j-1)\right)\\
&= n-1+\frac{4}{n}\left(\sum_{j=1}^{k-1}(n-j) + \sum_{j=k+1}^{n}(j-1)\right)\\
&= n-1+\frac{4}{n}\left(\sum_{j=1}^{k-1}(n-j) + \sum_{j=k}^{n-1} j\right)\\
&= n-1+\frac{4}{n}\left((k-1)n - \sum_{j=1}^{k-1} j + \sum_{j=1}^{n-1} j - \sum_{j=1}^{k-1} j\right)\\
&= n-1+\frac{4}{n}\left((k-1)n - 2\sum_{j=1}^{k-1} j + \sum_{j=1}^{n-1} j\right)\\
&= n-1+\frac{4}{n}\left((k-1)n - (k-1)k + \frac{(n-1)n}{2}\right)\\
&= n-1+\frac{4}{n}\left((k-1)(n-k) + \frac{(n-1)n}{2}\right).
\end{aligned}$$

A számtani és mértani közép közötti egyenlőtlenség szerint

$$(k-1)(n-k) \leqslant \left(\frac{(k-1)+(n-k)}{2}\right)^2 = \frac{(n-1)^2}{4},$$

így

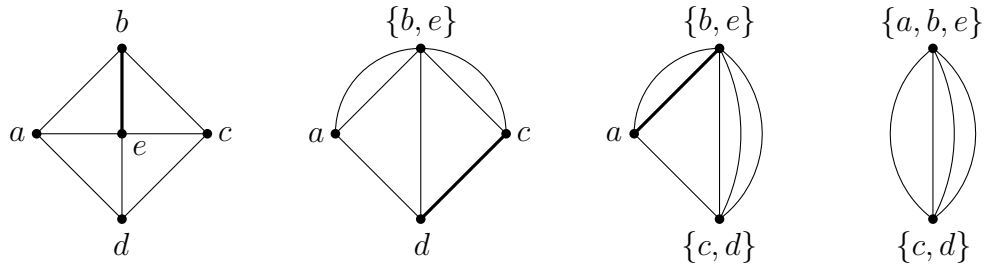
$$\begin{aligned} T(n,k) &\leqslant n-1+\frac{4}{n}\left(\frac{(n-1)^2}{4}+\frac{(n-1)n}{2}\right) \\ &= n-1+\frac{(n-1)^2}{n}+2(n-1) \\ &\leqslant 4(n-1) < 4n. \end{aligned}$$

Karger randomizált minimális vágás algoritmusa

Legyen $G = (V, E)$ egy $n \geqslant 2$ csúcsú, irányítatlan, összefüggő gráf. A gráf csúcsainak egy (A, B) partícióját, ahol A és B egyike sem üres halmaz, a gráf egy vágásának nevezzük. Egy (A, B) vágás méretén a gráf azon éleinek a számát értjük, amelyek egyik végpontja A-ban, a másik pedig B-ben van. Egy (A, B) vágást minimálisnak nevezünk, ha mérete nem haladja meg a gráf egyetlen más vágásának a méretét sem. Jegyezzük meg, hogy a G gráf egy minimális vágásának mérete megegyezik azon G-beli élek minimális számával, amelyek eltávolításával G legalább két összefüggő komponensre esik szét.

Futtassuk a G gráfon a következő algoritmust, amelyet kontrakció algoritmusnak is neveznek. A kontrakció algoritmus bemenete egy $G = (V, E)$ legalább két csúcsú, irányítatlan, összefüggő multigráf, vagyis olyan gráf, amelyben két csúcsot több (párhuzamos) él is összeköthet. Válasszuk ki véletlenszerűen (egyenletes eloszlás szerint) a G gráf egy (u, v) élét, és húzzuk össze ennek két végpontját egy új w csúcsba. Ez azt jelenti, hogy töröljük a gráfból az u és v csúcsokat az azokat összekötő élekkel együtt, majd vegyünk hozzá a gráfhoz egy új w csúcsot, amely legyen az új végpontja minden olyan élnek, amelynek eredetileg u vagy v volt az egyik végpontja (de nem mindkettő). Jelölje G' az így kapott multigráfot. Jegyezzük meg, hogy G'-ben akkor is megjelenhetnek párhuzamos élek, ha G-ben nem voltak ilyenek.

Ezután hívjuk meg a kontrakció algoritmust rekurzívan a G' gráfra. A rekurzív hívások során G' csúcsait "szupercsúcsoknak" kell tekinteni: minden w szupercsúcs megfelel a V-beli csúcsok azon $S(w)$ részhalmazának, amelyeket a w-hez vezető összehúzások során töröltünk a gráfból. A kontrakció algoritmus akkor fejeződik be, amikor G' két w_1 és w_2 szupercsúcsból áll (bizonyos számú párhuzamos éllel közöttük). Az algoritmus az $(S(w_1), S(w_2))$ vágást adja vissza.



Állítás.

- A kontrakció algoritmus a gráf egy minimális vágását adja vissza legalább $1/\binom{n}{2}$ valószínűséggel.
- Ha a kontrakció algoritmust $\binom{n}{2}\ln n$ alkalommal, egymástól függetlenül lefuttatjuk a gráfon, akkor a visszaadott vágások közül a legkisebb értékű egy minimális vágás legalább $1-1/n$ valószínűséggel.
- Egy $n \geqslant 2$ csúcsú, irányítatlan, összefüggő gráfnak legfeljebb $\binom{n}{2}$ minimális vágása létezik!

Bizonyítás.
Legyen először (A, B) a gráf egy minimális vágása. Legyen k a vágás mérete, vagyis azon élek F halmazának az elemszáma, amelyek egyik végpontja A-ban, másik végpontja pedig B-ben van. A kontrakció algoritmus pontosan akkor adja vissza az (A, B) vágást, ha egyik iterációban sem egy F-beli él két végpontját húzza össze.

Minden $1 \leqslant i \leqslant n-2$ esetén jelölje H_i azt az eseményt, hogy a kontrakció algoritmus az i-edik iterációban nem egy F-beli él két végpontját húzza össze. Célunk a $H_1 \cap H_2 \cap \cdots \cap H_{n-2}$ esemény valószínűségére alsó becslést adni. Ismert, hogy $\mathrm{Prob}[H_1 \cap H_2 \cap \cdots \cap H_{n-2}]$ felírható feltételes valószínűségek szorzataként:

$$\begin{aligned}\mathrm{Prob}[H_1 \cap H_2 \cap \cdots \cap H_{n-2}] = \mathrm{Prob}[H_1]\,\mathrm{Prob}[H_2 \mid H_1]\,\mathrm{Prob}[H_3 \mid H_1 \cap H_2] \\ \cdots \mathrm{Prob}[H_{n-2} \mid H_1 \cap H_2 \cap \cdots \cap H_{n-3}]\end{aligned}$$

Tekintsük az első iterációt. Vegyük észre, hogy G minden csúcsának a foka legalább k. Valóban, ha G valamely v csúcsának a foka kisebb lenne, mint k, akkor a $(\{v\}, V \setminus \{v\})$ vágás mérete kisebb lenne, mint k, ellentmondva az (A, B) vágás minimalitásának. Ennélfogva G éleinek száma legalább $\frac{1}{2}kn$. Ebből következik, hogy a kontrakciós algoritmus ebben az iterációban valamelyik F-beli él két végpontját legfeljebb

$$\frac{k}{\frac{1}{2}kn} = \frac{2}{n}$$

valószínűséggel húzza össze, így

$$\mathrm{Prob}[H_1] \geqslant 1 - \frac{2}{n}.$$

Legyen ezután $1 \leqslant j \leqslant n-3$, és tekintsük a $(j+1)$-edik iterációt. Most a G' gráfnak $n-j$ szupercsúcsa van. Tegyük fel, hogy a kontrakció algoritmus eddig egyetlen F-beli él két végpontját sem húzta össze. Mivel G' minden vágása egyben vágása G-nek is, ezért G' minden szupercsúcsára legalább k él illeszkedik. Ennélfogva G' éleinek száma legalább $\frac{1}{2}k(n-j)$. Ebből következik, hogy a kontrakciós algoritmus ebben az iterációban valamelyik F-beli él két végpontját legfeljebb

$$\frac{k}{\frac{1}{2}k(n-j)} = \frac{2}{n-j}$$

valószínűséggel húzza össze, így

$$\mathrm{Prob}[H_{j+1} \mid H_1 \cap H_2 \cap \cdots \cap H_j] \geqslant 1 - \frac{2}{n-j}.$$

Mindezeket felhasználva

$$\begin{aligned}
&\mathrm{Prob}[H_1 \cap H_2 \cap \cdots \cap H_{n-2}] \\
&\qquad = \mathrm{Prob}[H_1]\,\mathrm{Prob}[H_2 \mid H_1]\,\mathrm{Prob}[H_3 \mid H_1 \cap H_2] \cdots \\
&\qquad\qquad \cdots \mathrm{Prob}[H_{n-2} \mid H_1 \cap H_2 \cap \cdots \cap H_{n-3}] \\
&\qquad \geqslant \left(1 - \frac{2}{n}\right)\left(1 - \frac{2}{n-1}\right)\left(1 - \frac{2}{n-2}\right) \cdots \left(1 - \frac{2}{3}\right) \\
&\qquad = \left(\frac{n-2}{n}\right)\left(\frac{n-3}{n-1}\right)\left(\frac{n-4}{n-2}\right) \cdots \left(\frac{1}{3}\right) \\
&\qquad = \frac{2}{n(n-1)} = \binom{n}{2}^{-1}.
\end{aligned}$$

adódik.

Az előző gondolatmenettel összhangban a kontrakció algoritmus legfeljebb $1 - 1/\binom{n}{2}$ valószínűséggel ad vissza olyan vágást, amely nem minimális. Így annak a valószínűsége, hogy a kontrakció algoritmust $\binom{n}{2}\ln n$ alkalommal, egymástól függetlenül lefuttatva minden esetben olyan vágást kapunk, amely nem minimális, legfeljebb

$$\left(1 - 1/\binom{n}{2}\right)^{\binom{n}{2}\ln n} \leqslant e^{-\ln n} = 1/n.$$

Itt felhasználtuk, hogy tetszőleges $n \geqslant 2$ egész számra

$$\left(1 - 1/\binom{n}{2}\right)^{\binom{n}{2}} \leqslant e^{-1}.$$

Legyen végül G egy $n \geqslant 2$ csúcsú, irányítatlan, összefüggő gráf, és legyenek $C_1, C_2, \ldots, C_r$ a gráf minimális vágásai. Minden $1 \leqslant i \leqslant r$ esetén jelölje M_i azt az eseményt, hogy a kontrakció algoritmus a C_i minimális vágást adja vissza, továbbá legyen $M = M_1 \cup M_2 \cup \cdots \cup M_r$.

Vegyük észre, hogy az (A) pont bizonyításánál valójában azt mutattuk meg, hogy $\mathrm{Prob}[M_i] \geqslant 1/\binom{n}{2}$ minden $1 \leqslant i \leqslant r$ esetén. Mivel tetszőleges $1 \leqslant i < j \leqslant r$ esetén az M_i és M_j események nyilvánvaló módon diszjunktak, így

$$1 \geqslant \mathrm{Prob}[M] = \mathrm{Prob}\left[\bigcup_{i=1}^{r} M_i\right] = \sum_{i=1}^{r} \mathrm{Prob}[M_i] \geqslant r/\binom{n}{2},$$

ahonnan $r \leqslant \binom{n}{2}$ adódik.

Jegyezzük meg, hogy ez a korlát éles: egy n csúcsú körnek éppen $\binom{n}{2}$ minimális vágása van, ezek mind 2 értékűek.